第5章

统一建模语言

陈龙彪
longbiaochen@xmu.edu.cn

厦门大学信息学院
2019年3月

教学内容

- UML概述
- UML需求建模
- UML静态建模
- UML动态建模
- UML架构建模



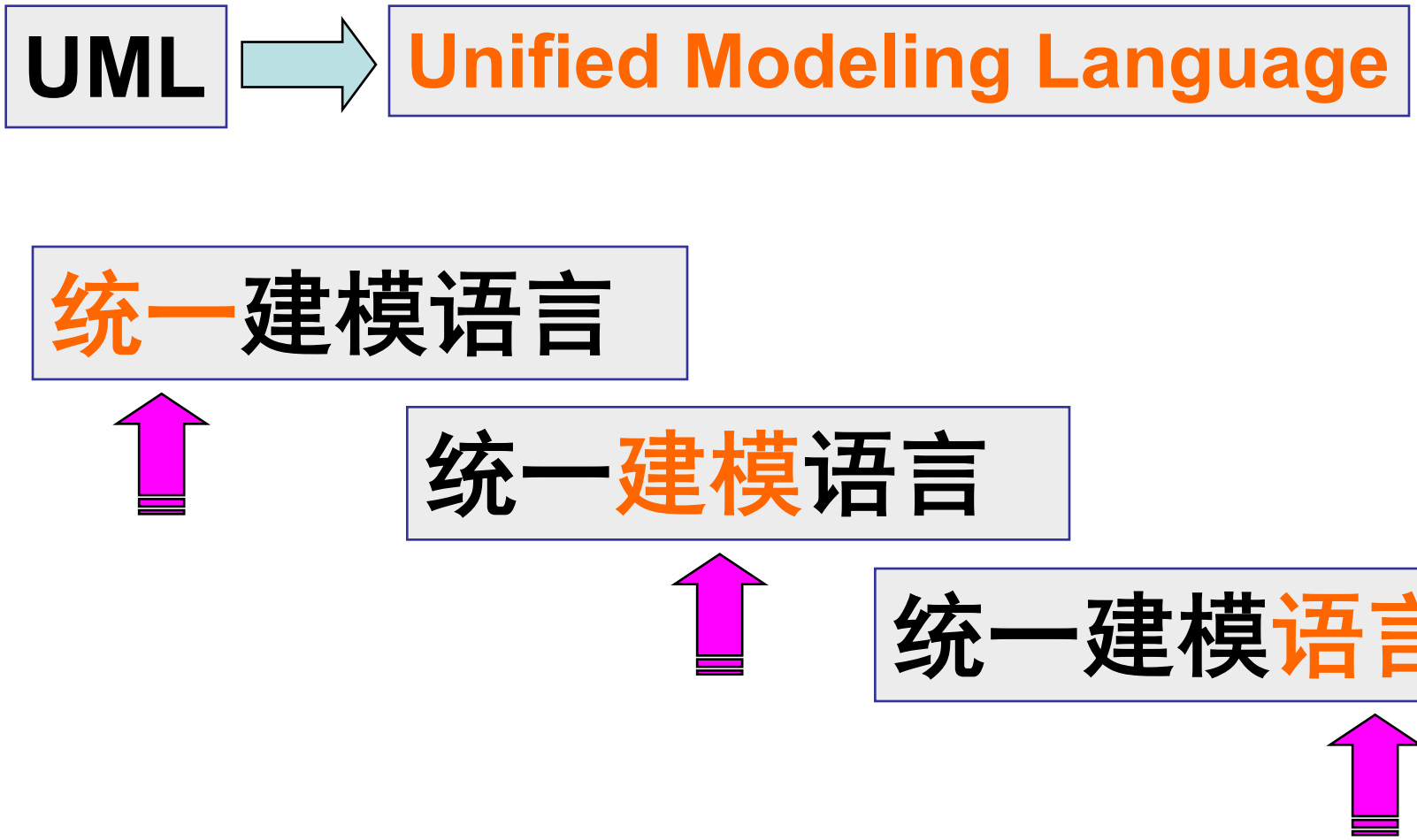
第一部分

UML概述

UML诞生记

- 1995年，Grady Booch、James Rumbaugh和Ivar Jacobson在Rational公司创立UML
- 1997年11月,在Ivar Jacoboson、Grady Booch以及James Rumbaugh的共同努力下，UML1.1版本提交给OMG （对象管理组织）并获得通过，成为业界标准的建模语言
- 1998年秋季，UML1.3版本发布
- 2003年6月，OMG技术会议上UML 2.0获得正式通过

UML名词解释



UML三友







Ivar Jacoboson

Grady Booch

James Rumbaugh

Object-Oriented Software Engineering(OOSE)

Booch开发方法

Object Modeling Technique(OMT)

UML

为什么建模

- 在现代化工程中，人们要相互沟通和合作，必须使用标准的工业化设计语言，用这些语言来对开发产品进行建模
- 在实际软件开发过程中，现实世界的复杂性以致用户经常不能精确地理解和描述将要开发的软件系统，建模可以使用户更好地理解需要开发的系统
- 建模是开发优秀软件的所有活动中最核心的部分，其目的是把所要设计的结构和系统的行为联系起来，并对系统的体系结构进行可视化和控制
- 建模过程把复杂的问题分解成为易于理解的小问题， 以达到问题的求解

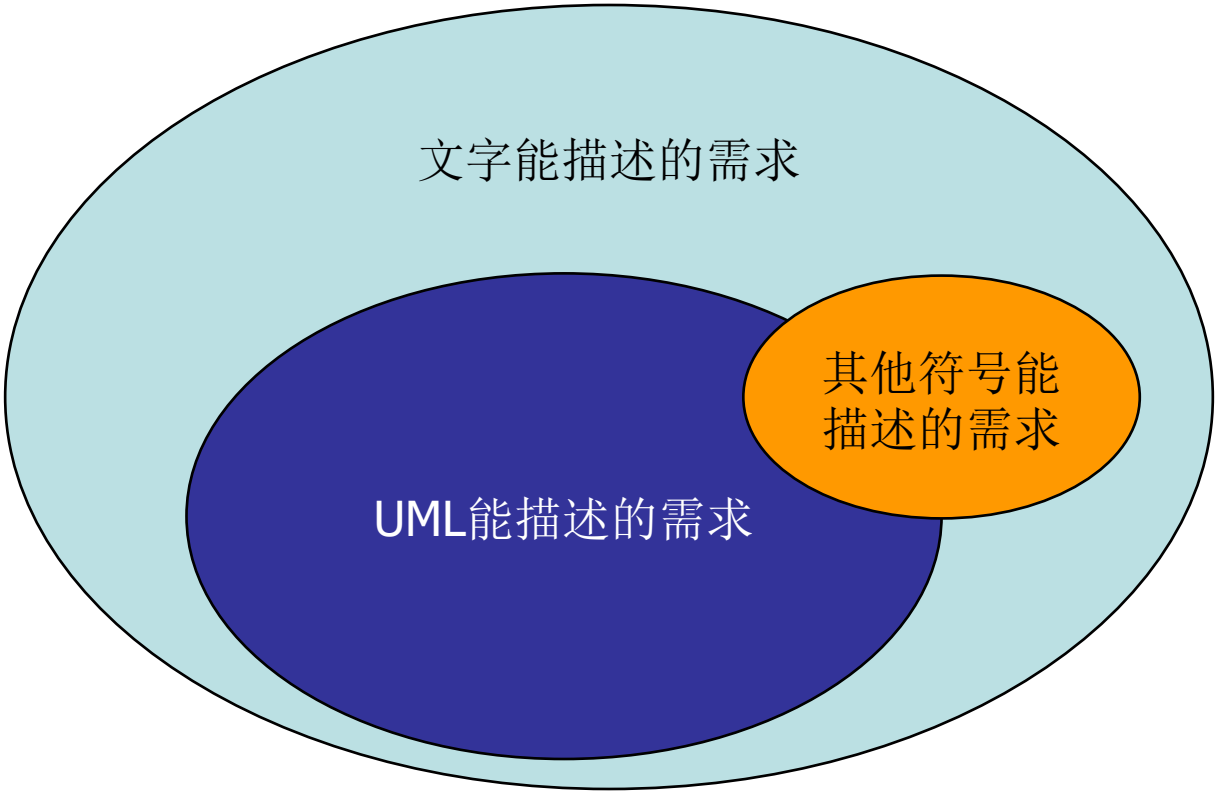
UML

- UML是用于描绘软件蓝图的标准语言
- UML是一种语言，也就意味着它有属于自己的标准表达规则。它不是一种类似Java、C++的编程语言，而是一种分析设计语言，也就是一种建模语言
- UML是由图形符号表达的建模语言



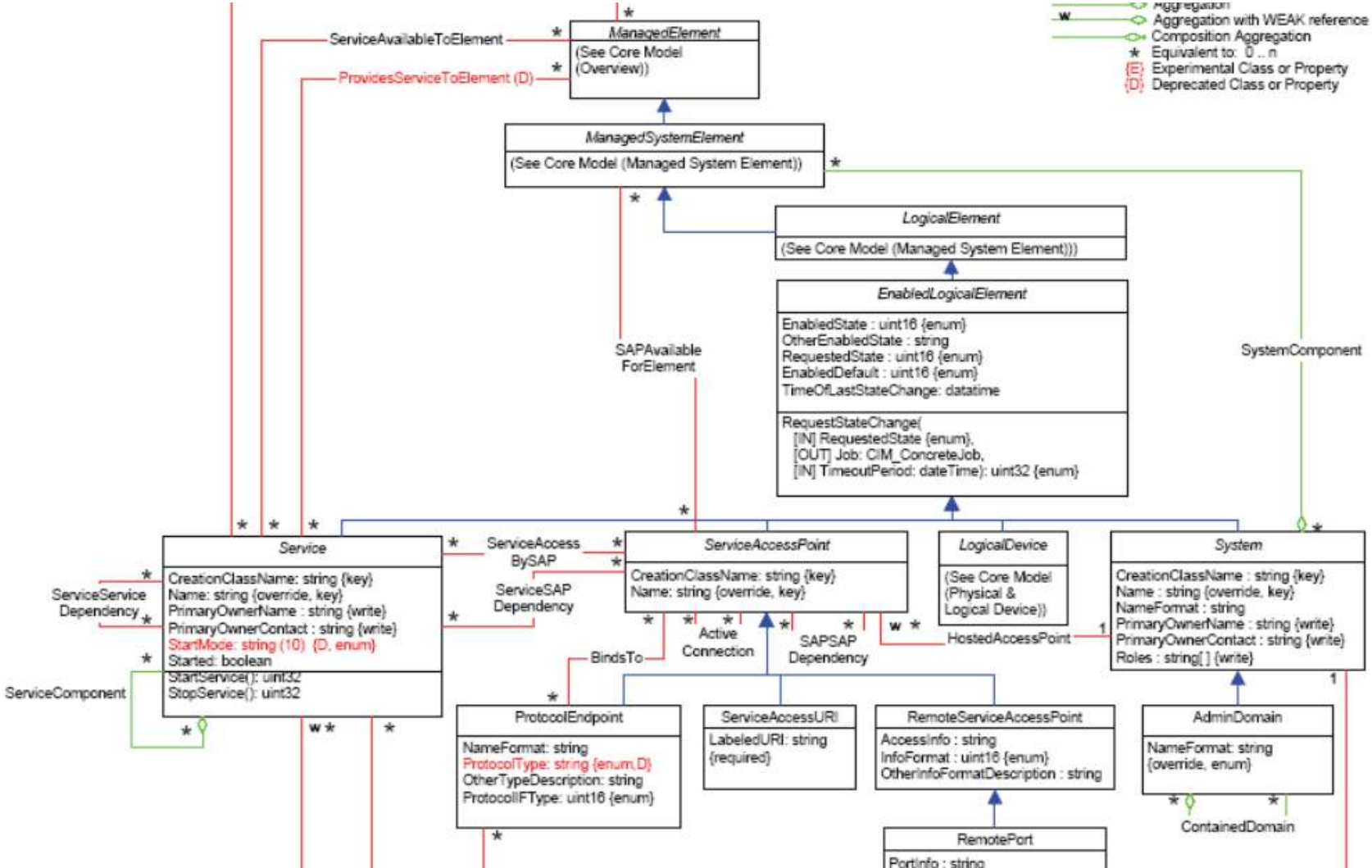
Customer
- cNo : String - cName : String - cTel : String - cAddress : String - cEmail : String
+ <<Getter>> getCNo () : String + <<Setter>> setCNo (String newCNo) : void + <<Getter>> getCName () : String + <<Setter>> setCName (String newCName) : void

UML能做什么



UML能做什么

- 制造业

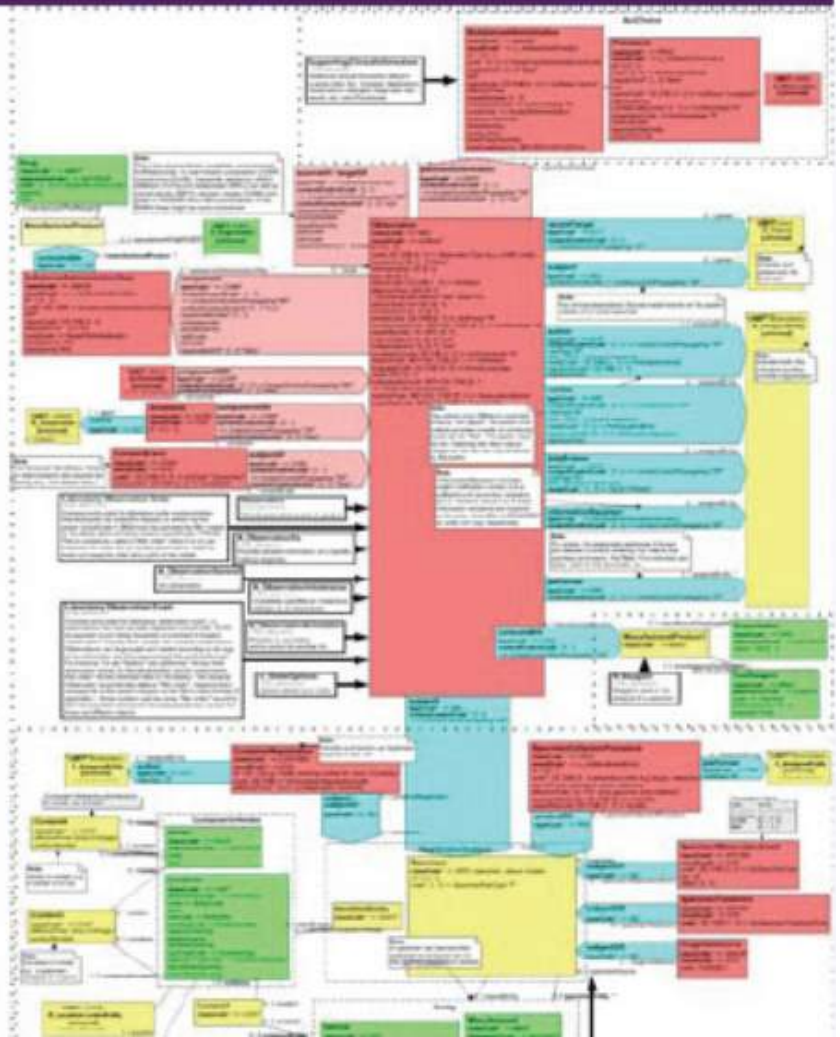

Aggregation with WEAK reference
Composition Aggregation
Equivalent to: 0 .. n
{E} Experimental Class or Property
{D} Deprecated Class or Property
ManagedElement
(See Core Model (Overview))
ServiceAvailableToElement
ProvidesServiceToElement (D)
ManagedSystemElement
(See Core Model (Managed System Element))
LogicalElement
(See Core Model (Managed System Element)))
EnabledLogicalElement
EnabledState : uint16 {enum}
OtherEnabledState : string
RequestedState : uint16 {enum}
EnabledDefault : uint16 {enum}
TimeOfLastStateChange: datatime
RequestStateChange(
[IN] RequestedState {enum},
[OUT] Job: CIM_ConcreteJob,
[IN] TimeoutPeriod: dateTime): uint32 {enum}
SAPAvailable ForElement
SystemComponent
Service
CreationClassName: string {key}
Name: string {override, key}
PrimaryOwnerName : string {write}
PrimaryOwnerContact : string {write}
StartMode: string (10) {D, enum}
Started: boolean
StartService(): uint32
StopService(): uint32
ServiceService Dependency
ServiceComponent
ServiceAccess BySAP
ServiceSAP Dependency
ServiceAccessPoint
CreationClassName: string {key}
Name: string {override, key}
Active Connection
SAPSAP Dependency
BindsTo
LogicalDevice
(See Core Model (Physical & Logical Device))
HostedAccessPoint
System
CreationClassName : string {key}
Name : string {override, key}
NameFormat : string
PrimaryOwnerName : string {write}
PrimaryOwnerContact : string {write}
Roles : string[ ] {write}
ProtocolEndpoint
NameFormat: string
ProtocolType: string {enum,D}
OtherTypeDescription: string
ProtocolIFType: uint16 {enum}
ServiceAccessURI
LabeledURI: string {required}
RemoteServiceAccessPoint
AccessInfo : string
InfoFormat : uint16 {enum}
OtherInfoFormatDescription : string
AdminDomain
NameFormat: string {override, enum}
ContainedDomain
RemotePort
PortInfo : string

UML能做什么

- 医疗卫生

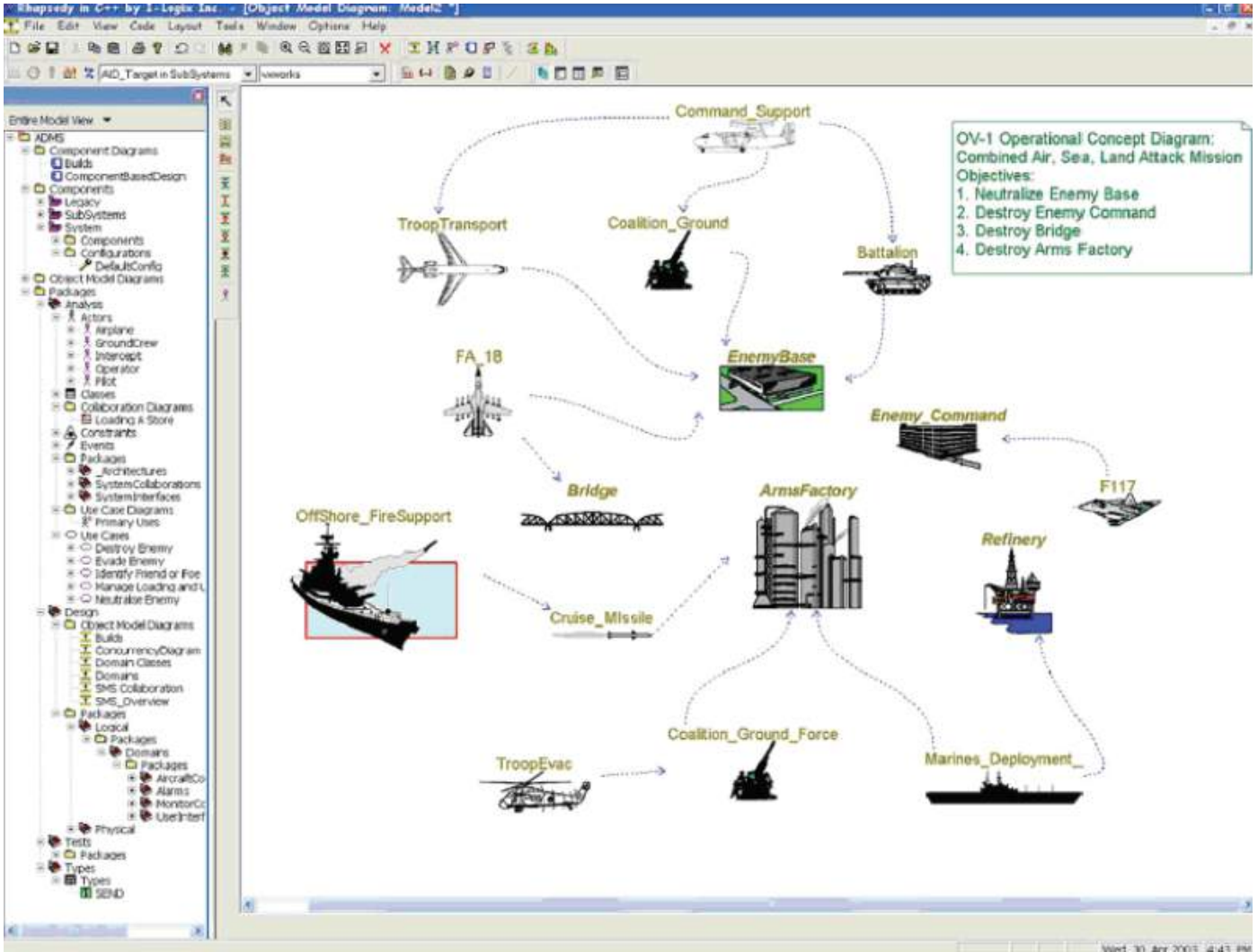
HL7 Version 3

- Model Driven Architecture
- Entire Healthcare Domain
- ANSI and International Standard
- Focus on vocabulary and content
- Focus on UML compliance
- Needs implementation testing, enhancement, services capability



UML能做什么

- 军事领域



要学UML吗？

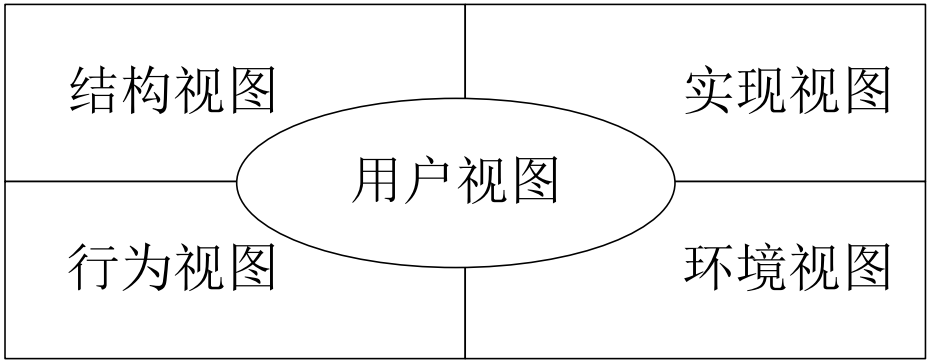
- 你应该使用UML吗？是！旧的面向对象符号正在快速消失，新的书、文章将全部采用UML作为符号。如果你正要开始使用建模符号，你就该直接学习UML。

——*Martin Fowler*



UML 五大视图

- **用户视图：**以用户的观点表示系统的目标，它是所有视图的核心，该视图描述系统的需求
- **结构视图：**表示系统的静态行为，描述系统的静态元素如包、类与对象，以及它们之间的关系
- **行为视图：**表示系统的动态行为，描述系统的组成元素如对象在系统运行时的交互关系
- **实现视图：**表示系统中逻辑元素的分布，描述系统中物理文件以及它们之间的关系
- **环境视图：**表示系统中物理元素的分布，描述系统中硬件设备以及它们之间的关系



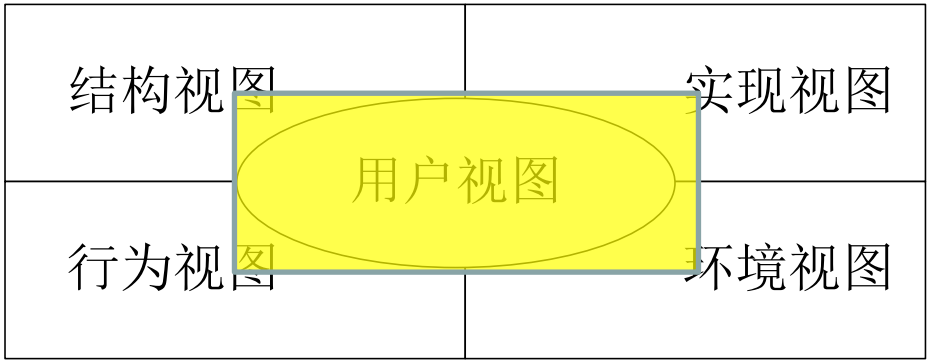
UML 2.0的十三种图

- 用例图(用户视图)
- 类图(结构视图)
- 对象图(结构视图)
- 包图 (结构视图，UML2.0)
- 组合结构图 (结构视图，UML2.0)
- 顺序图(行为视图)
- 通信图（行为视图，UML1.0 协作图）
- 定时图 (行为视图，UML2.0)
- 状态图(行为视图)
- 活动图(行为视图)
- 交互概览图 (行为视图，UML2.0)
- 组件图(实现视图)
- 部署图(环境视图)



第二部分

UML需求建模



用例建模

- 用例建模（Use Case Modeling）使用用例的方法来描述系统功能需求。主要包括以下两部分内容：
  - 用例图(Use Case Diagram)
  - 用例描述文档 (Use Case Specification)

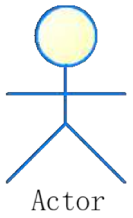
用例建模——步骤

- 识别执行者
- 识别用例
- 绘制用例图
- 书写用例文档
- 检查用例模型

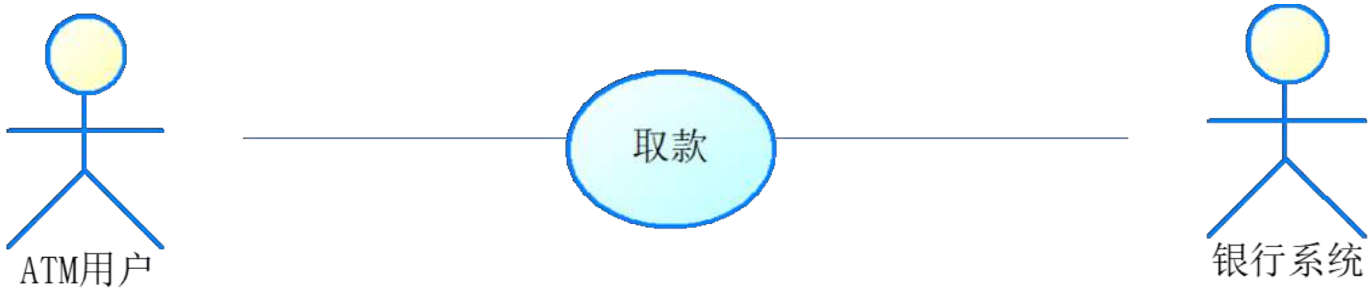
用例建模——步骤

- 识别执行者
- 识别用例
- 绘制用例图
- 书写用例文档
- 检查用例模型

识别执行者

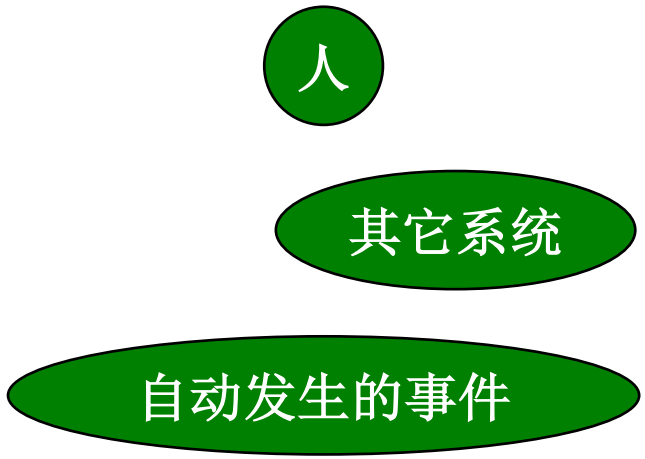


- 执行者（Actor）
  - 在系统之外，透过系统边界与系统进行有意义交互的任何事物
  - 引入执行者的目的：帮助确定系统边界



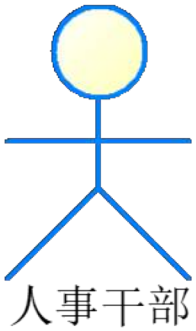
识别执行者

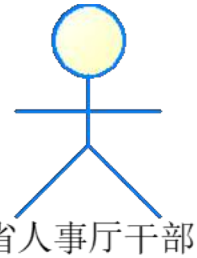
- 如何识别执行者
  - 谁使用系统？
  - 谁改变系统的数据？
  - 谁从系统获取信息？
  - 谁需要系统的支持以完成日常工作任务？
  - 谁负责维护、管理并保持系统正常运行？
  - 系统需要和哪些外部系统交互？
  - 有没有自动发生的事件？



识别执行者

- 哪个是正确的执行者？？





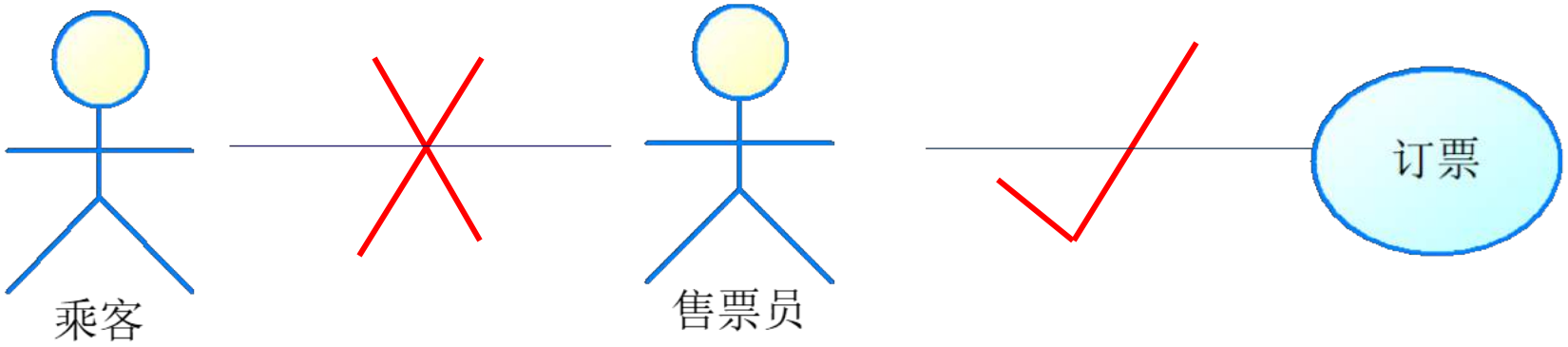




都对

不丢用例就行（慢慢整理）

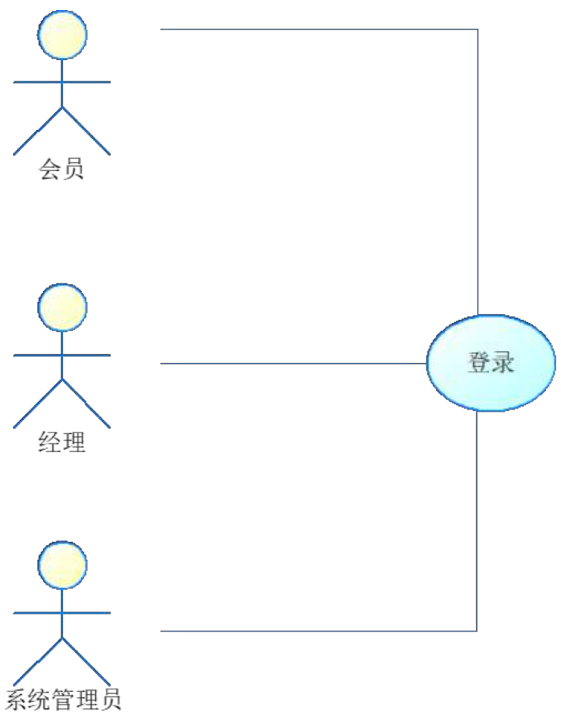
识别执行者

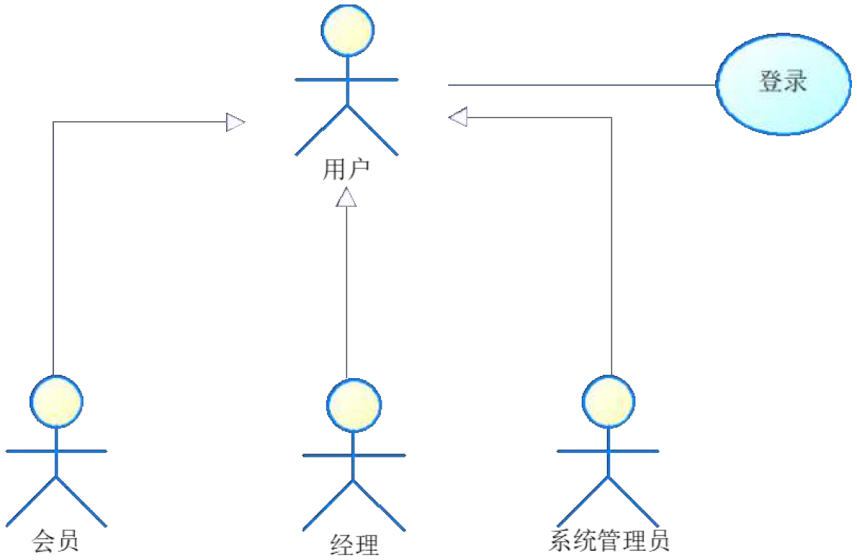






识别执行者





用例建模——步骤

- 识别执行者
- 识别用例
- 绘制用例图
- 书写用例文档
- 检查用例模型

识别用例



- 用例（Use Case）
  - 在系统中执行的一系列动作，这些动作将生成特定的、可见的结果
  - 一个用例定义一组用例实例



识别用例

- 形式检查

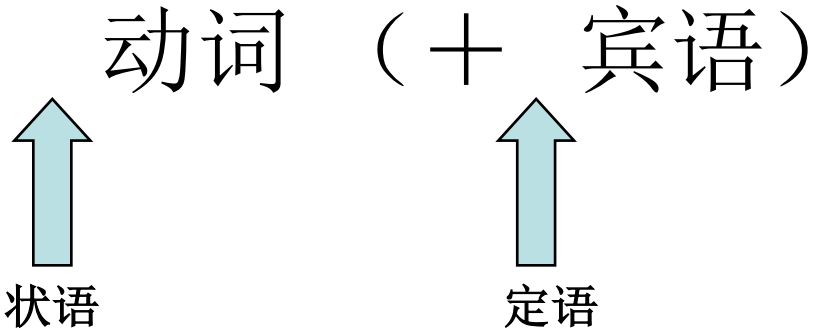
【执行者】使用系统来【用例】

识别用例

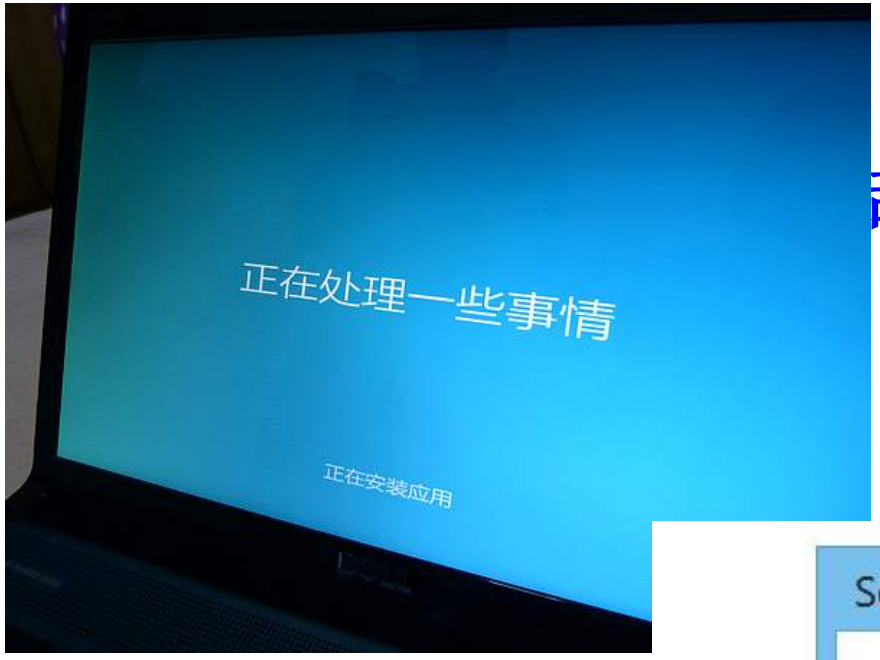
- 用例识别要点
  - 用例的命名
  - 有意义的目标
  - 使用业务语言
  - 面向用户观点
  - 注意用例粒度

识别用例

- **用例的命名**

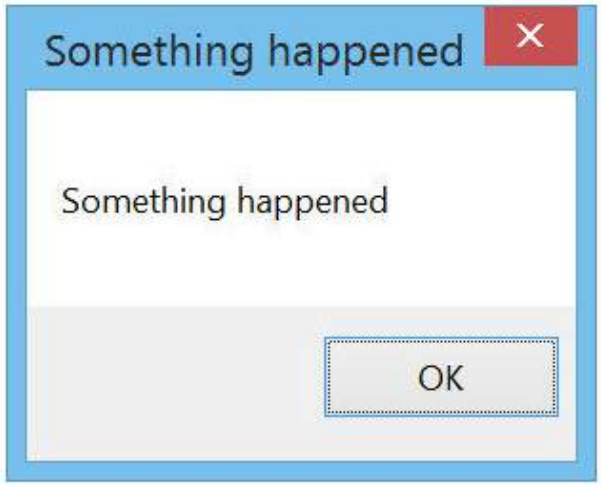


识别用例



词弱名词

加载、重复



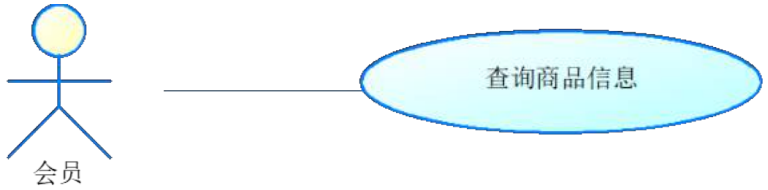
会掩盖真

识别用例

- 有意义的目标



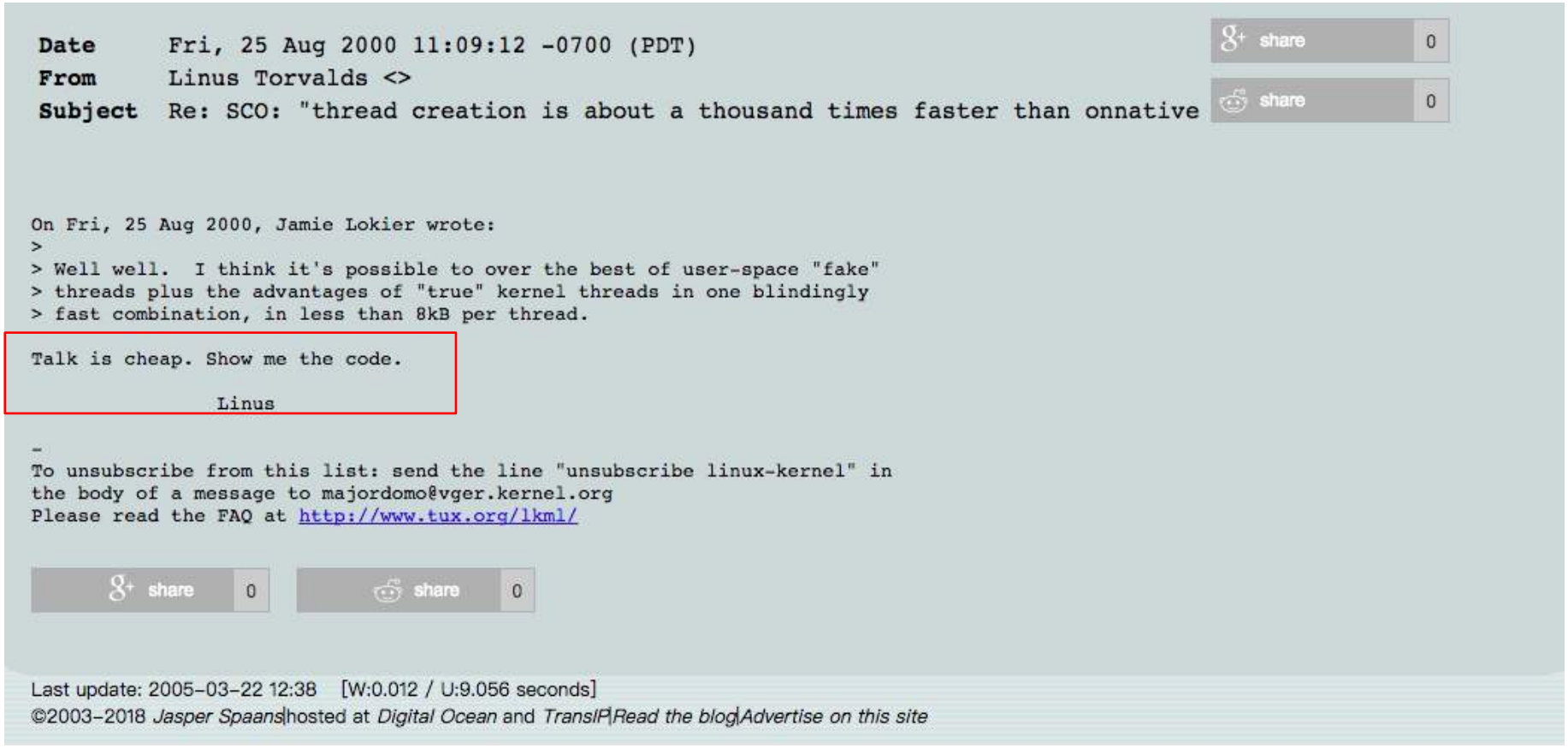
错



对

识别用例

- 使用业务语言而非技术语言



Date Fri, 25 Aug 2000 11:09:12 -0700 (PDT)
From Linus Torvalds <>
Subject Re: SCO: "thread creation is about a thousand times faster than onnative

share 0 share 0

On Fri, 25 Aug 2000, Jamie Lokier wrote:
>
> Well well. I think it's possible to over the best of user-space "fake"
> threads plus the advantages of "true" kernel threads in one blindingly
> fast combination, in less than 8kB per thread.

Talk is cheap. Show me the code.

Linus

-
To unsubscribe from this list: send the line "unsubscribe linux-kernel" in
the body of a message to majordomo@vger.kernel.org
Please read the FAQ at http://www.tux.org/lkml/

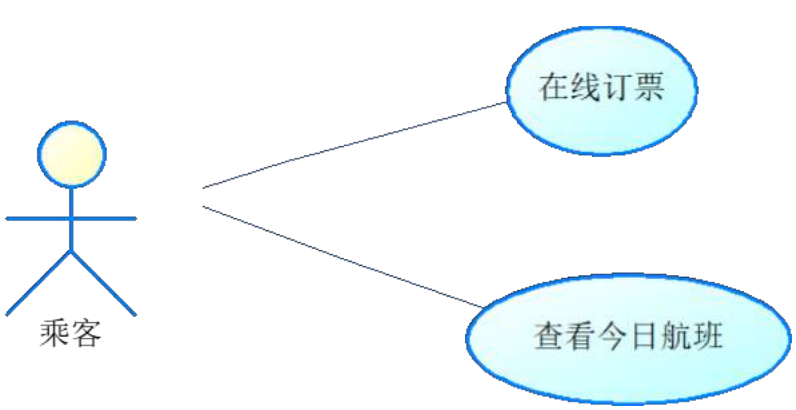
share 0 share 0

Last update: 2005-03-22 12:38 [W:0.012 / U:9.056 seconds]
©2003-2018 *Jasper Spaans*|hosted at *Digital Ocean* and *TransIP*|*Read the blog*|*Advertise on this site*

识别用例

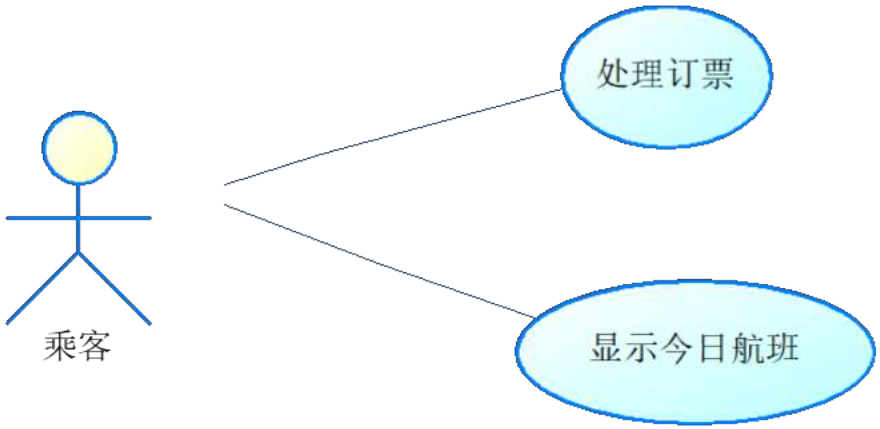
- 面向用户观点而非系统观点

用户观点



对

系统观点

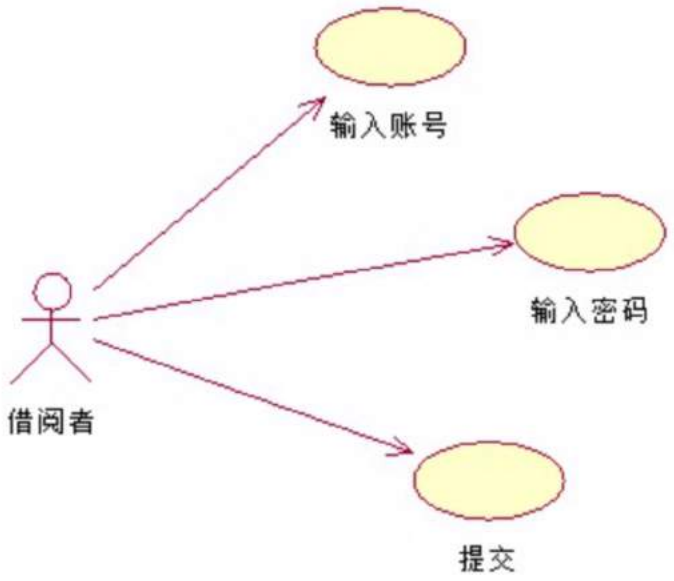


错

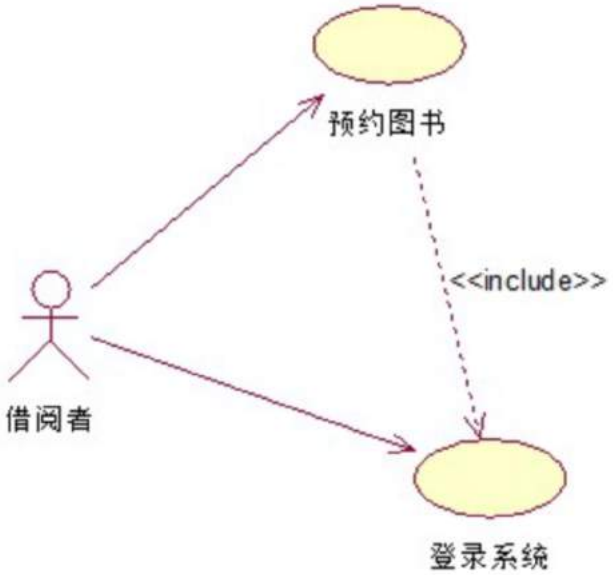
识别用例

➢ 注意用例粒度

原则：用例要有路径，路径要有步骤。而这一切都是可观测的



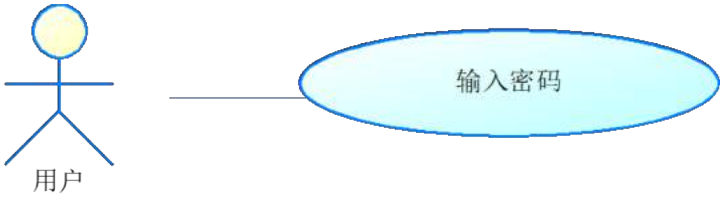
图（1）粒度过细



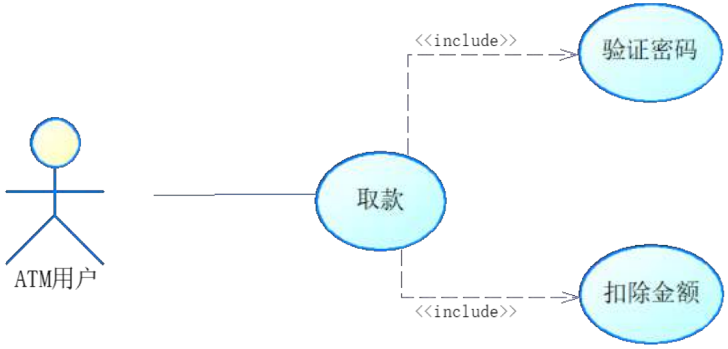
图（2）粒度适中的用例

识别用例

- **用例粒度常见错误：把步骤当作用例**
  - 把执行者动作当作用例

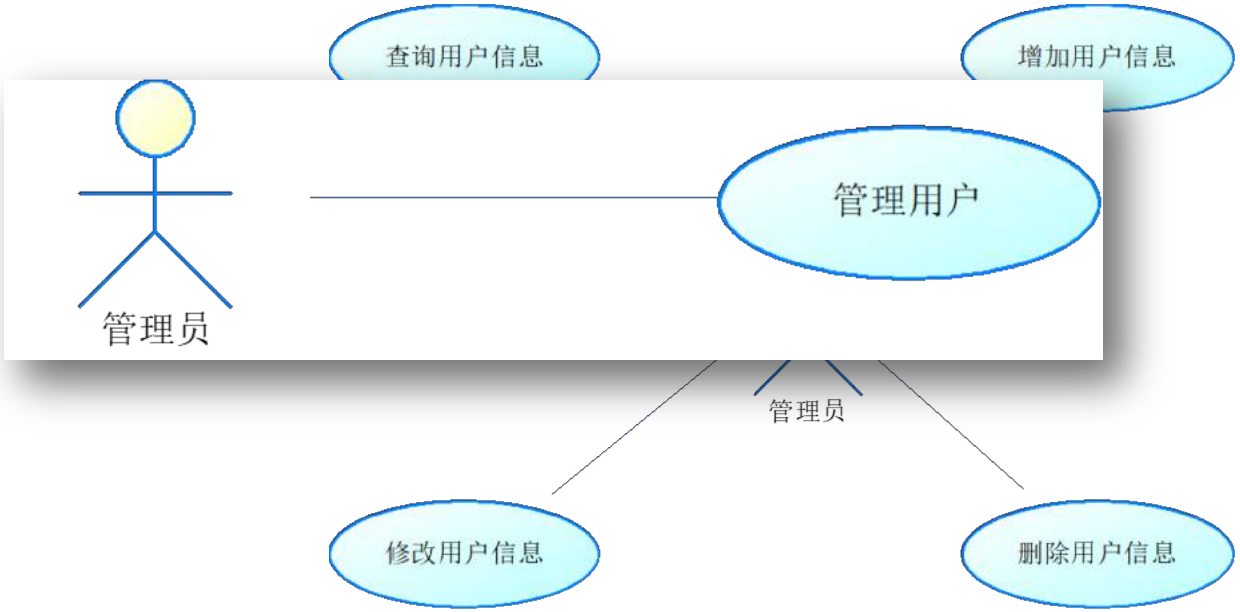


  - 把系统活动当作用例



识别用例

- **用例粒度常见错误：把步骤当作用例**
  - 警惕CRUD泛滥



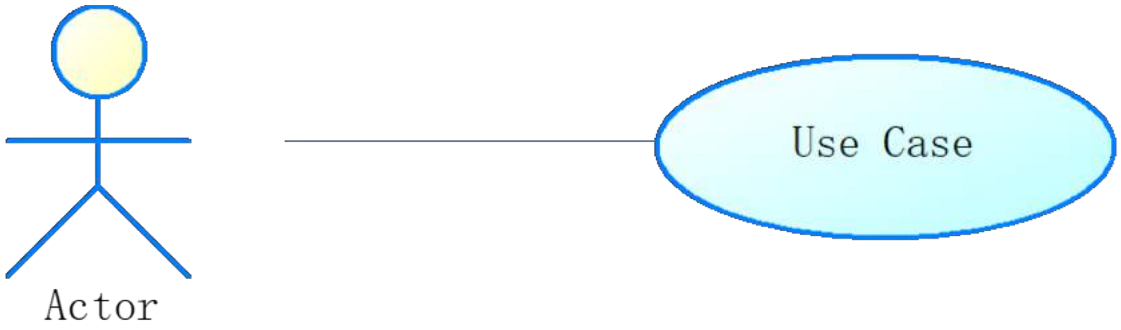
CRUD：新增、读取、修改、删除

用例建模——步骤

- 识别执行者
- 识别用例
- 绘制用例图
- 书写用例文档
- 检查用例模型

绘制用例图

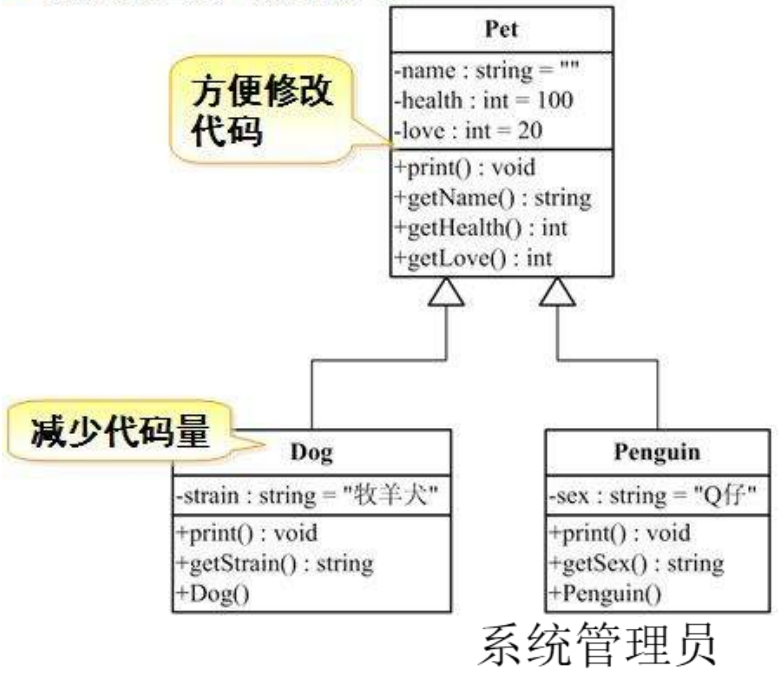
- 执行者与用例之间的关系：关联关系
  - 在用例图中，执行者和用例之间进行交互，相互之间的关系用一根直线来表示，称为关联关系(Association)或通信关系(Communication)



绘制用例图

- 执行者之间的关系：泛
  - 泛化(Generalization)关

- 使用继承优化后：



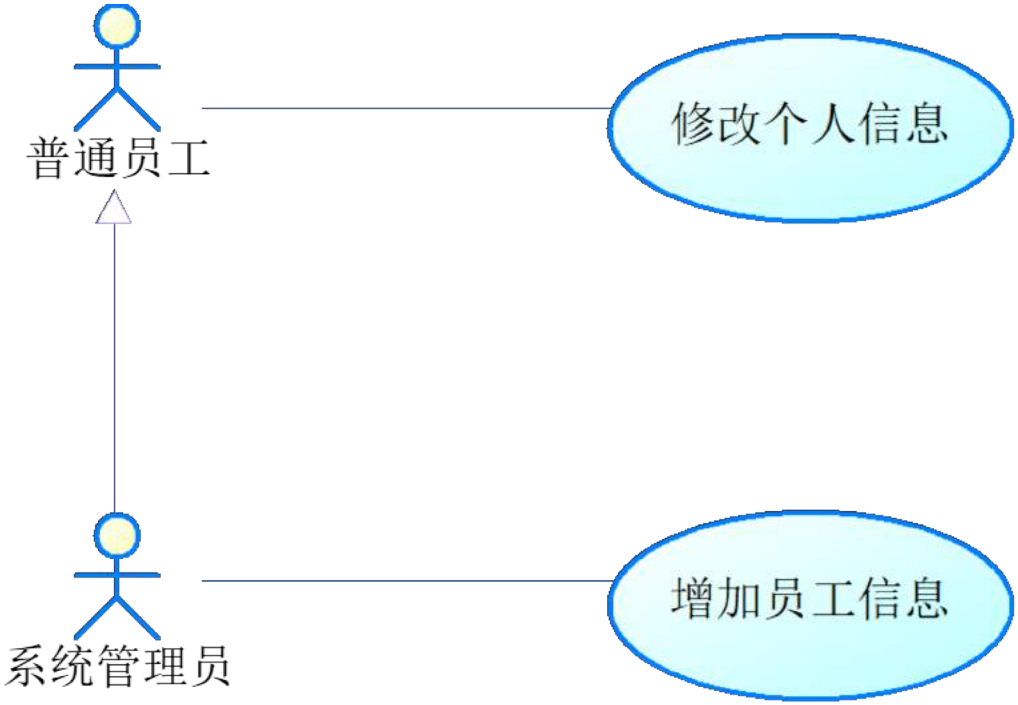
系统管理员



生物进化谱系树

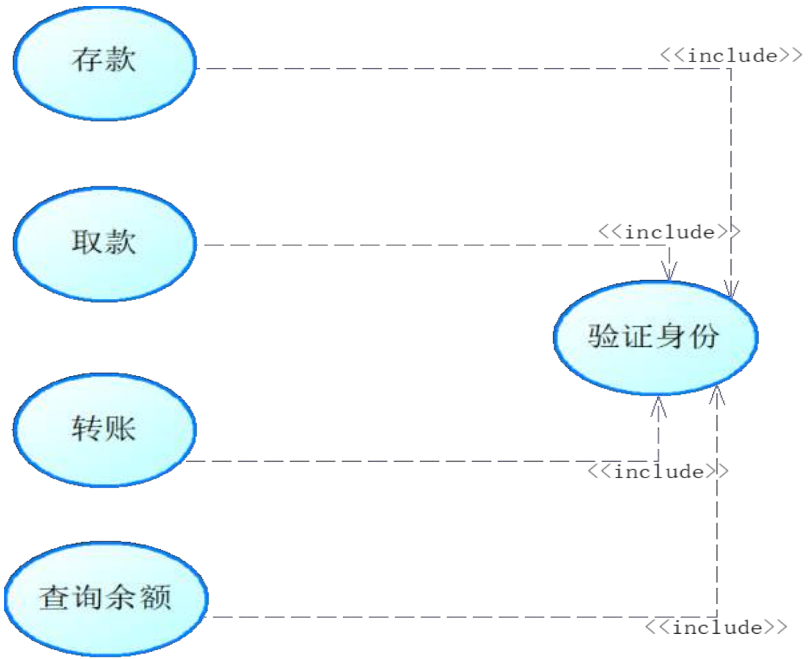
绘制用例图

- 执行者之间的关系：泛化与用例



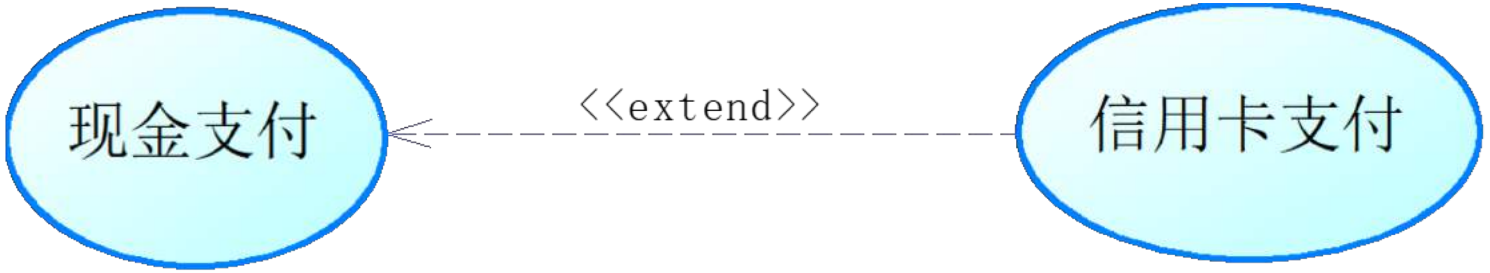
绘制用例图

- 用例之间的关系：包含关系
  - 描述在多个用例中都有的公共行为
  - 由用例A指向用例B，表示用例A中使用了用例B中的行为或功能
  - 包含关系通过<<include>>构造型（衍型）来表示



绘制用例图

- 用例之间的关系：扩展关系
  - 扩展用例可以在基础用例之上添加新的行为
  - 基础用例（Base）中定义有一至多个已命名的扩展点，扩展关系将扩展用例（Extension）的事件流在一定的条件下按照相应的扩展点插入基础用例中
  - 扩展关系是通过应用<<extend>>构造型（衍型）来表示



用例建模——步骤

- 识别执行者
- 识别用例
- 绘制用例图
- 书写用例文档
- 检查用例模型

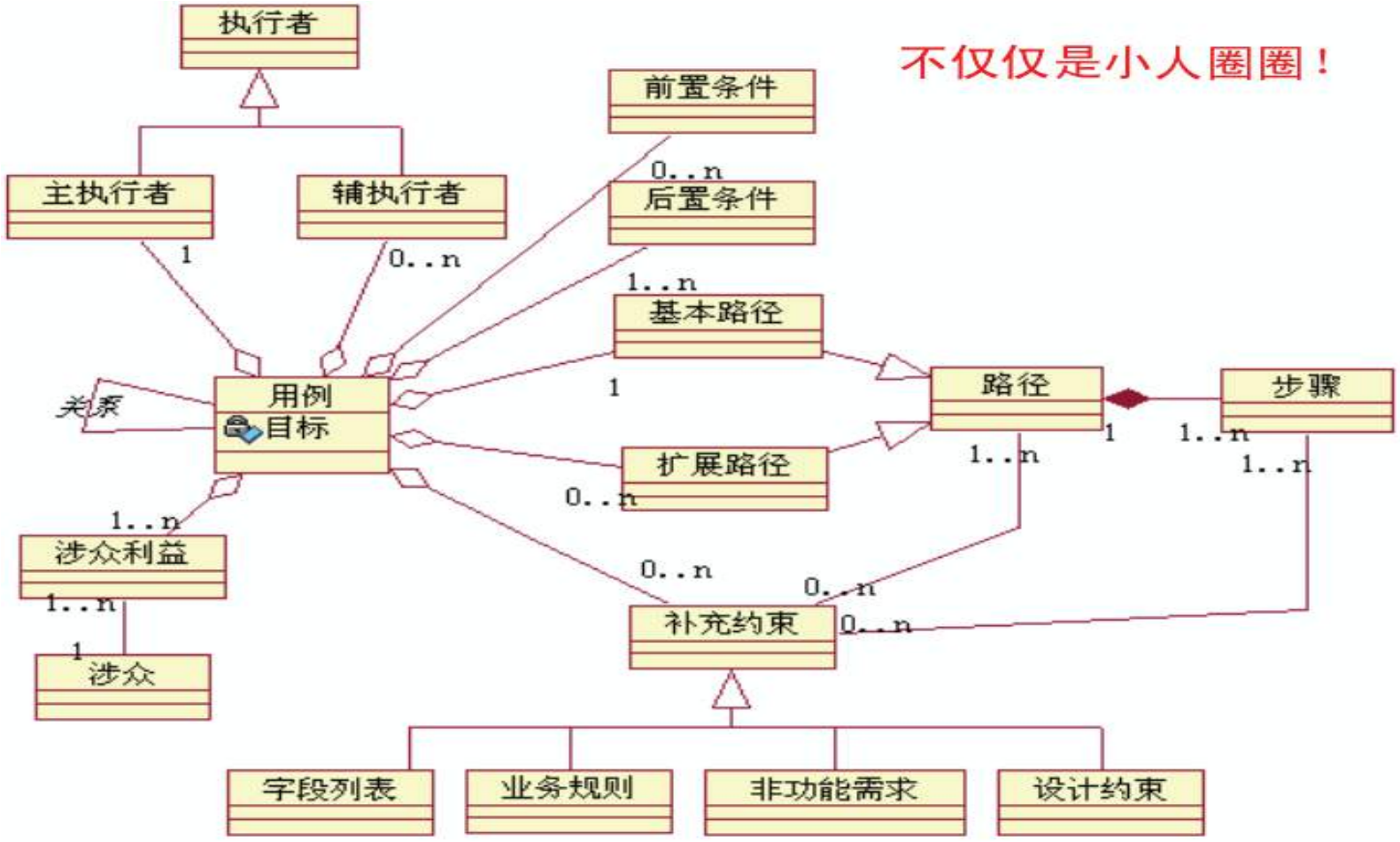
书写用例文档

- 注意要点
  - 用例是文本文档，而非图形
  - 用例建模主要是编写文本的活动，而非制图
  - 用例不是面向对象的，编写用例时也不会进行OO分析
  - 用例是经典OOA/D的关键需求输入

书写用例文档

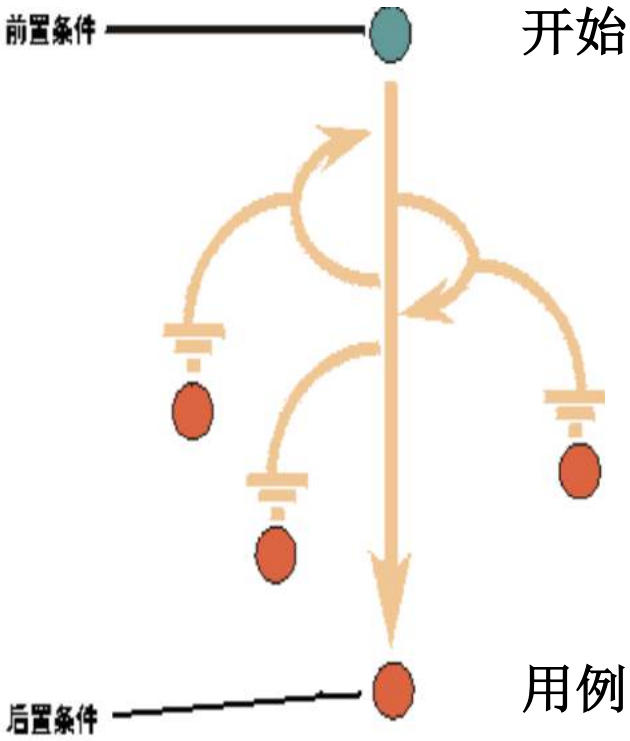
- 用例的内容
  - 用例编号
  - 用例名
  - 执行者
  - 前置条件
  - 后置条件
  - 涉众利益
  - 基本路径
    - 1…..××××
    - 2……××××
    - 3…..××××
  - 扩展路径
    - 2a.××××：
      - 2a1….×××××
  - 字段列表
  - 业务规则
  - 非功能需求
  - 设计约束

书写用例文档


执行者
主执行者
辅执行者
1
0..n
前置条件
0..n
后置条件
1..n
基本路径
1
用例
目标
关系
路径
1
1..n
步骤
1..n
1..n
扩展路径
0..n
1..n
涉众利益
1..n
1
涉众
0..n
0..n
0..n
补充约束
字段列表
业务规则
非功能需求
设计约束
不仅仅是小人圈圈！

书写用例文档

- 前置、后置条件

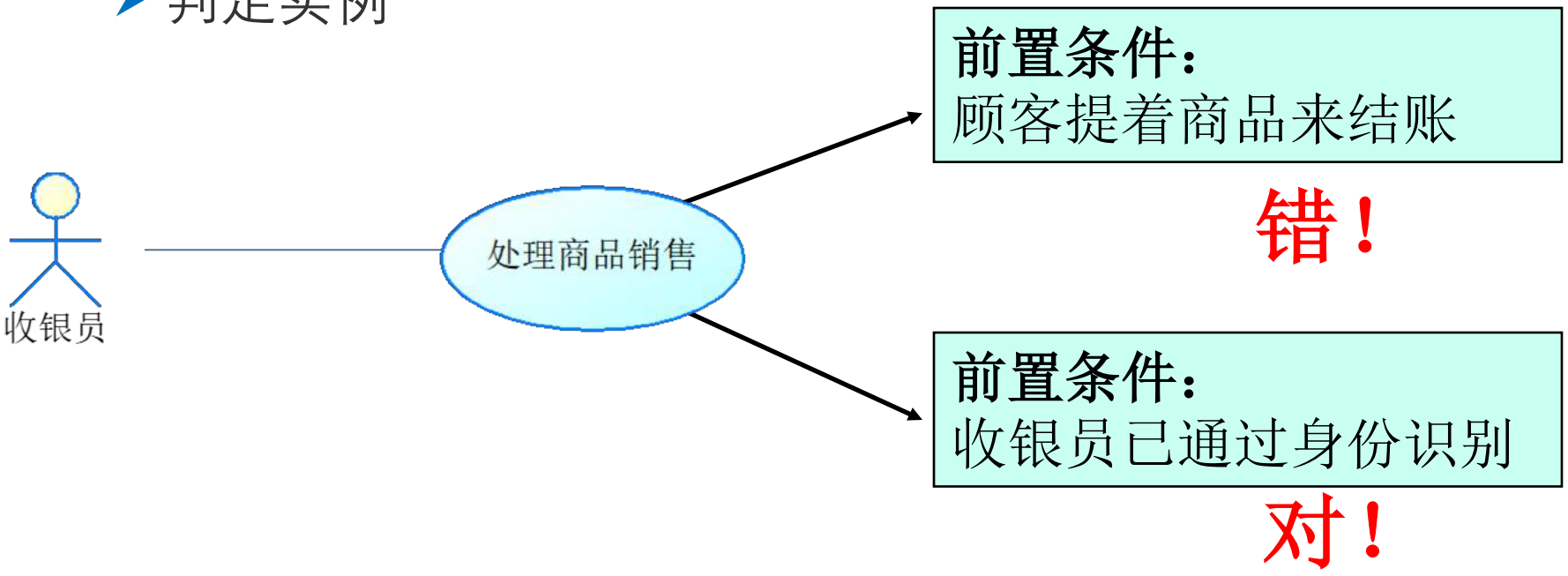


开始用例前所必需的系统及其环境的状态

用例成功结束后系统应该具备的状态

书写用例文档

- 前置、后置条件
  - 判定实例

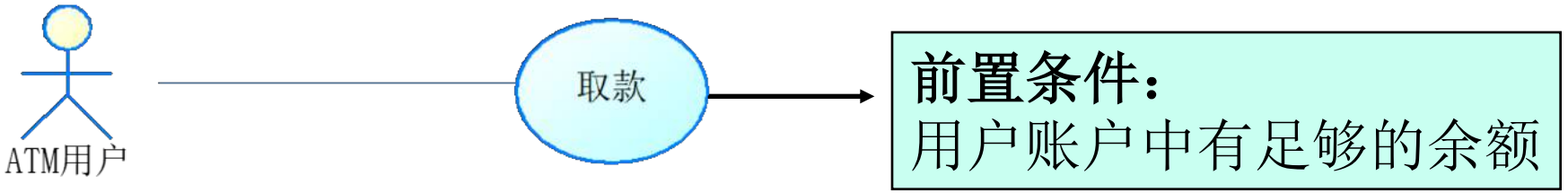


必须是系统能检测到的

书写用例文档

- 前置、后置条件

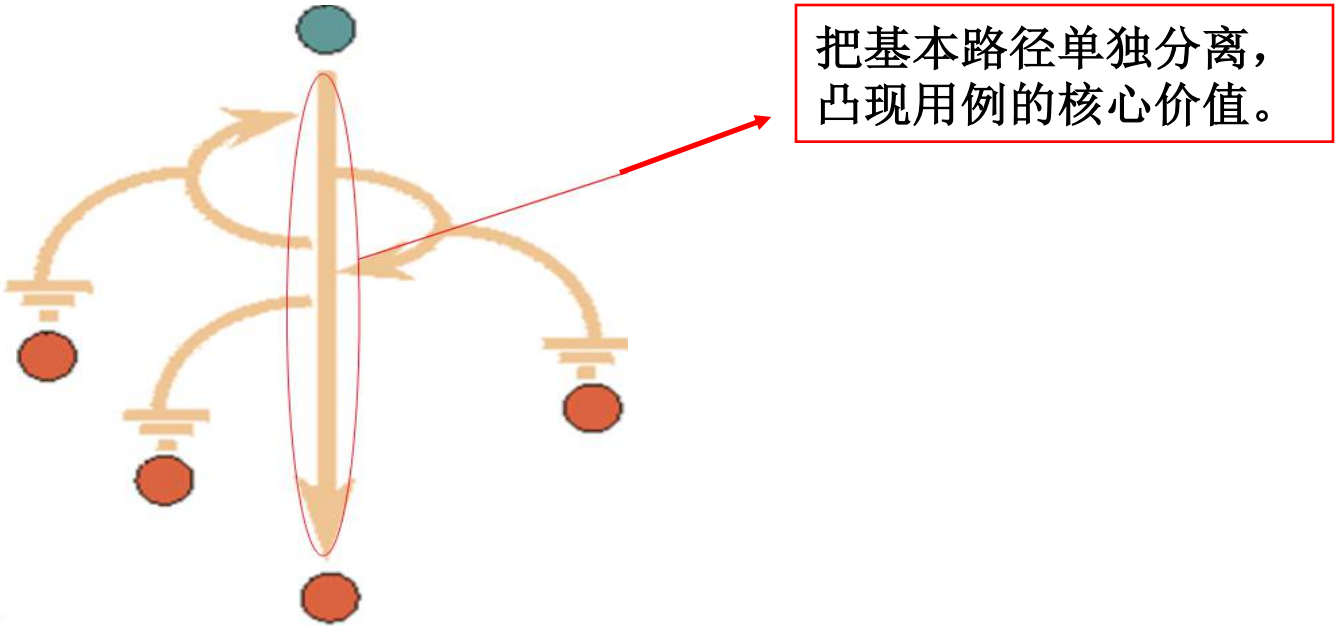
错！



前置条件必须是系统在用例开始前能检测到的

书写用例文档

- 基本路径
  - 通过控制流程图分离出主要的用例



核心的核心：客户最想看到、最关心的路径

书写用例文档

- 基本路径书写准则
  - 只书写“可观测”的（说人话）
  - 使用主动语句
  - 句子必须以执行者或系统作为主语
  - 每一句都要朝目标迈进
  - 分支和循环
  - 不要涉及界面细节

书写用例文档

- 基本路径

错

系统通过ADO建立数据库连接，传送SQL查询语句，从“零件”表查询…

对

系统按照查询条件搜索零件

只书写“可观测”的

书写用例文档

- 基本路径

错

欧文从贝克汉姆处得到传球，守门员…

对

贝克汉姆传球给欧文，欧文射门，守门员扑救….

使用主动语句

书写用例文档

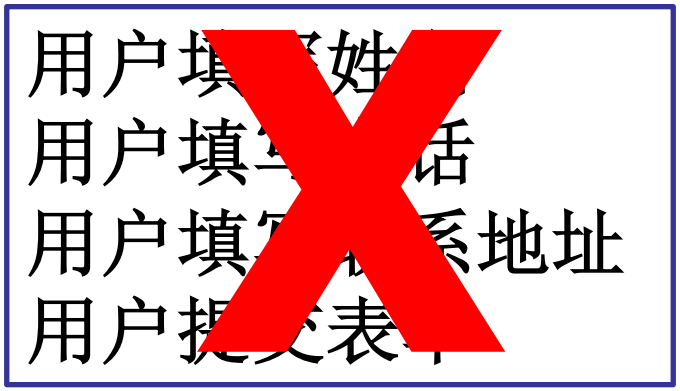
- 基本路径

错	系统从会员处获取用户名和密码
对	会员提交用户名和密码
错	用户名和密码被验证
对	系统验证用户名和密码

使用主动语句

书写用例文档

- 基本路径



每一句都要朝目标迈进

书写用例文档

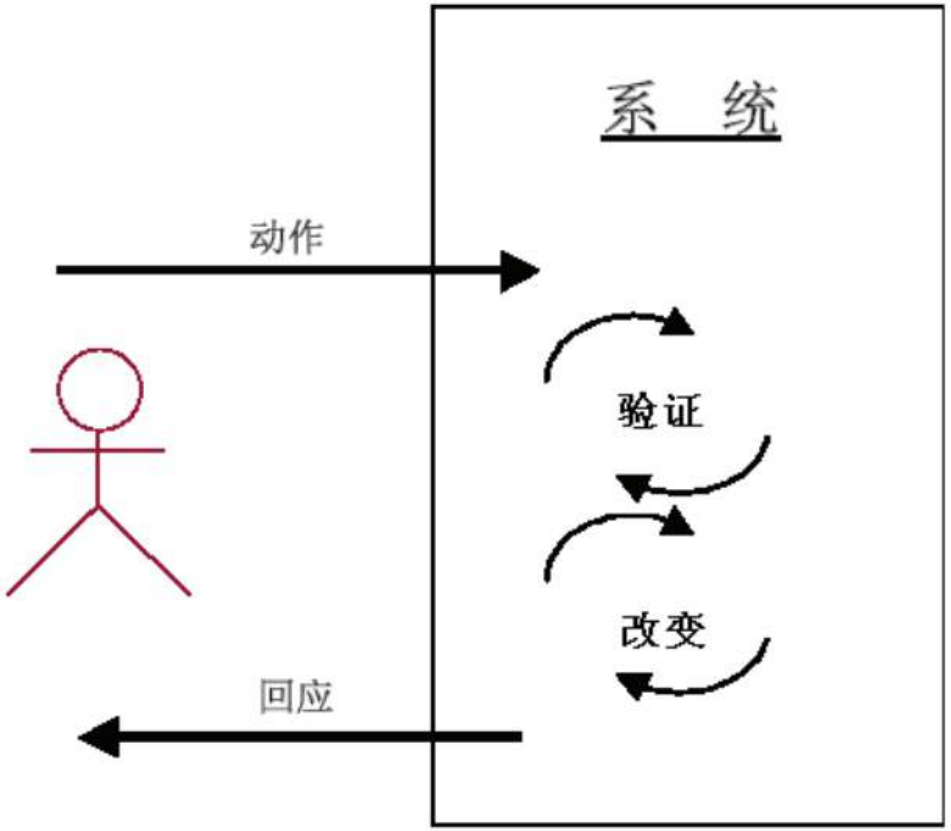
- 基本路径

会员从下拉框中选择类别
会员在相应文本框中输入查询条件
会员点击“确定”按钮……

不要涉及界面细节

书写用例文档

- 基本路径
  - 用例交互四步曲



书写用

1. 用例编号：UC0201
2. 用例编号：UC0202
3. 用例编号：UC0203
4. 用例编号：UC0204
5. 用例编号：UC0205
6. 用例编号：UC0301
7. 用例编号：UC0302
8. 用例编号：UC0303
9. 用例编号：UC0304
10. 用例编号：UC0305
11. 用例编号：UC0701
12. 用例编号：UC0402
13. 用例编号：UC0703
14. 用例编号：UC0404
15. 用例编号：UC0405
16. 用例编号：UC0706
17. 用例编号：UC0501
18. 用例编号：UC0502
19. 用例编号：UC0503

10. 用例编号：UC0305 用例名：查看配送点间运输价格

执行者

配送点员工

前置条件

员工已登录

后置条件

系统按查询条件显示配送点之间的运输价格

涉众利益

配送点员工：需要通过配送点间运输价格估算运费

基本路径

1. 配送点员工打开配送点间运输价格查询页面
2. 配送点员工输入发货配送点、收货配送点的名称
3. 系统处理查询请求
4. 系统显示配送点之间的运输价格

扩展

2a1. 输入配送点名称有误或为空，查询结果为空

3b1. 系统出错，不能查询出正确的结果

字段列表

2. 发货配送点名称，收货配送点名称

业务规则

2. 可按配送点名称模糊查询；

非功能需求

设计约束

用例建模——步骤

- 识别执行者
- 识别用例
- 绘制用例图
- 书写用例文档
- 检查用例模型

检查用例模型

- 用例模型完成之后，需要对用例模型进行检查，看看是否有遗漏或错误之处。主要可以从以下几个方面来进行检查：
  - 功能需求的完备性
  - 模型是否易于理解
  - 是否存在不一致性
  - 避免语义二义性

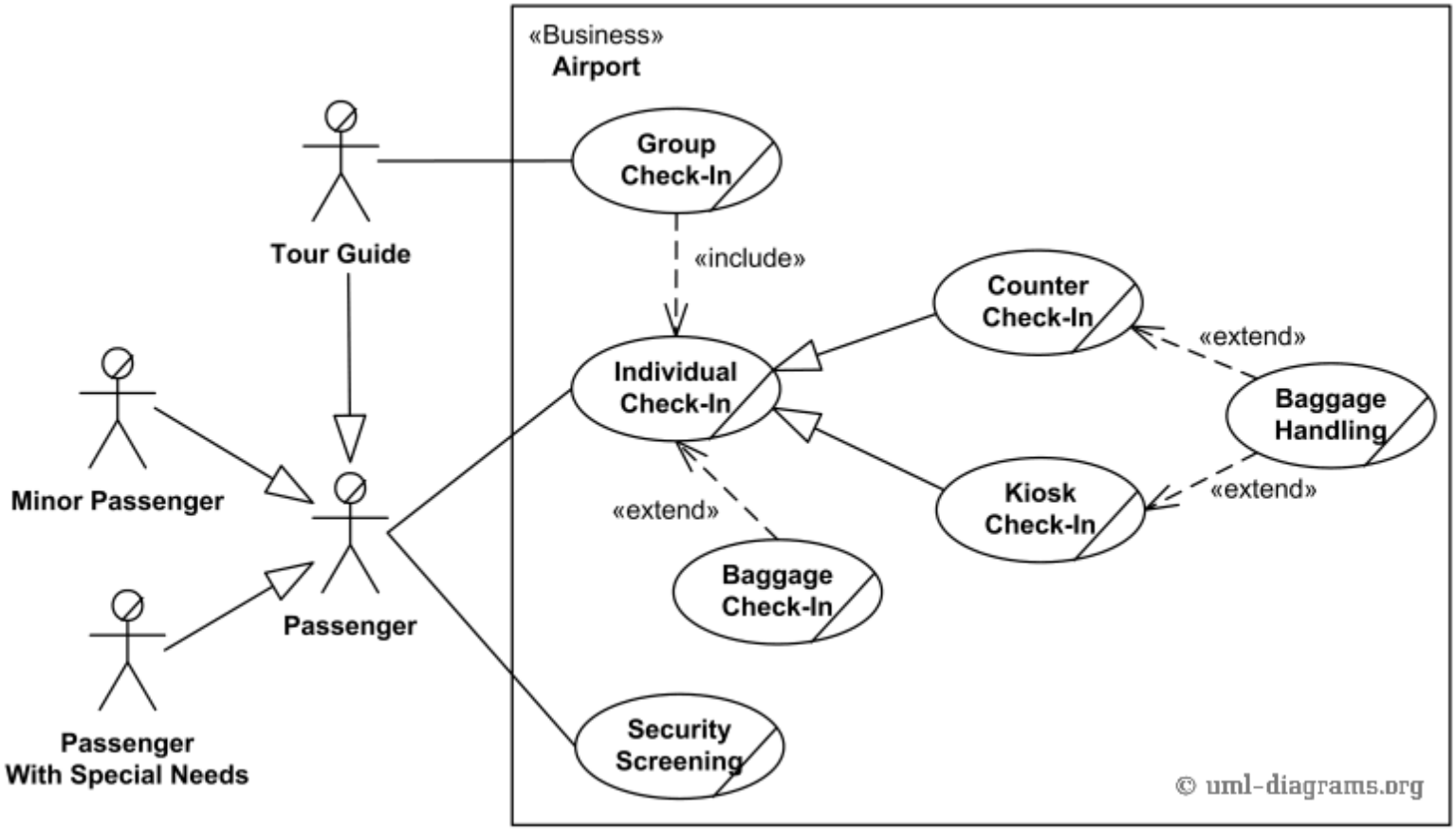
检查用例模型

- 用例检查表

	用例1	用例2	**用例3**	用例4	用例5	…
需求1	●	●				
需求2					●	
需求3						
需求4				●		
需求5	●			●		
…						

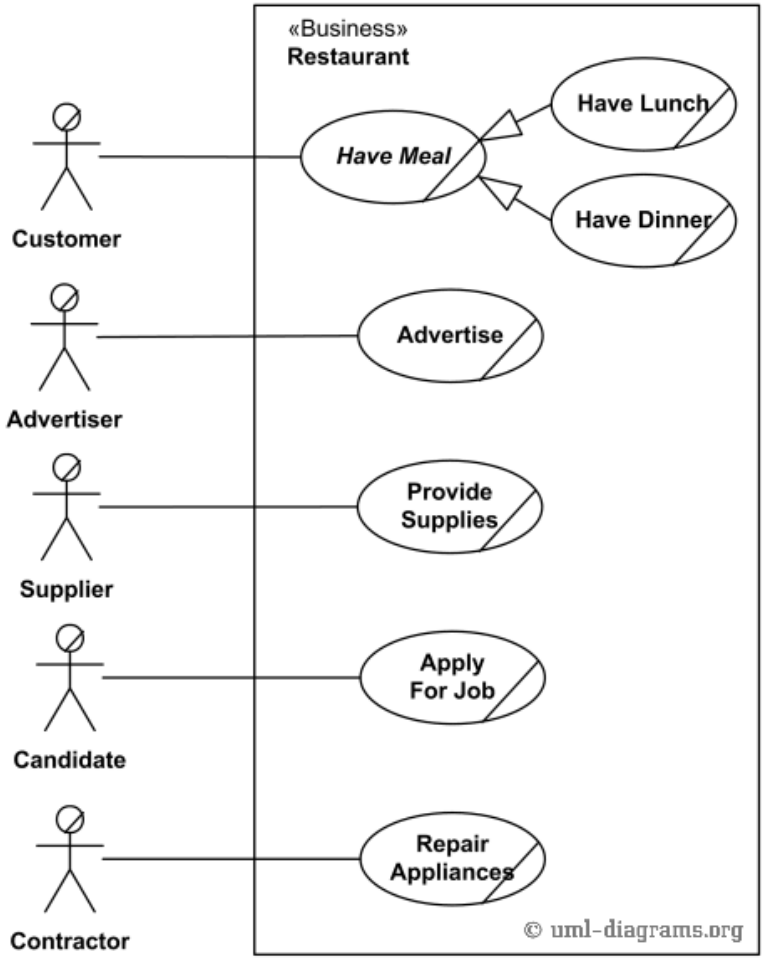
用例建模举例

- Airport Check-In and Security Screening



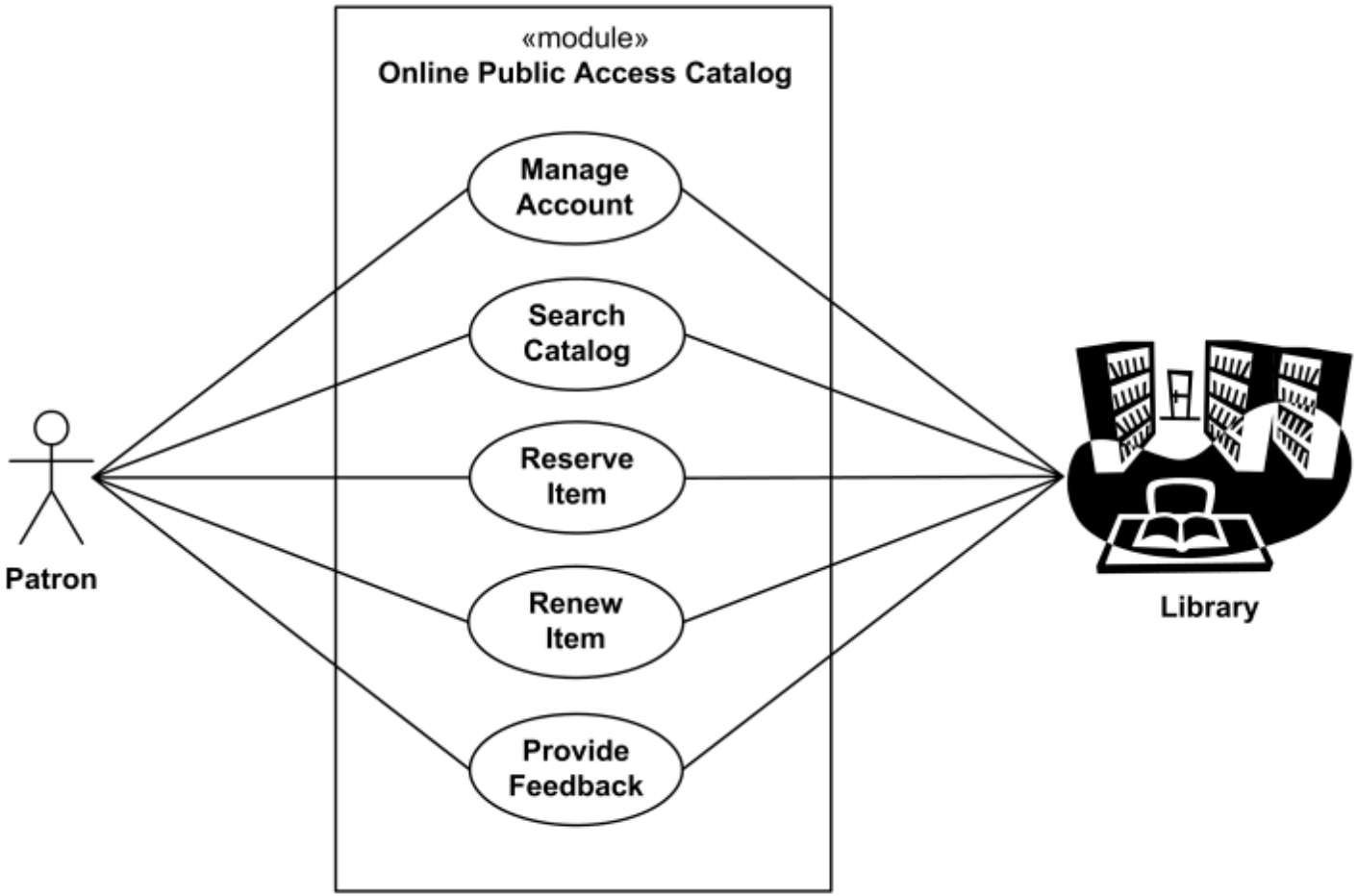
用例建模举例

- Restaurant



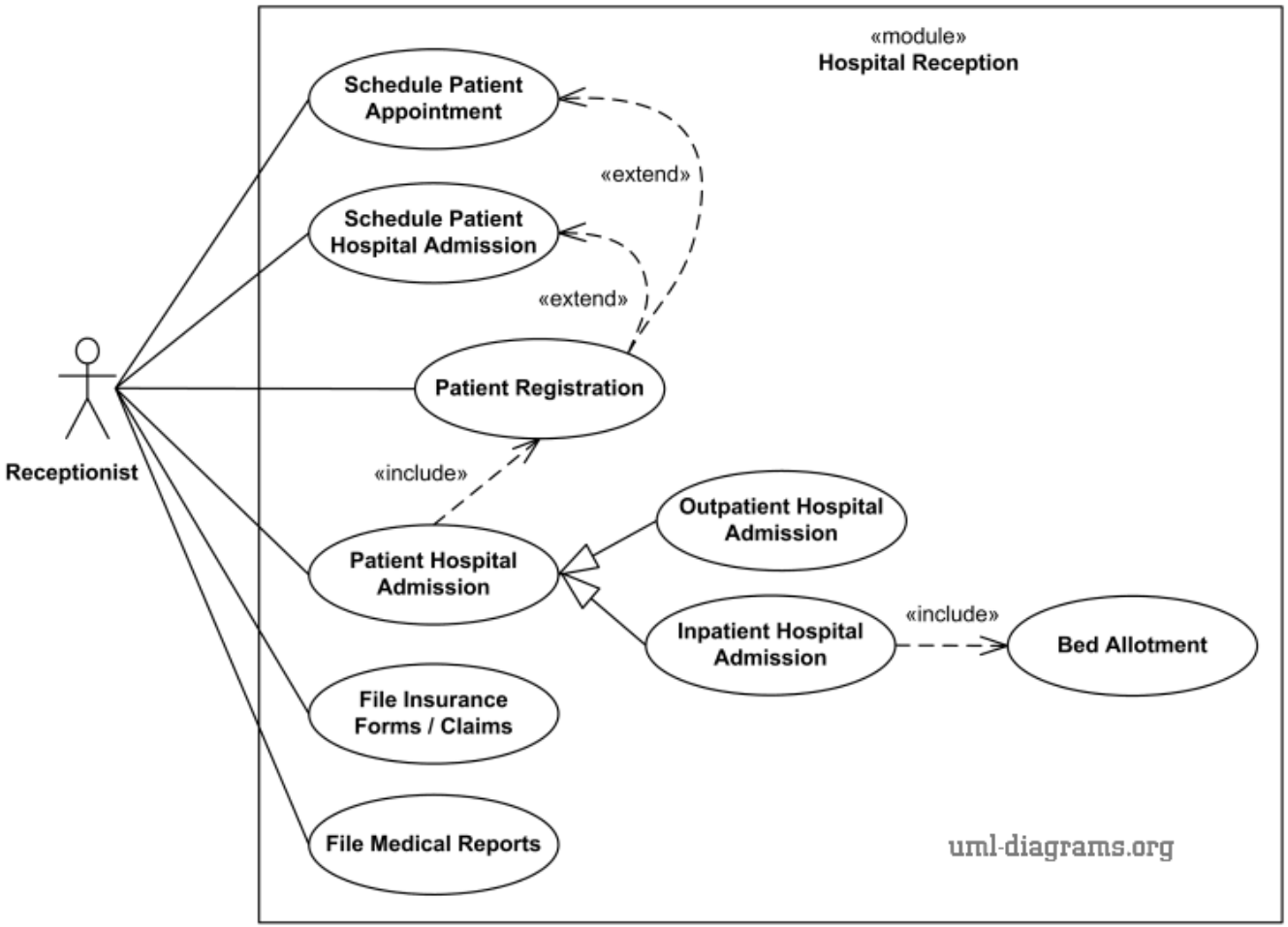
用例建模举例

- 图书管理系统



用例建模举例

- 医院管理系统

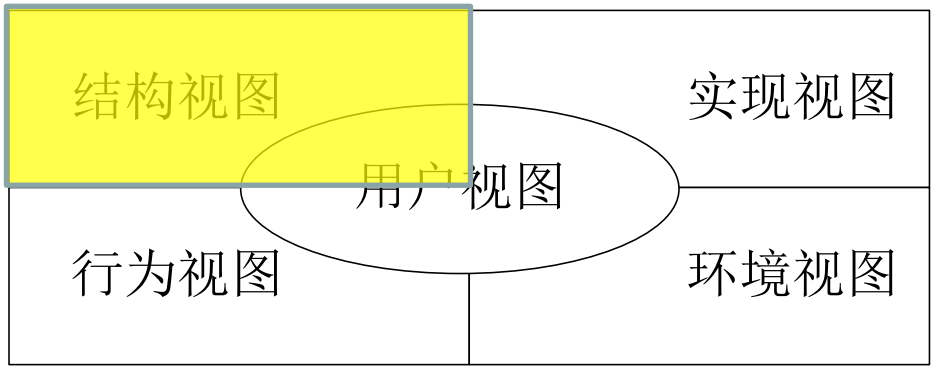


分割线

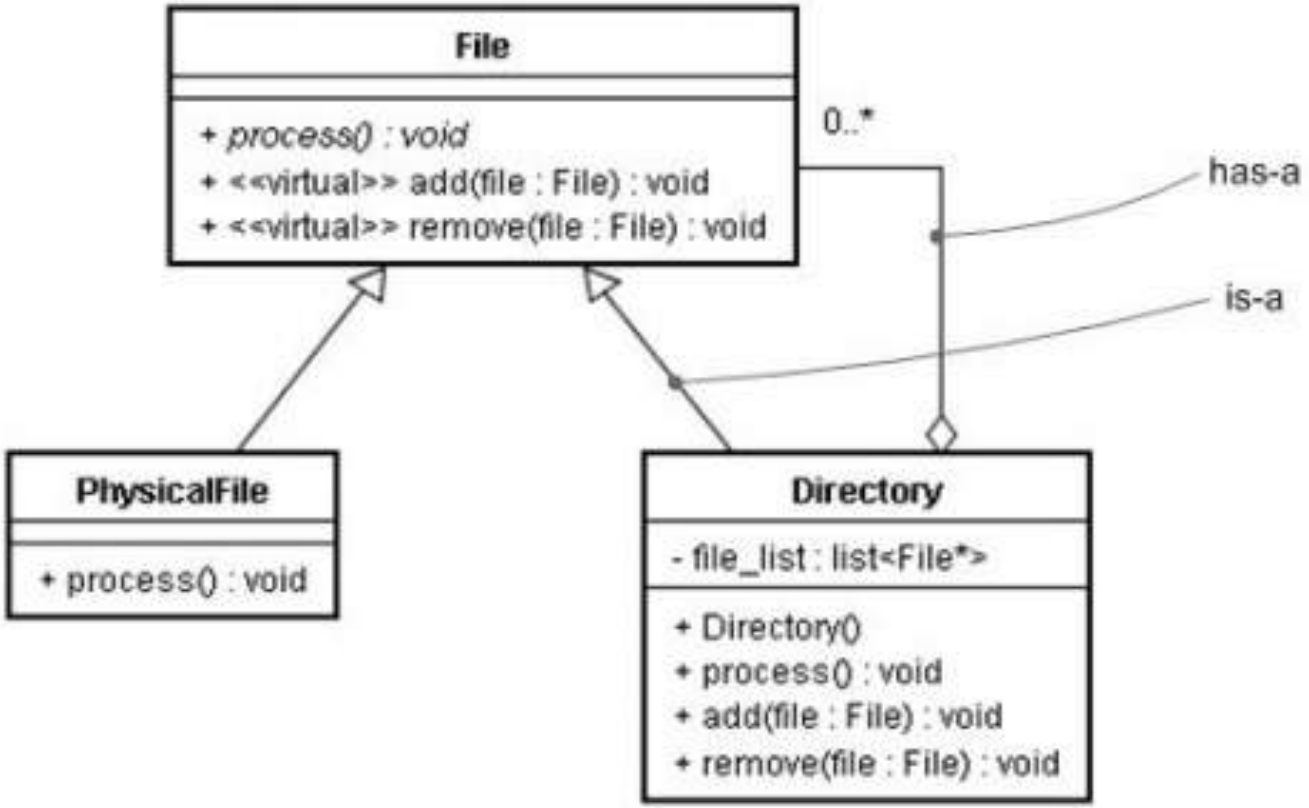
- 第五周
- 第六周

第三部分

UML静态建模



静态建模概述


File
+ process() : void
+ <<virtual>> add(file : File) : void
+ <<virtual>> remove(file : File) : void
0..*
has-a
is-a
PhysicalFile
+ process() : void
Directory
- file_list : list<File*>
+ Directory()
+ process() : void
+ add(file : File) : void
+ remove(file : File) : void

术语

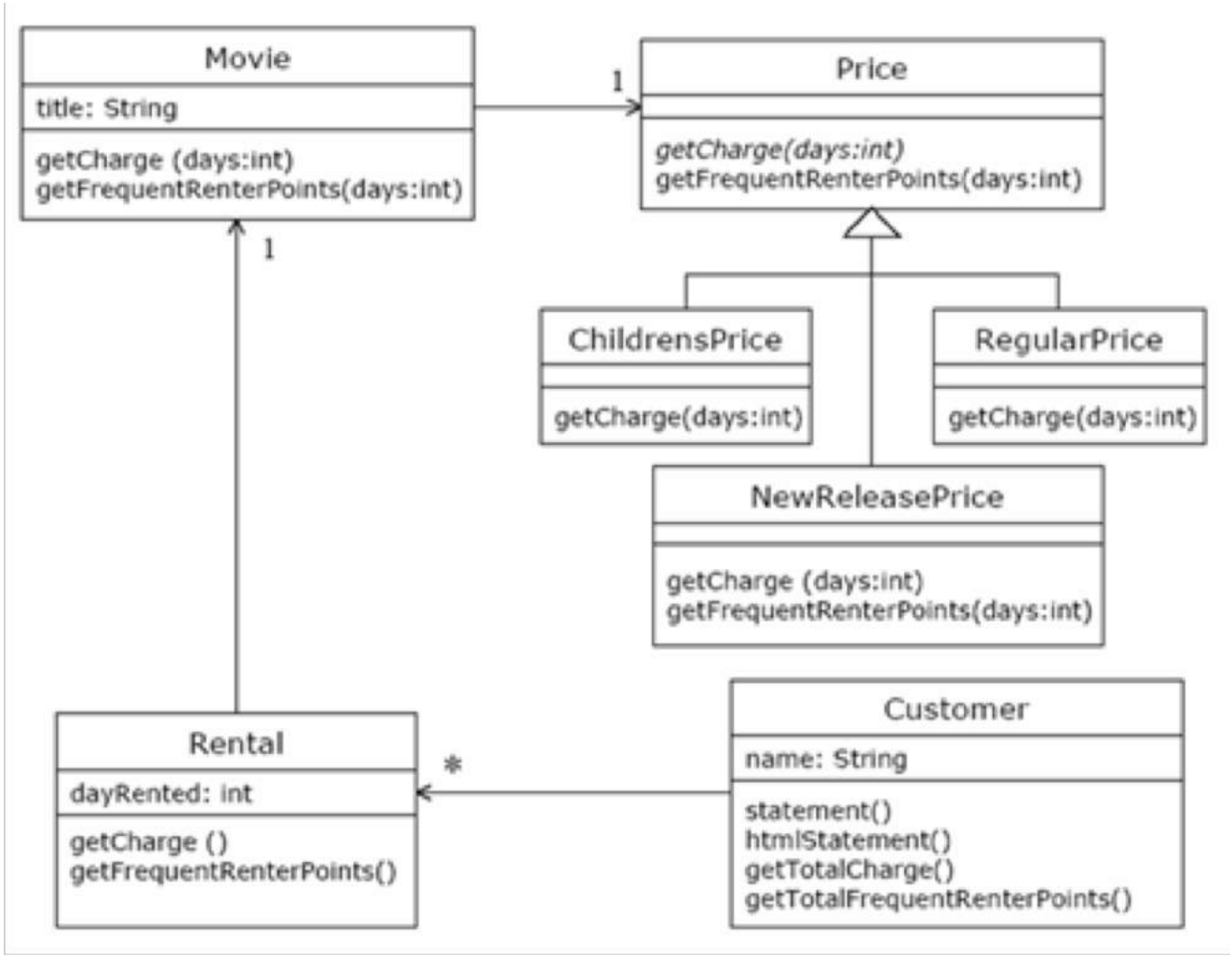
- DAO (Data Access Object): 数据访问对象
  - 主要用来封装对数据库的访问
- DTO (Data Transfer Object): 数据传输对象
  - 主要用于远程调用等需要大量传输对象的地方

比如我们一张表有100个字段，那么对应的DAO就有100个属性。但是我们界面上只要显示10个字段，客户端用WEB Service来获取数据，没有必要把整个DAO对象传递到客户端， 这时我们就可以用只有这10个属性的DTO来传递结果到客户端，这样也不会暴露服务端表结构.到达客户端以后，如果用这个对象来对应界面显示.

静态建模概述

- UML静态模型描述了数据如何封装到对象中，类和对象的职责如何划分以及它们之间关系如何
- 静态建模涉及为开发系统的静态结构而识别系统的静态元素和它们的关系
- UML静态建模可以使用类图、对象图、包图和组成结构图。其中最重要的是类图，类图也是UML建模中最为常用的图形之一，大多数UML图都是类图

类图


Movie
title: String
getCharge (days:int)
getFrequentRenterPoints(days:int)
1
Price
getCharge(days:int)
getFrequentRenterPoints(days:int)
1
ChildrensPrice
getCharge(days:int)
RegularPrice
getCharge(days:int)
NewReleasePrice
getCharge (days:int)
getFrequentRenterPoints(days:int)
Rental
dayRented: int
getCharge ()
getFrequentRenterPoints()
*
Customer
name: String
statement()
htmlStatement()
getTotalCharge()
getTotalFrequentRenterPoints()

类图

- 类的定义
  - 类（Class）包装了信息和行为，类是具有相同属性、操作的对象集合
  - 每个类具有一定的职责，职责指的是类所承担的任务，即类需要完成的功能
  - 一个类可以有多种职责，设计得好的类一般只有一种职责（单一职责原则）
  - 在定义类的时候，将类的职责分解成为类的属性和操作

类图

- 定义属性
  - 在UML中，类的属性表示方式如下：

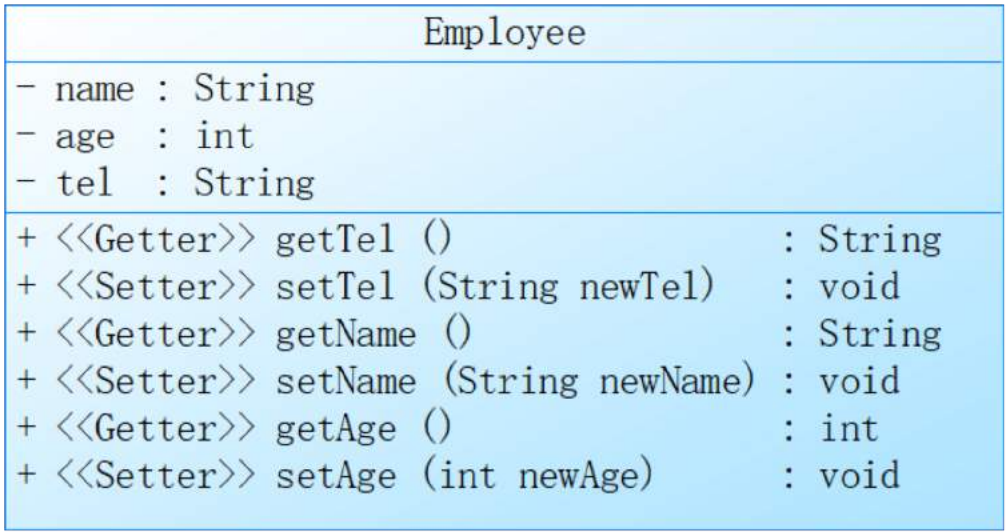
可见性 名称:类型 = 缺省值

- 定义操作
  - 在UML中，类的操作表示方式如下：

可见性 名称(参数列表):返回类型

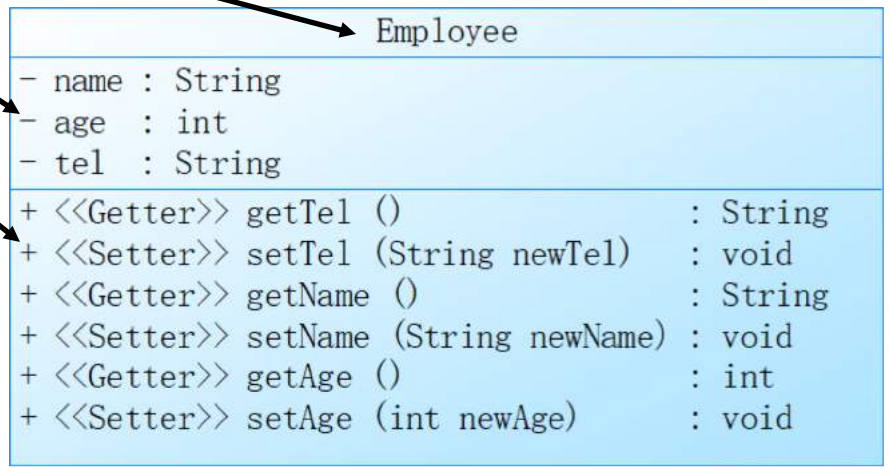
类图

- 类图
  - 类图使用系统内的不同类来描述系统的静态结构
  - 类图包含类和它们之间的关系。它描述系统内所声明的类，但它没有描述系统运行时类的行为
  - 在UML中，类使用具有类名称、属性、操作分隔的长方形来表示



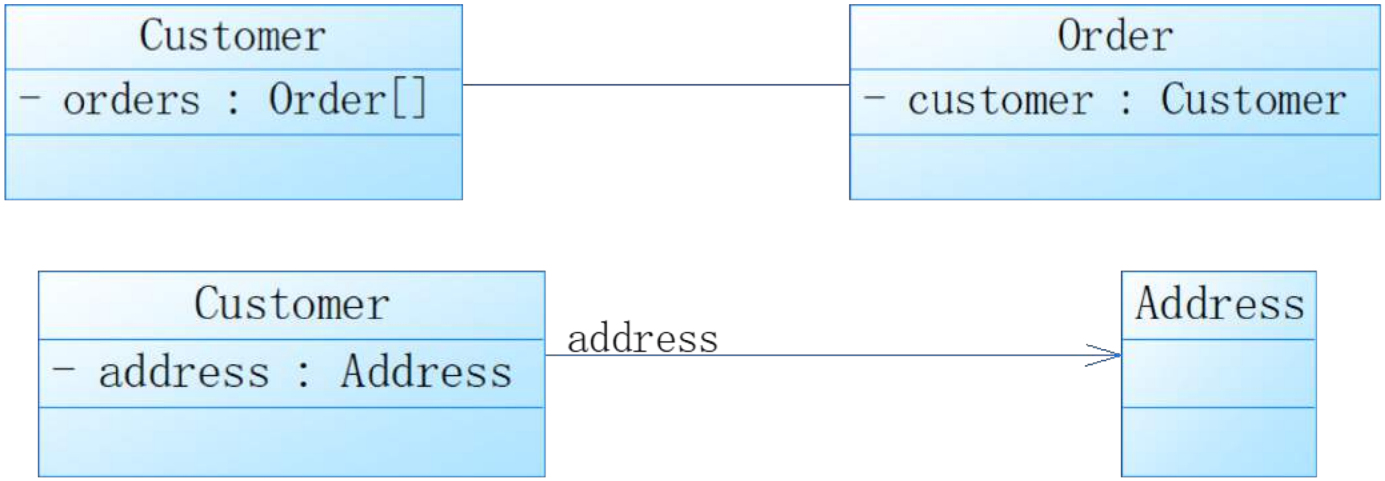
类图

- 类图表示：类由三格组成
  - 第一格是类名
  - 第二格是类的属性
  - 第三格是类的操作



类图

- 关联（Association）关系
  - 关联是一种结构化的关系，指一种对象和另一种对象有联系。给定有关联的两个类，其中一个类的对象包含另一个类的对象



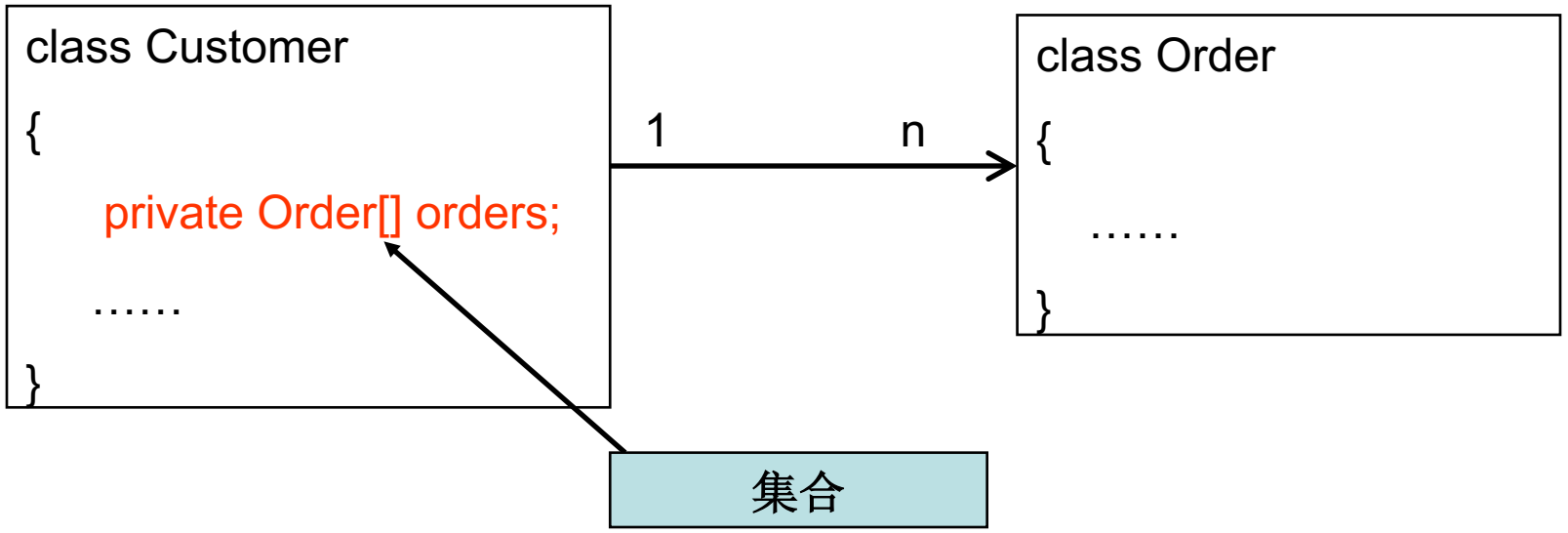
类图

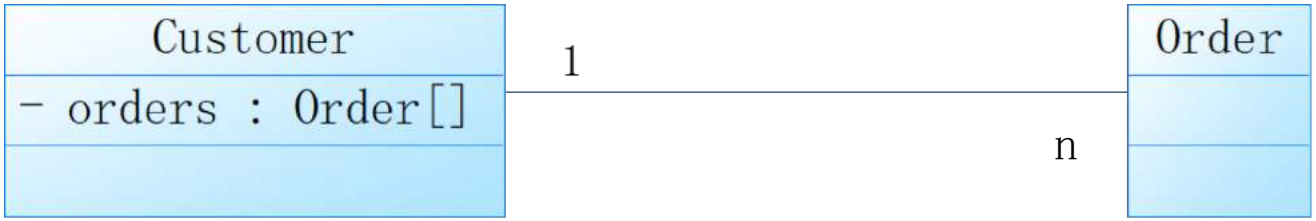
- 关联关系
  - 关联有双向关联和单向关联



类图

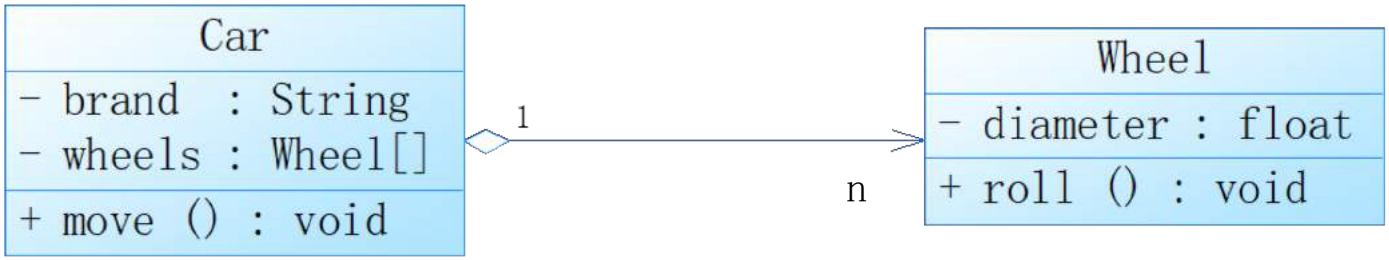
- 关联关系
  - 多重性关联：在关联直线上增加一个数字表示与之对应的另一个类的对象的个数





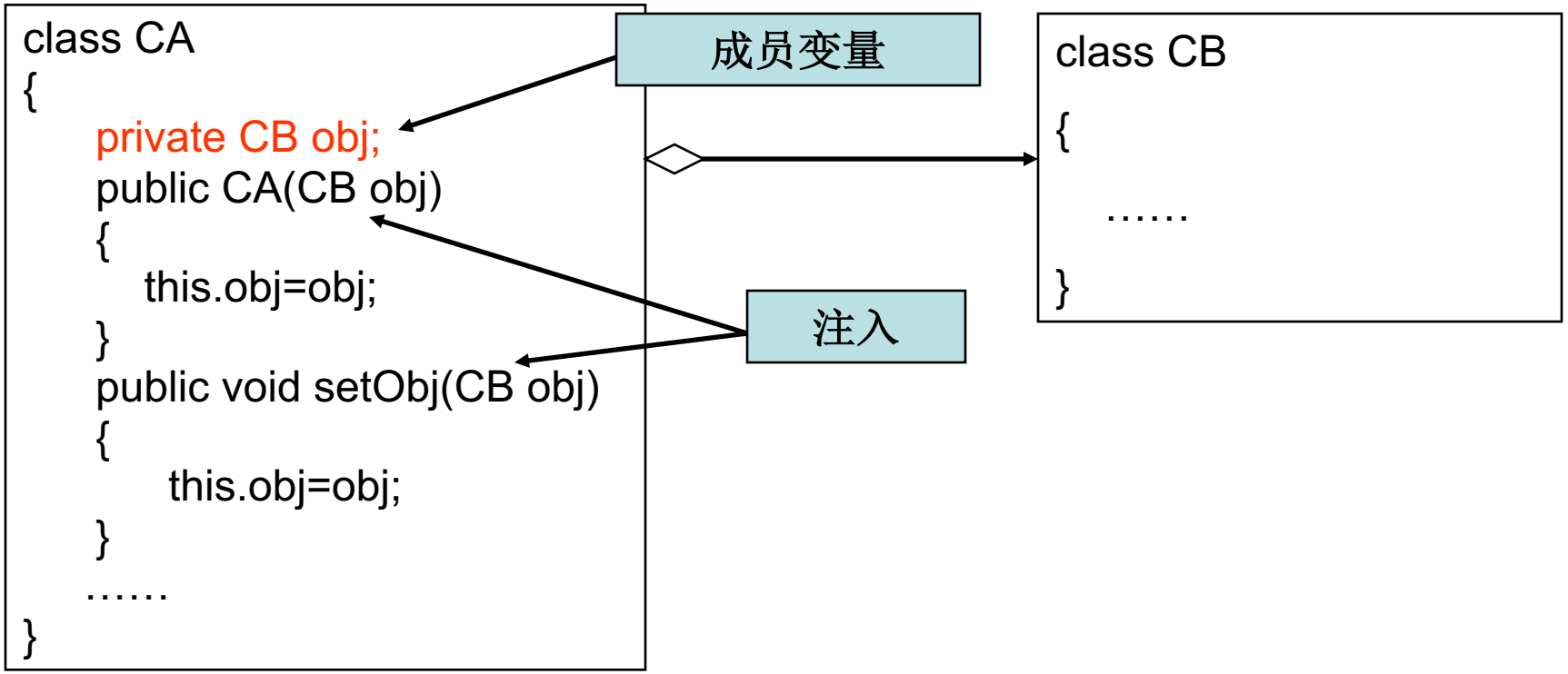
类图

- 聚合（Aggregation）关系
  - 聚合关系指的是整体与部分的关系
  - 通常在定义一个整体类后，再去分析这个整体类的组成结构，从而找出一些成员类，该整体类和成员类之间就形成了聚合关系
  - 在聚合关系中，类A是类B的一部分，但是类A可以独立存在，在UML中，聚合关系用带空心菱形的直线表示



类图

- 聚合关系


class CA
{
private CB obj;
public CA(CB obj)
{
this.obj=obj;
}
public void setObj(CB obj)
{
this.obj=obj;
}
……
}
成员变量
注入
class CB
{
……
}

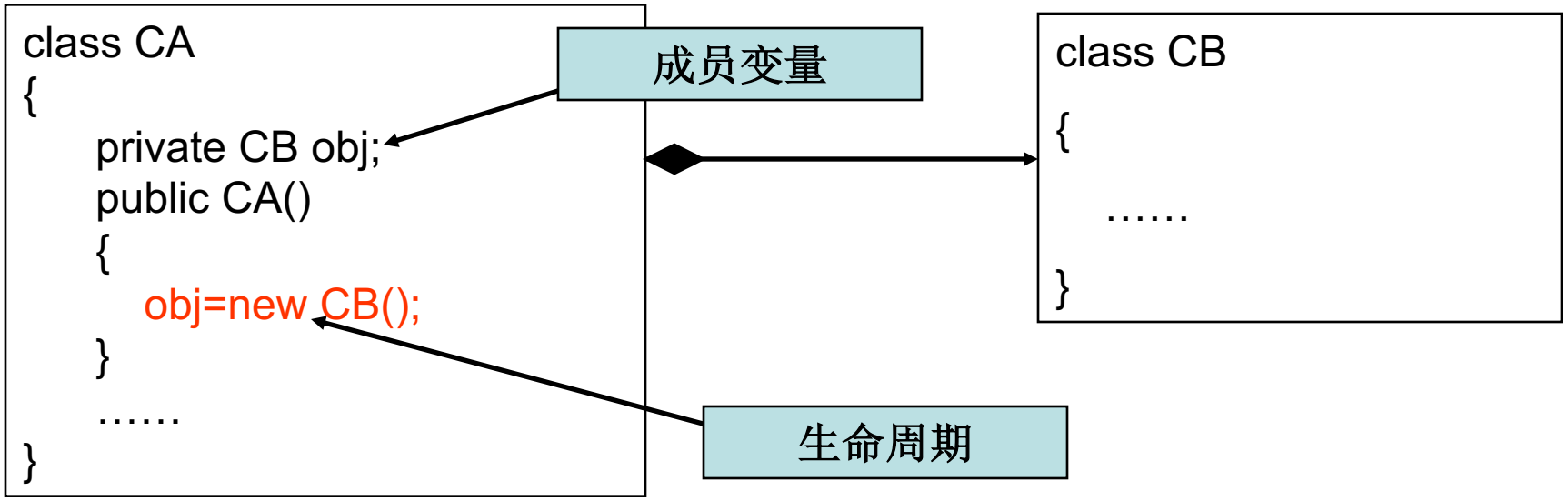
类图

- 组合（Composition）关系
  - 组合关系也表示类之间整体和部分的关系，但是组合关系中部分和整体具有相同的生存期。一旦整体对象不存在，部分对象也将不存在，部分对象与整体对象之间具有共生死的关系
  - 在组合关系中，类A包含类B，而且可以控制类B的生命周期。类A控制类B的生命周期意味着类B的存在依赖于类A。在UML中，组合关系用带实心菱形的直线表示



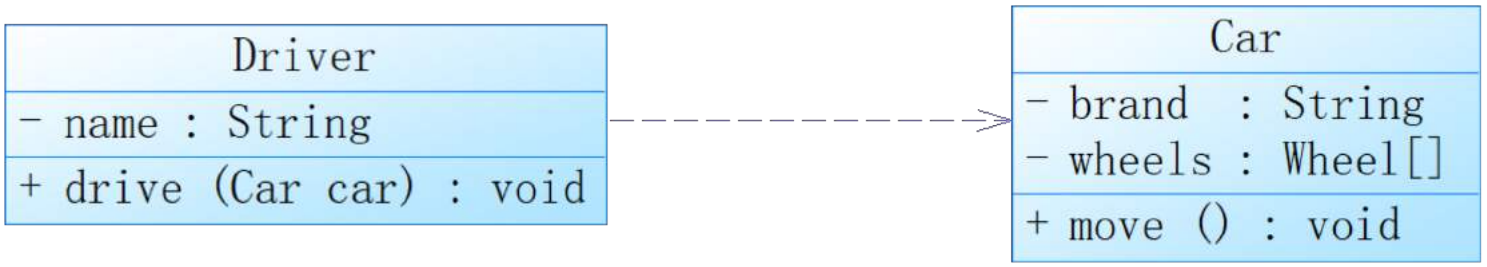
类图

- 组合关系


class CA
{
private CB obj;
public CA()
{
obj=new CB();
}
……
}
成员变量
生命周期
class CB
{
……
}

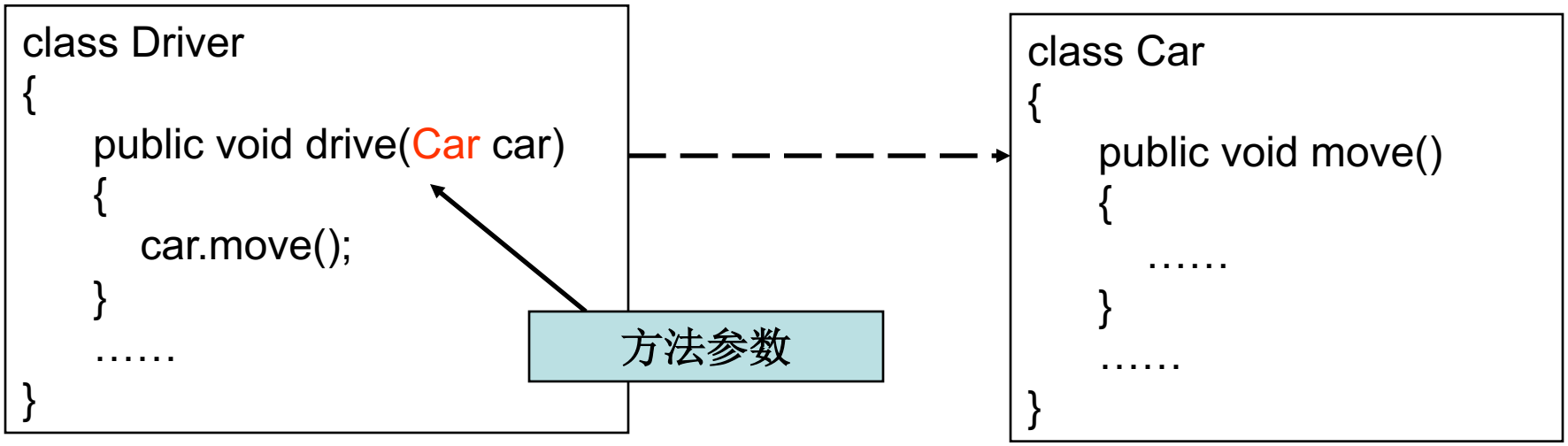
类图

- 依赖（Dependency）关系
  - 依赖关系是一种使用关系，特定事物的改变有可能会影响到使用该事物的其他事物，反之不成立，在需要表示一个事物使用另一个事物时使用依赖关系。
  - 依赖关系体现在某个类的方法使用另一个类作为参数。在UML中也可以在其他的事物之间使用依赖关系，如节点之间的关系。依赖关系用带箭头的虚线表示，由依赖的一方指向被依赖的一方。



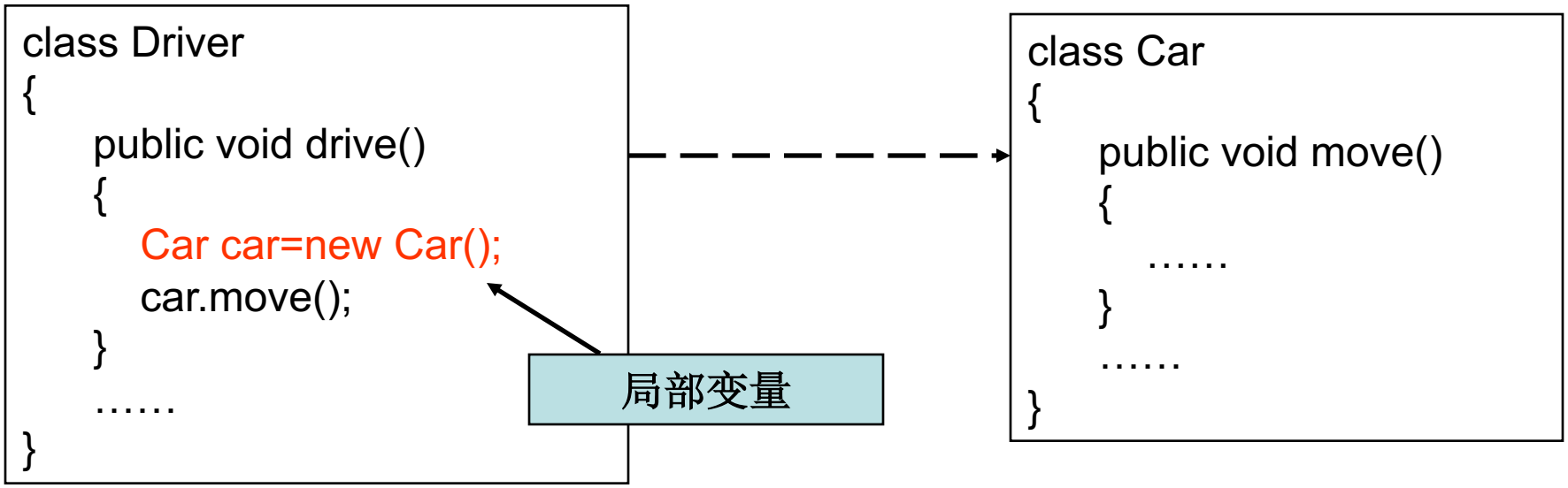
类图

- 依赖关系


class Driver
{
public void drive(Car car)
{
car.move();
}
……
}
方法参数
class Car
{
public void move()
{
……
}
……
}

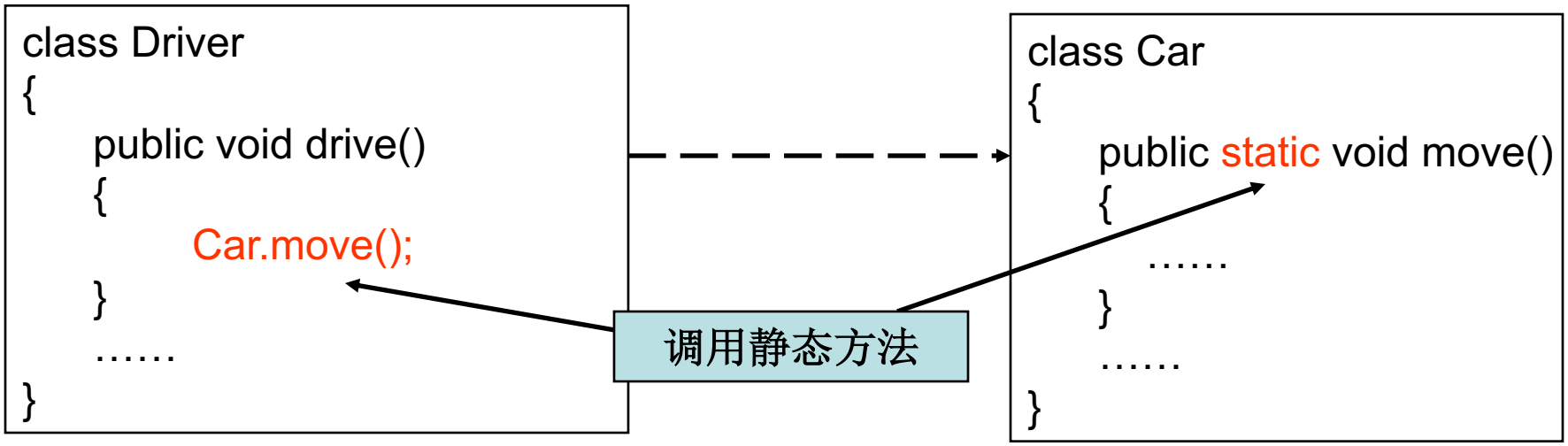
类图

- 依赖关系


class Driver
{
public void drive()
{
Car car=new Car();
car.move();
}
……
}
局部变量
class Car
{
public void move()
{
……
}
……
}

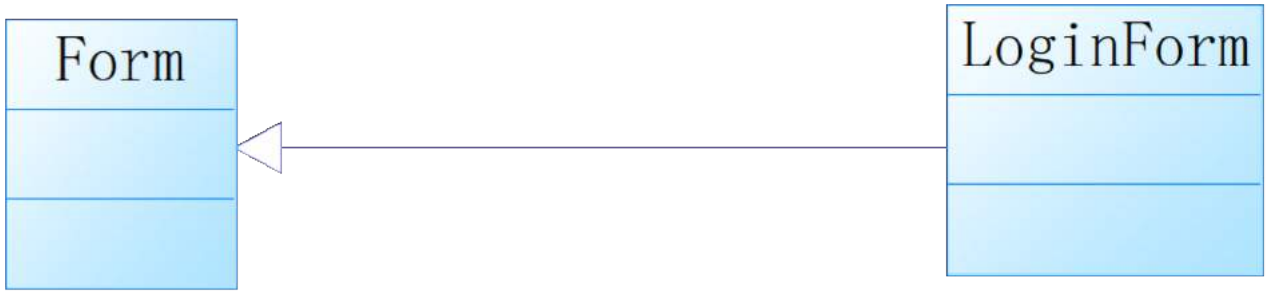
类图

- 依赖关系


class Driver
{
public void drive()
{
Car.move();
}
……
}
调用静态方法
class Car
{
public static void move()
{
……
}
……
}

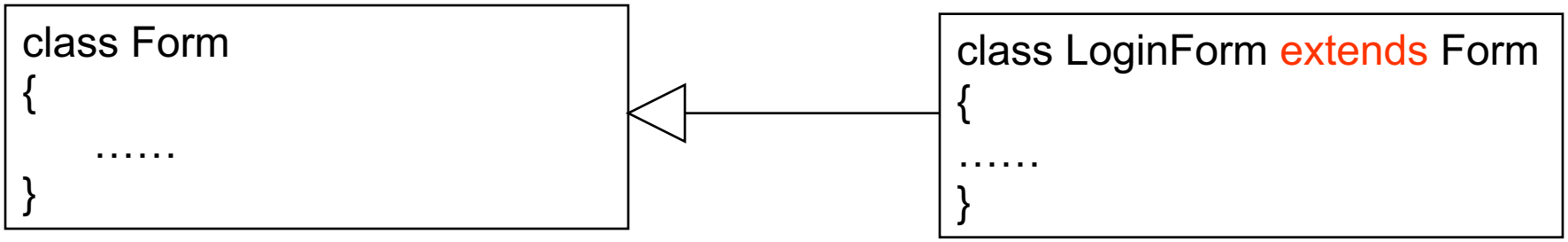
类图

- 泛化（Generalization）关系
  - 泛化也就是继承关系，也称为 “is-a-kind-of”关系，泛化关系描述了超类与子类之间的关系，超类又叫做基类，子类又叫做派生类。在UML中，泛化关系用带空心三角形的直线来表示。



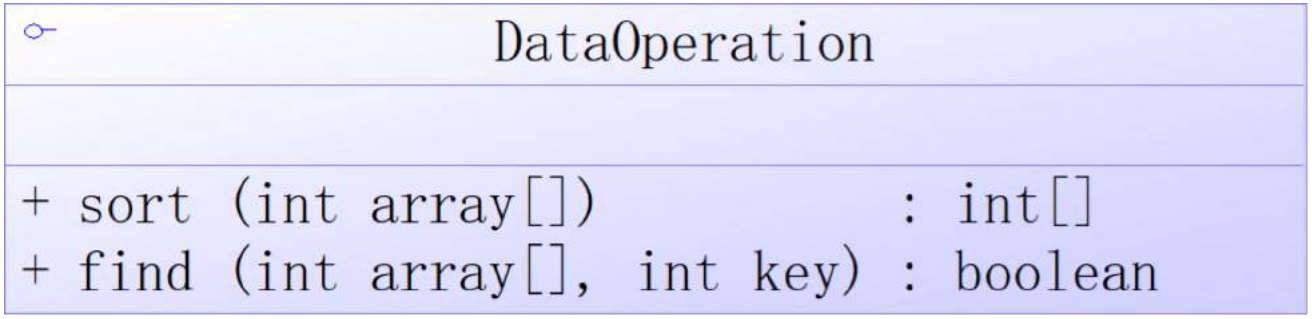
类图

- 泛化关系



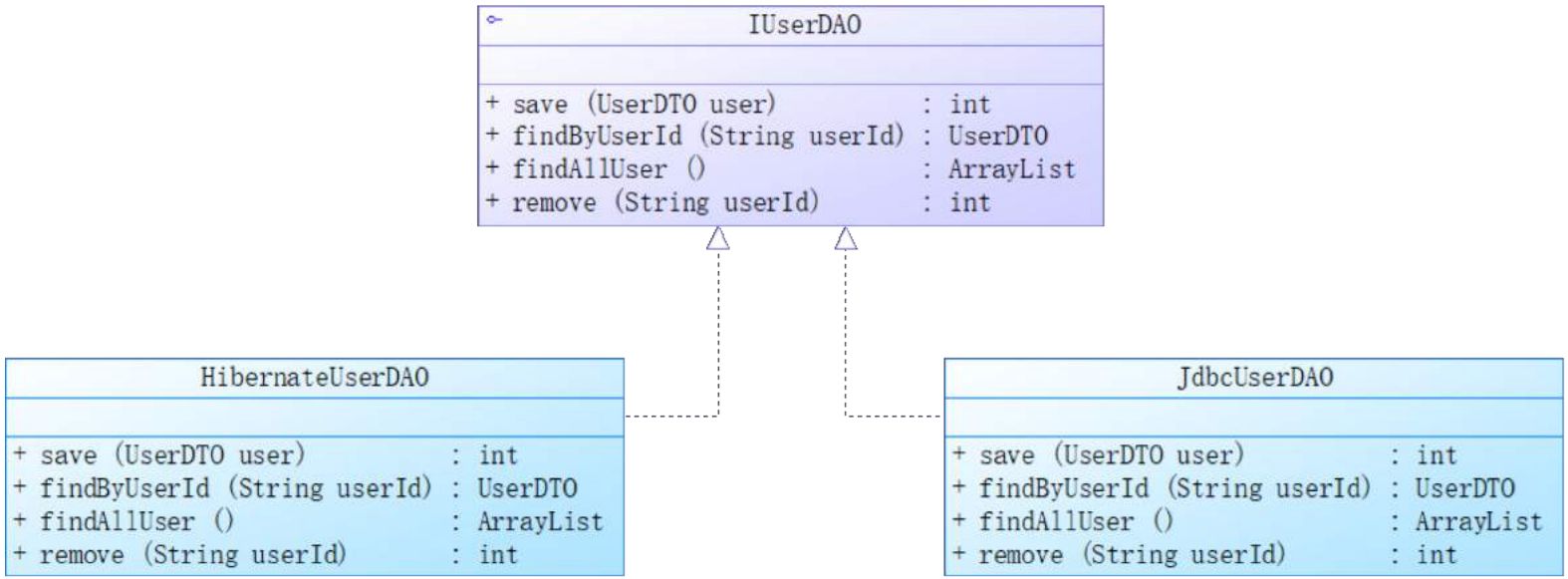
类图

- 实现（Realization）关系
  - 是一种类与接口的关系，表示类是接口所有特征和行为的实现
  - 在接口定义中，一般没有属性；而且所有的操作都是抽象的，只有操作的定义，没有操作的实现



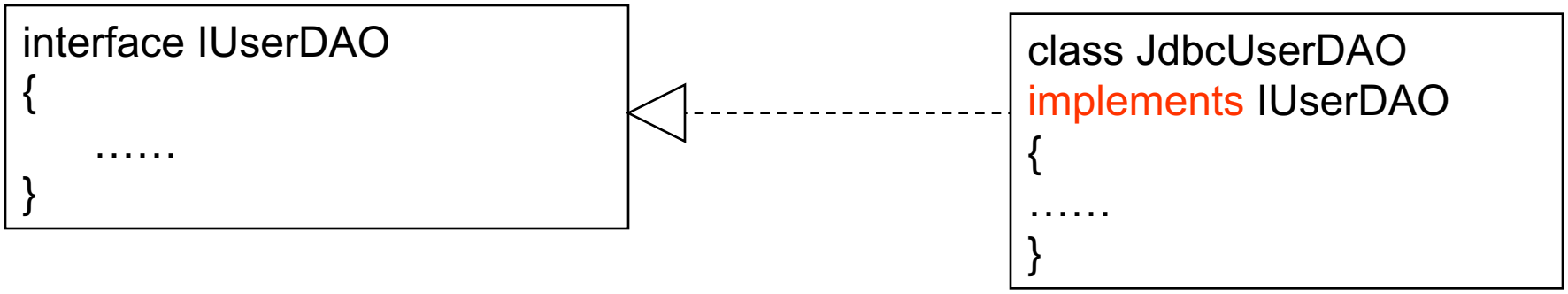
类图

- 实现关系
  - 类实现了接口，类中的操作实现了接口中所定义的操作。在UML中，类与接口之间的实现关系用带空心三角形的虚线来表示



类图

- 实现关系

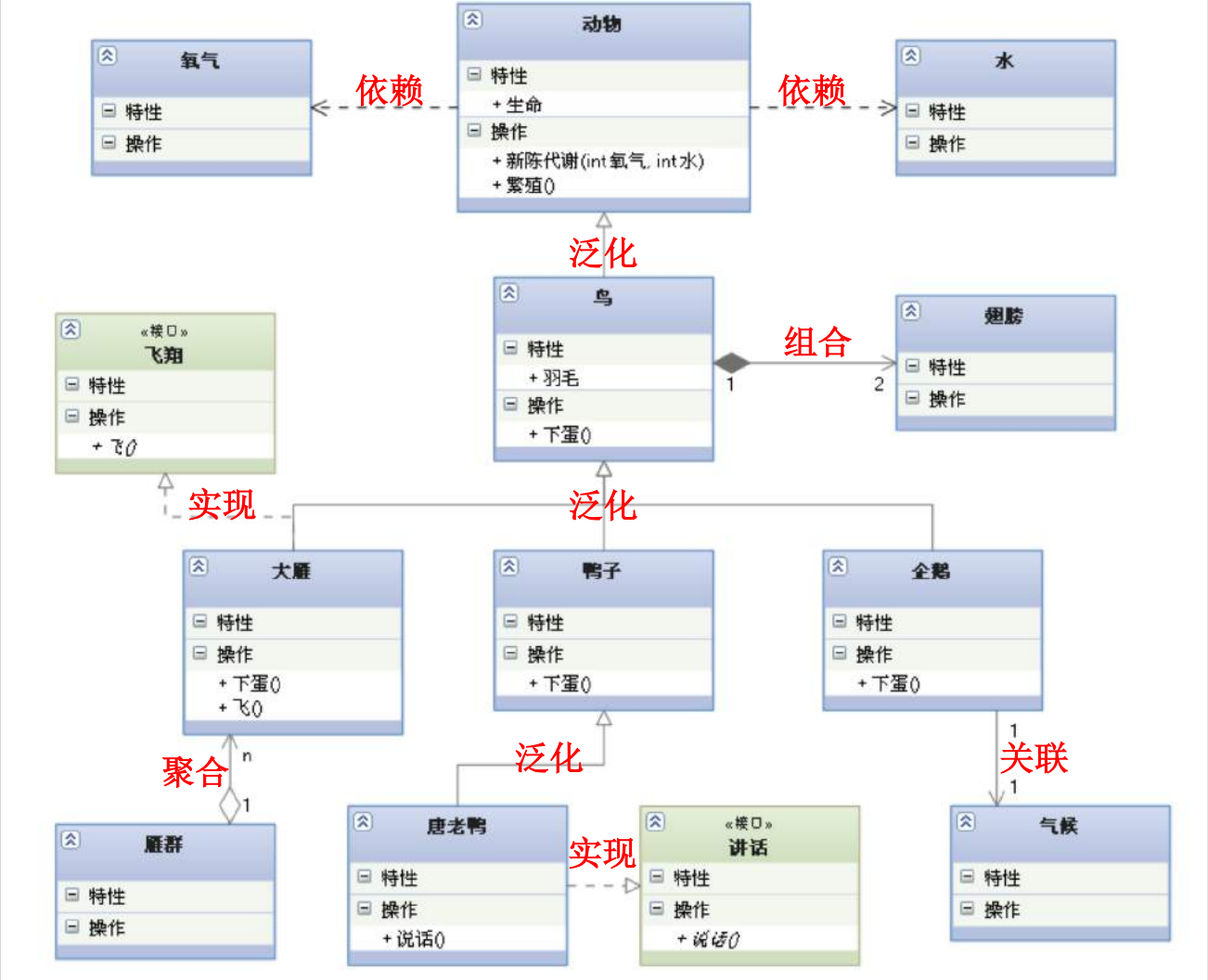

interface IUserDAO
{
……
}
class JdbcUserDAO
implements IUserDAO
{
……
}

类图

- 各种关系的强弱顺序
  - **泛化= 实现> 组合> 聚合> 关联> 依赖**

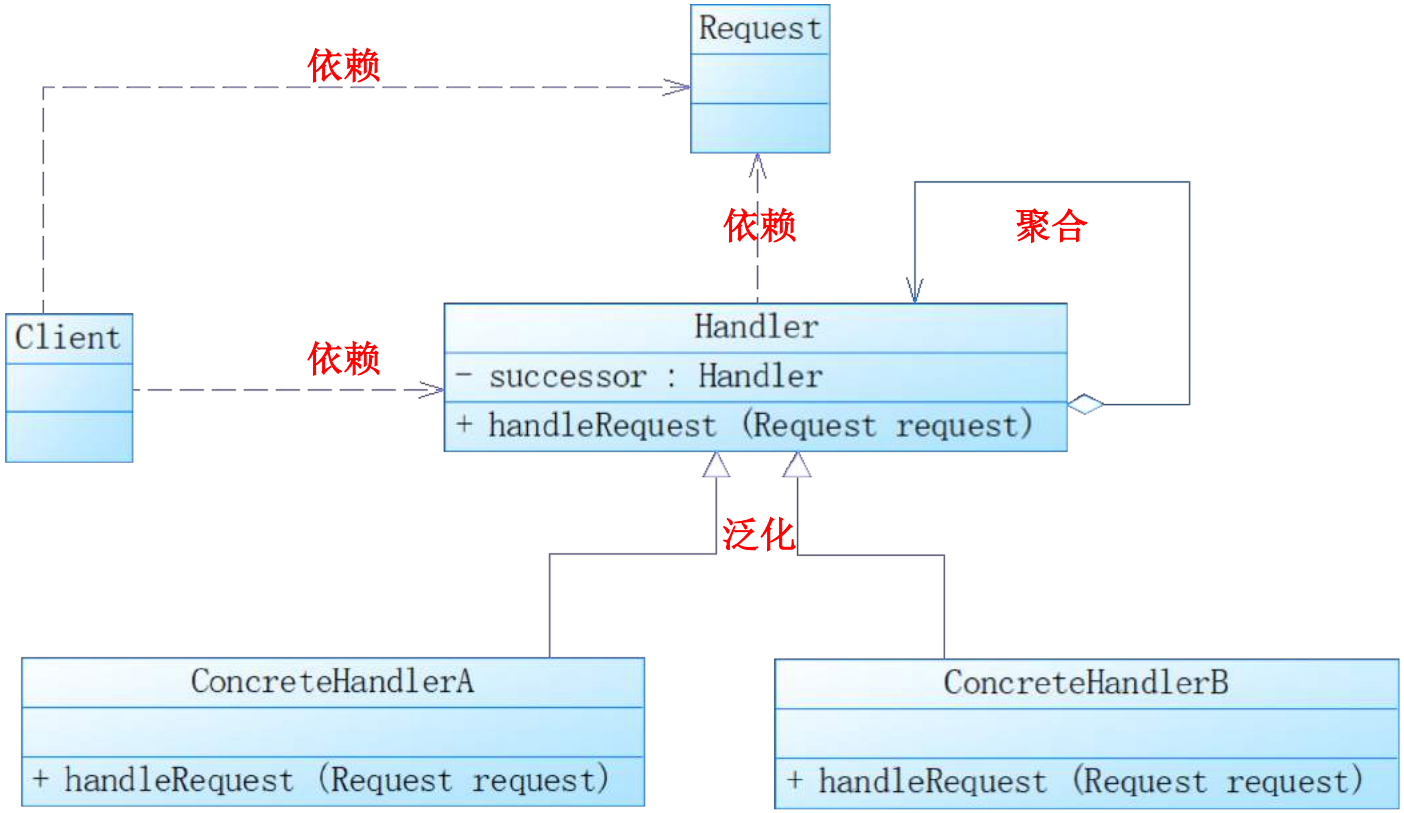
类图

- 实例一：迪士尼乐园的UML世界



类图

- 实例二：职责链

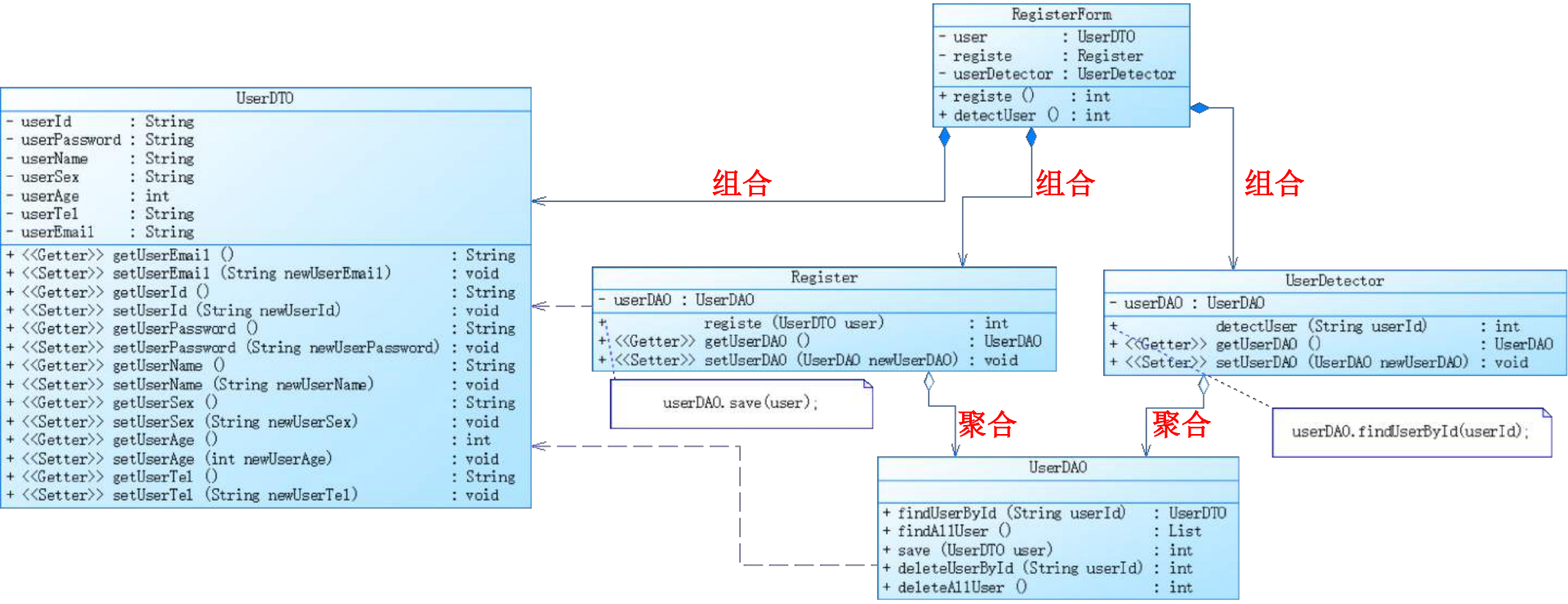

Request
依赖
依赖
聚合
Client
依赖
Handler
- successor : Handler
+ handleRequest (Request request)
泛化
ConcreteHandlerA
+ handleRequest (Request request)
ConcreteHandlerB
+ handleRequest (Request request)

类图

- 实例三：组合与聚合
  - 某C/S系统需要提供注册功能，注册用例描述如下：
    - 用户输入新帐号，系统检测该帐号是否已存在，如果不存在则可注册成功，否则提示“帐号已存在”，用户再次输入帐号；
    - 用户输入其他个人信息；
    - 系统保存用户个人信息；
    - 用户个人信息包括帐号、密码、姓名、性别、年龄、电话、电子邮箱。
  - 请画出实例中对应的类图

类图

- 实例三：组合与聚合


RegisterForm
- user : UserDTO
- registe : Register
- userDetector : UserDetector
+ registe () : int
+ detectUser () : int
UserDTO
- userId : String
- userPassword : String
- userName : String
- userSex : String
- userAge : int
- userTel : String
- userEmail : String
+ <<Getter>> getUserEmail () : String
+ <<Setter>> setUserEmail (String newUserEmail) : void
+ <<Getter>> getUserId () : String
+ <<Setter>> setUserId (String newUserId) : void
+ <<Getter>> getUserPassword () : String
+ <<Setter>> setUserPassword (String newUserPassword) : void
+ <<Getter>> getUserName () : String
+ <<Setter>> setUserName (String newUserName) : void
+ <<Getter>> getUserSex () : String
+ <<Setter>> setUserSex (String newUserSex) : void
+ <<Getter>> getUserAge () : int
+ <<Setter>> setUserAge (int newUserAge) : void
+ <<Getter>> getUserTel () : String
+ <<Setter>> setUserTel (String newUserTel) : void
组合
组合
组合
Register
- userDAO : UserDAO
+ registe (UserDTO user) : int
+ <<Getter>> getUserDAO () : UserDAO
+ <<Setter>> setUserDAO (UserDAO newUserDAO) : void
UserDetector
- userDAO : UserDAO
+ detectUser (String userId) : int
+ <<Getter>> getUserDAO () : UserDAO
+ <<Setter>> setUserDAO (UserDAO newUserDAO) : void
userDAO.save(user);
userDAO.findUserById(userId);
聚合
聚合
UserDAO
+ findUserById (String userId) : UserDTO
+ findAllUser () : List
+ save (UserDTO user) : int
+ deleteUserById (String userId) : int
+ deleteAllUser () : int

对象图

- 定义
  - 对象是指某个事物，大多对应于真实世界中的某个客观实体，所有的对象都是有唯一标识的独立实体
  - 对象之间的区别是由它们固有的存在性所决定的，任何对象都是某个类的实例
  - 对象图是类图在某一时刻的一个实例

对象图

- 表示
  - 在UML中对象表示为一个有两栏的矩形框。第一栏表示对象和类的名，第二栏表示对象的属性和值
    - 例如，empObj对象表示Employee 类的特定实例的属性值

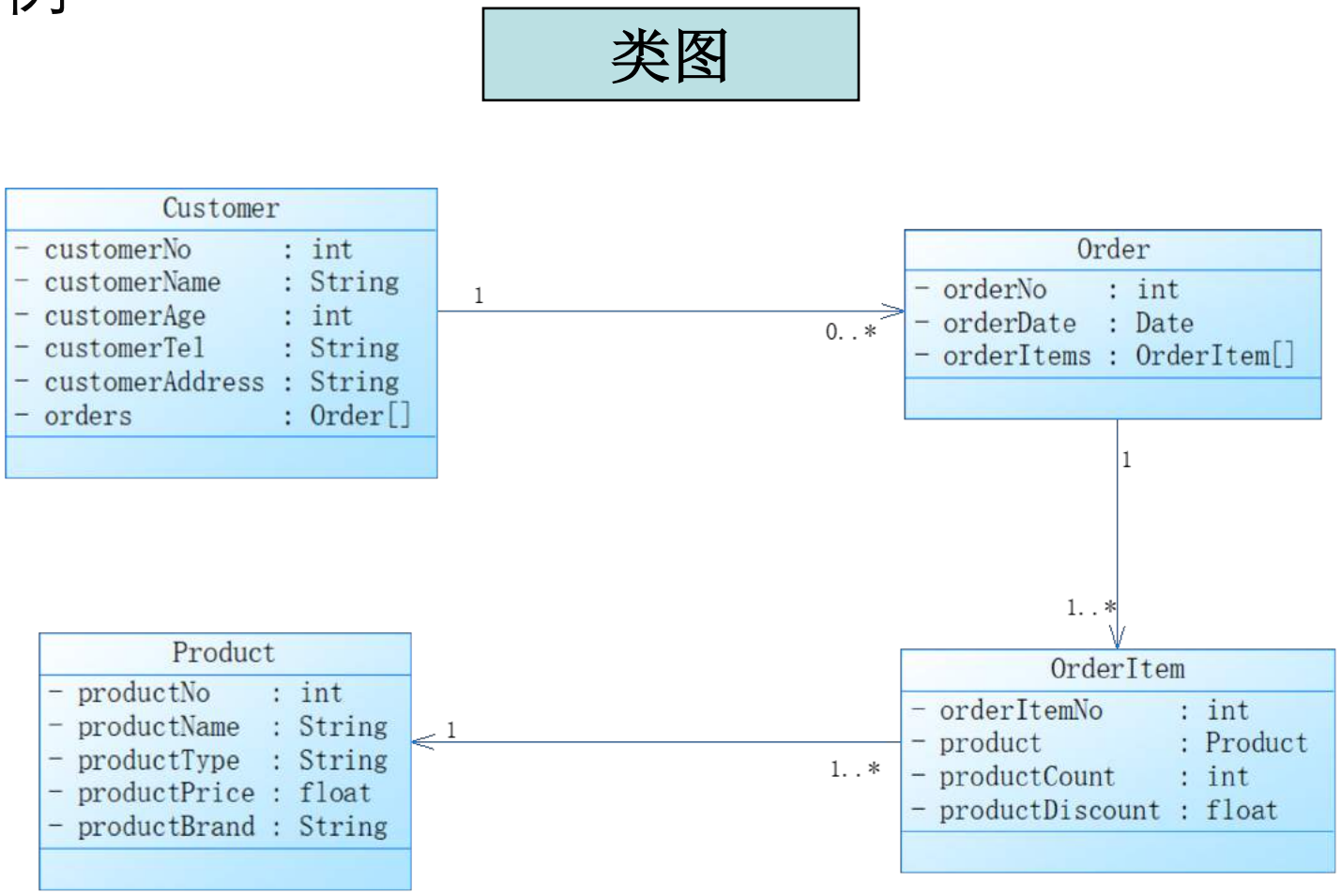
empObj:Employee	
empNo	= D0123
empName	= 王军
empAge	= 32
empSex	= 男
empDepartment	= 开发部
empPosition	= 项目经理
empTel	= 13000000000
empEmail	= junwang@test.com

对象图

- 解释
  - 对象图描述一段时间里特定实例的静态结构，此图描述了类图的实例
  - 由于对象存在生命周期，因此对象图只能在系统某一时间段存在
  - 通常在静态建模时只需要绘制类图，很少绘制对象图

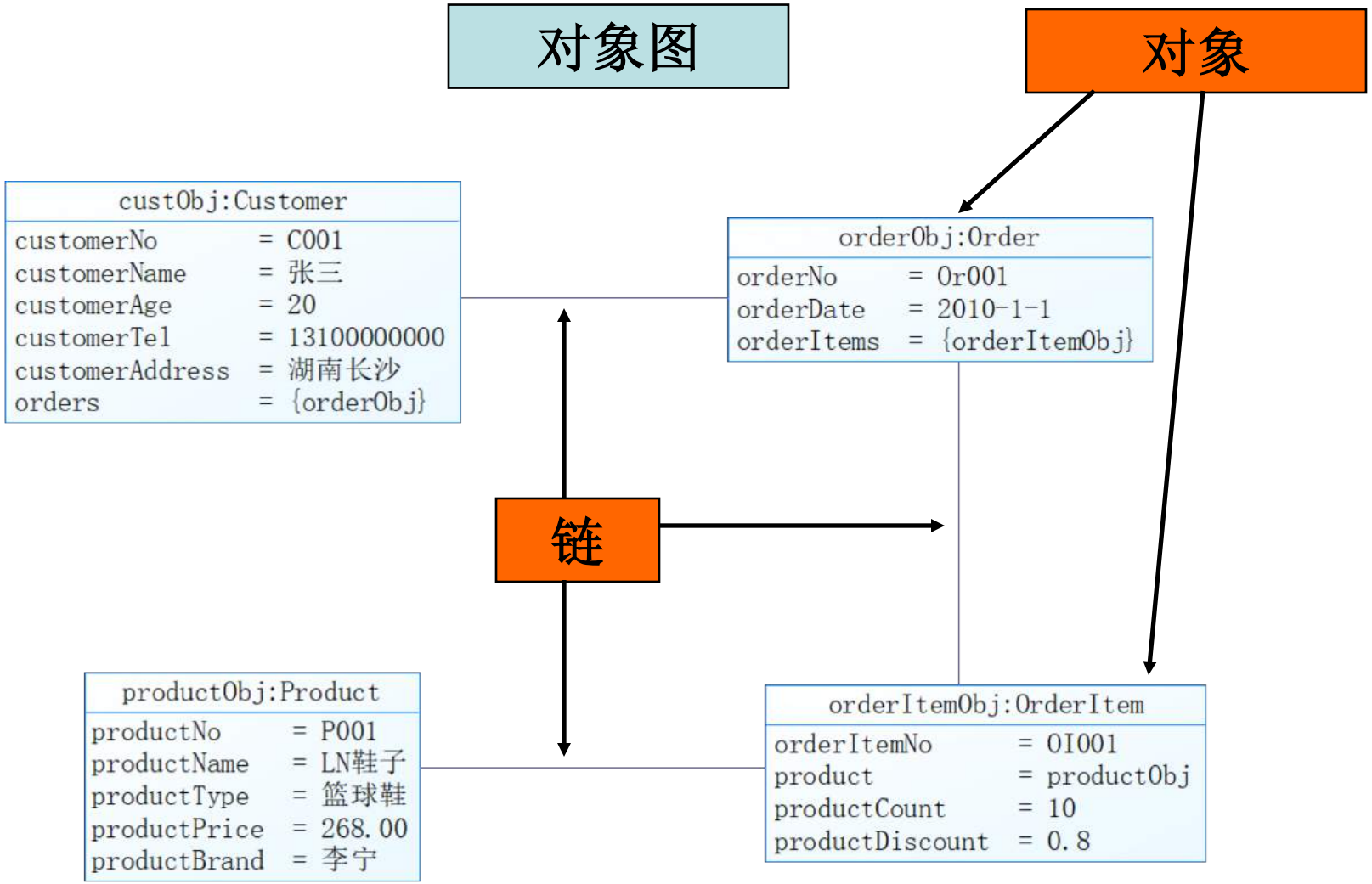
对象图

- 实例


类图
Customer
- customerNo : int
- customerName : String
- customerAge : int
- customerTel : String
- customerAddress : String
- orders : Order[]
1
0..*
Order
- orderNo : int
- orderDate : Date
- orderItems : OrderItem[]
1
1..*
Product
- productNo : int
- productName : String
- productType : String
- productPrice : float
- productBrand : String
1
1..*
OrderItem
- orderItemNo : int
- product : Product
- productCount : int
- productDiscount : float

对象图

- 实例


对象图
对象
custObj:Customer
customerNo = C001
customerName = 张三
customerAge = 20
customerTel = 13100000000
customerAddress = 湖南长沙
orders = {orderObj}
orderObj:Order
orderNo = Or001
orderDate = 2010-1-1
orderItems = {orderItemObj}
链
productObj:Product
productNo = P001
productName = LN鞋子
productType = 篮球鞋
productPrice = 268.00
productBrand = 李宁
orderItemObj:OrderItem
orderItemNo = OI001
product = productObj
productCount = 10
productDiscount = 0.8

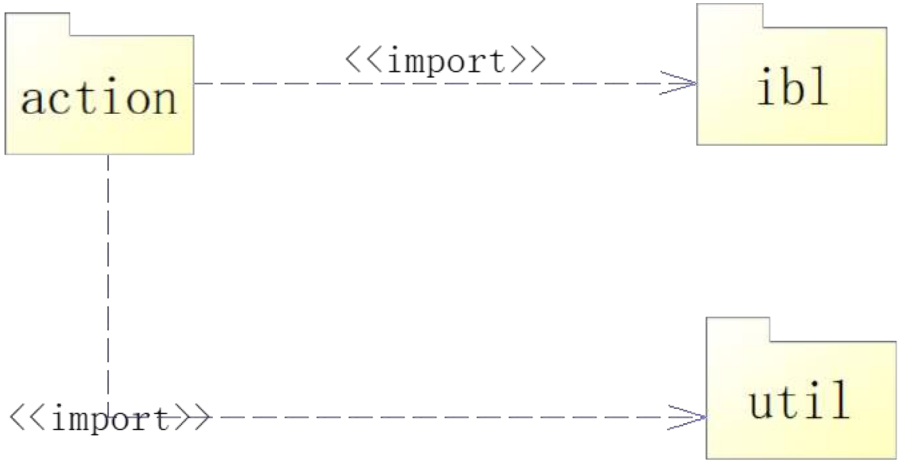
包图

- 概念
  - 包是一种把元素组织到一起的通用机制
  - 包可以嵌套于其他包中
  - 包图用于描述包与包之间的关系
  - 包的图标是一个带标签的文件夹



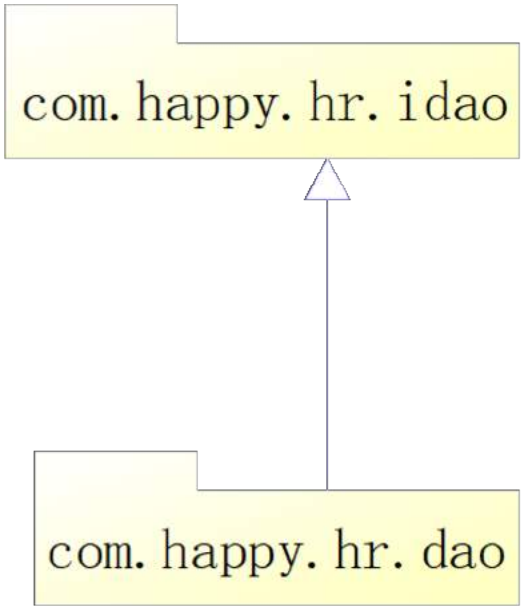
包图

- 关系
  - 引入关系：一个包中的类可以被另一个包中的类引用
  - 引入关系是依赖关系的一种，需要在依赖线上增加一个<<import>>衍型
  - 包之间一般依赖关系都属于引入关系



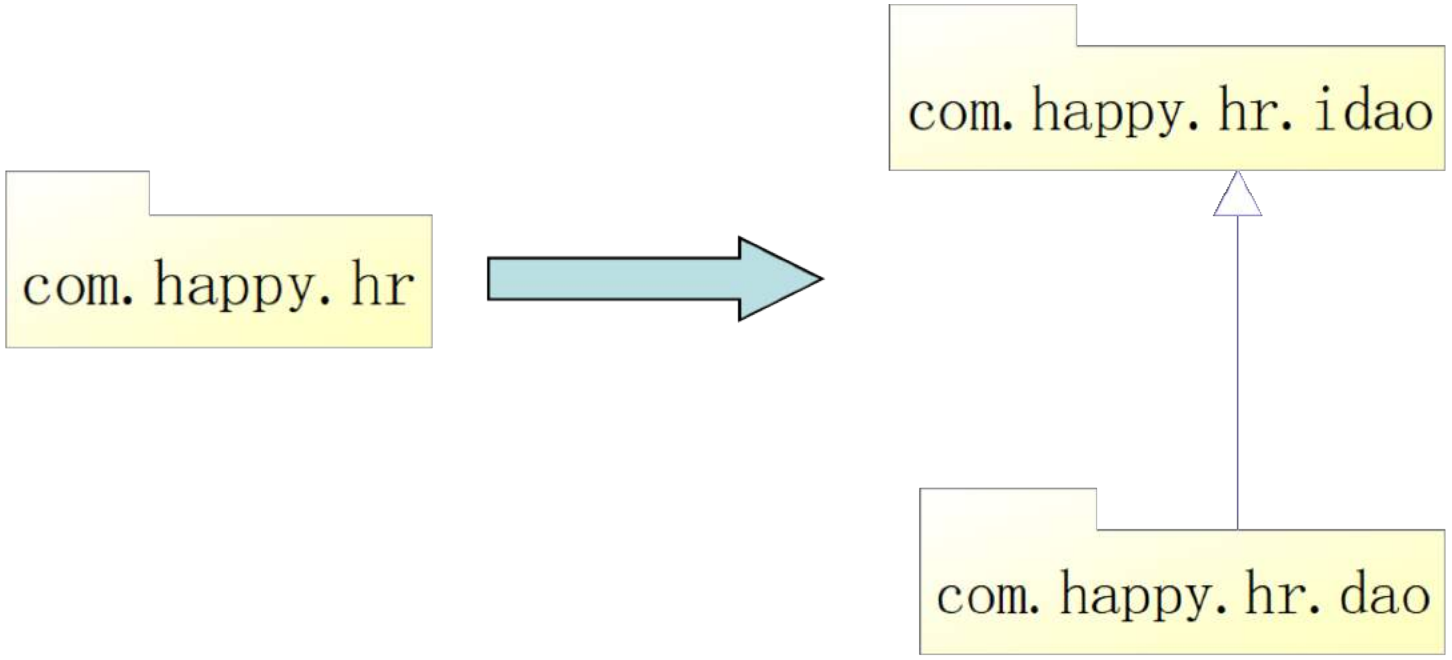
包图

- 关系
  - 泛化关系：表示一个包继承了另一个包的全部内容，同时又补充自己增加的内容。



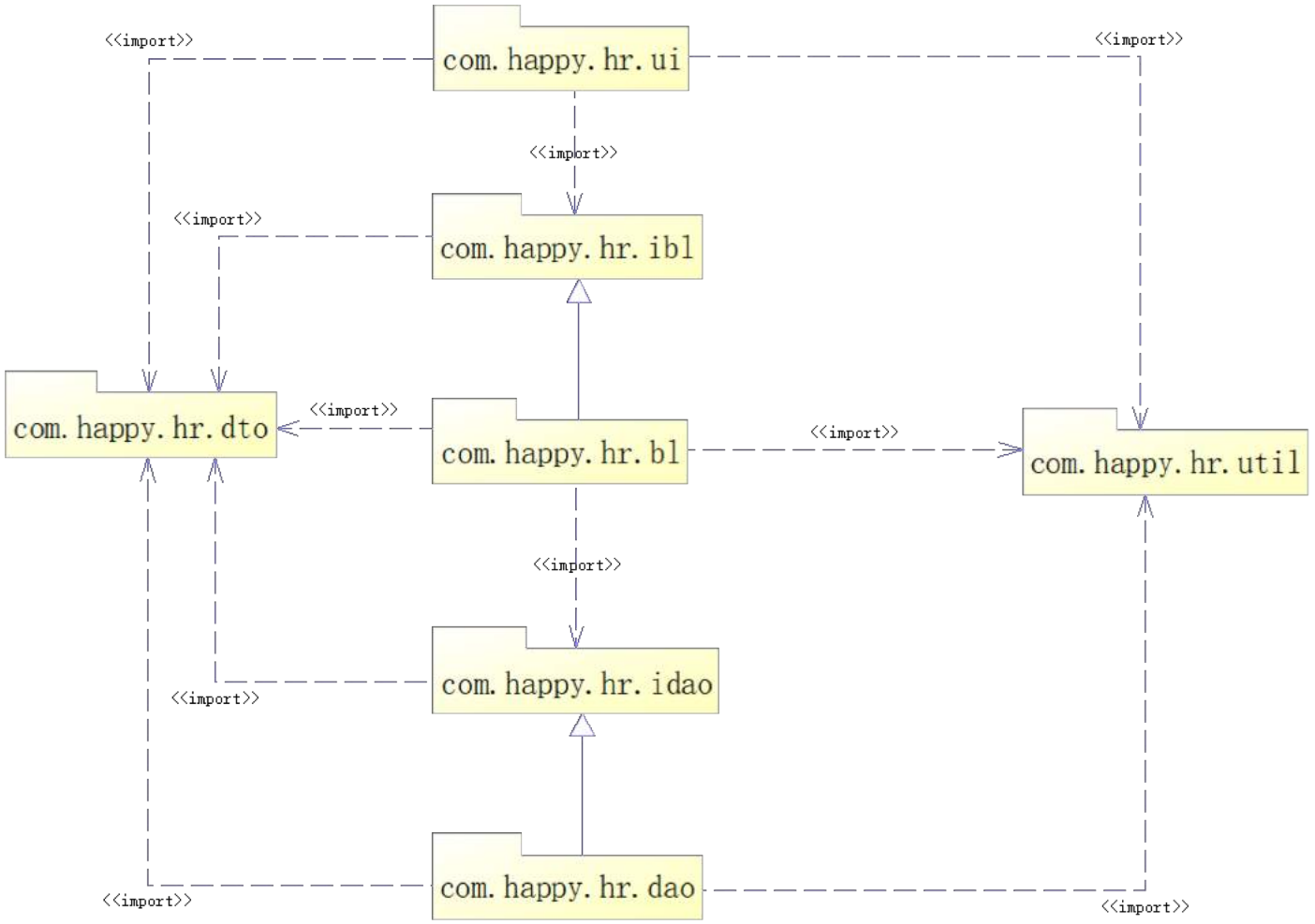
包图

- 包之间的关系
  - 嵌套关系：一个包中可以包含若干个子包，构成了包的嵌套层次结构。



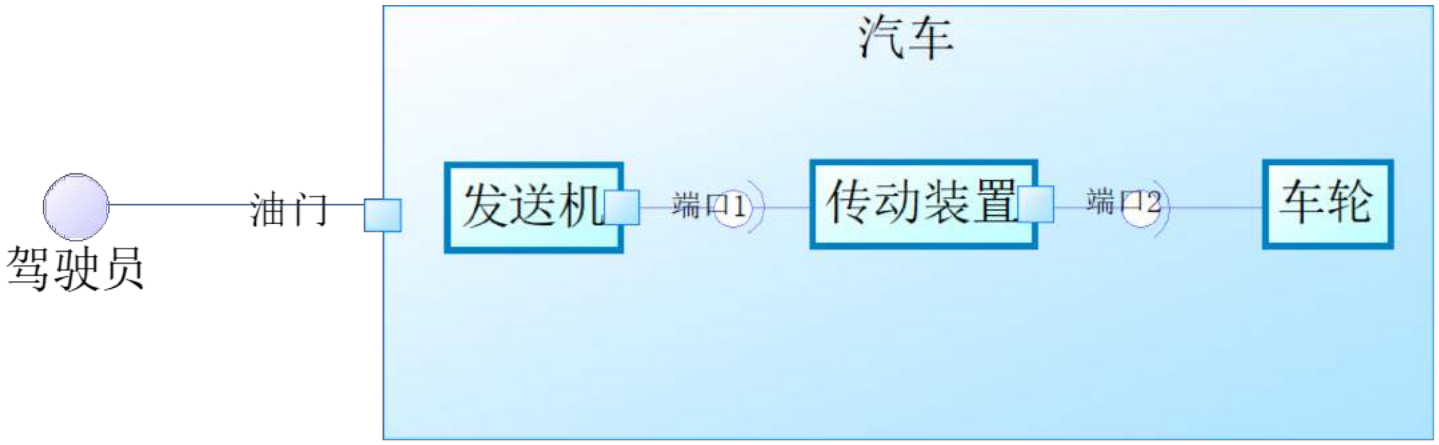
包图

- 实例：基于C/S的人力资源管理系统



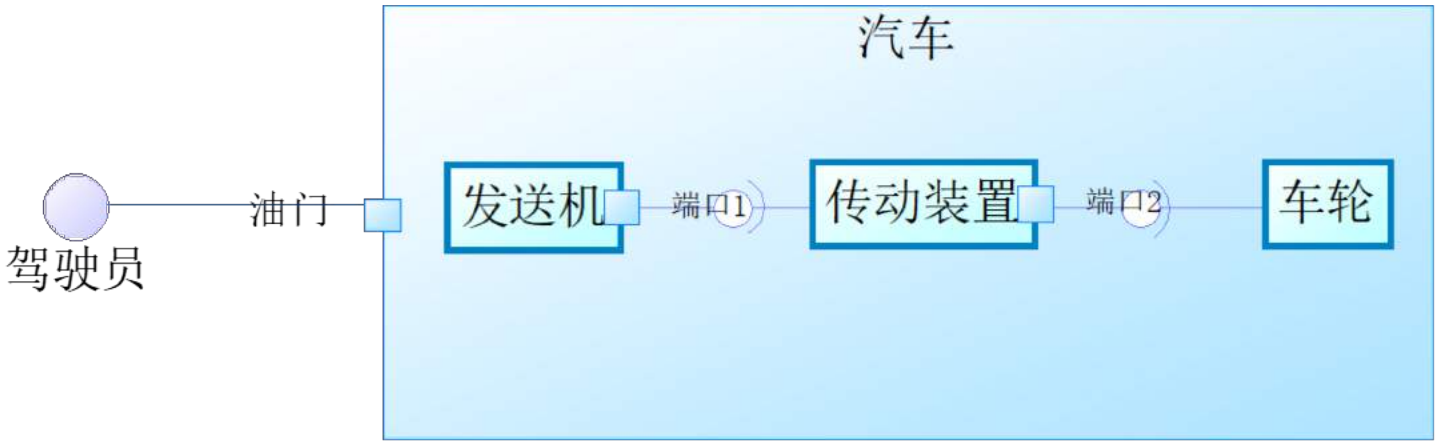
组合结构图

- 概述
  - 组合结构图将每一个类放在一个整体中，从类的内部结构来审视一个类
  - 组合结构图可用于表示一个类的内部结构



组合结构图

- 主要组成元素
  - 部件(Part)：表示被描述事物所拥有的内部成分
  - 连接件(Connector)：表示部件之间的关系
  - 端口(Port)：表示部件和外部环境的交互点

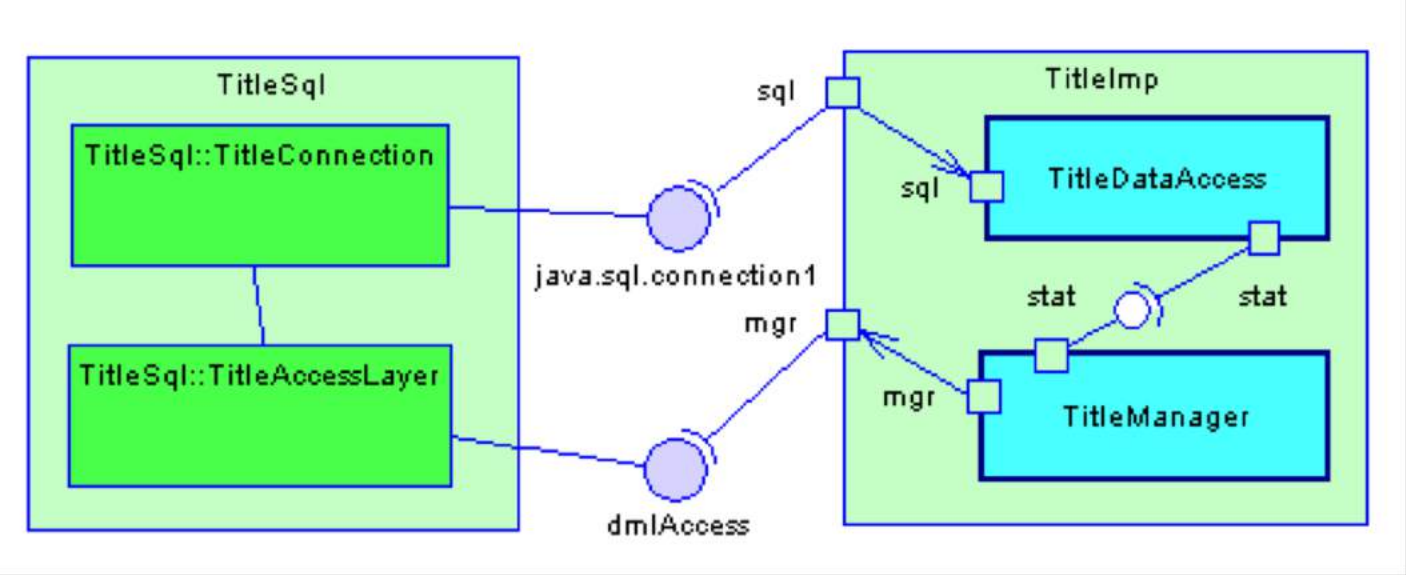


组合结构图

- 用途
  - 组合结构图所能够表达的信息，使用组合或者聚合也能够表示，只是一种新的表达形式
  - 组合结构图可以表示一个类的内部成员对象之间的相互关系，是对传统类图的一个补充

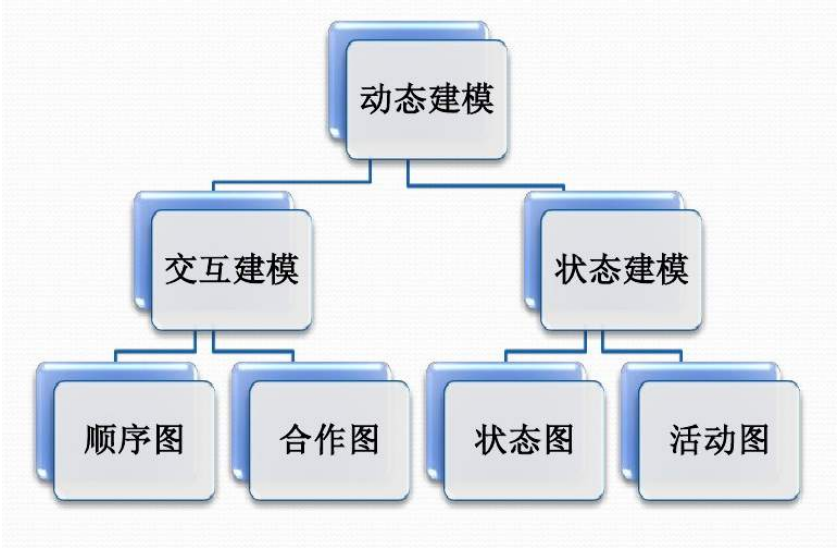
组合结构图

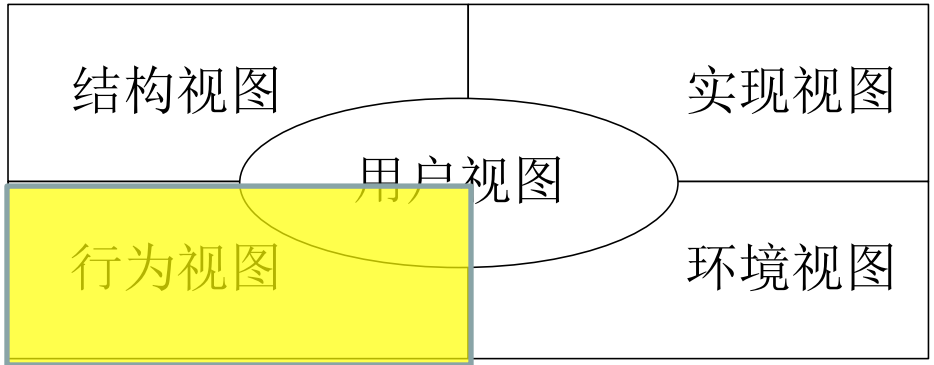
- 实例
  - TitleSql类包含两个内部类，TitleImp类包含两个部件，它们之间通过接口dmlAccess和java.sql.connection1产生连接。



第四部分

UML动态建模



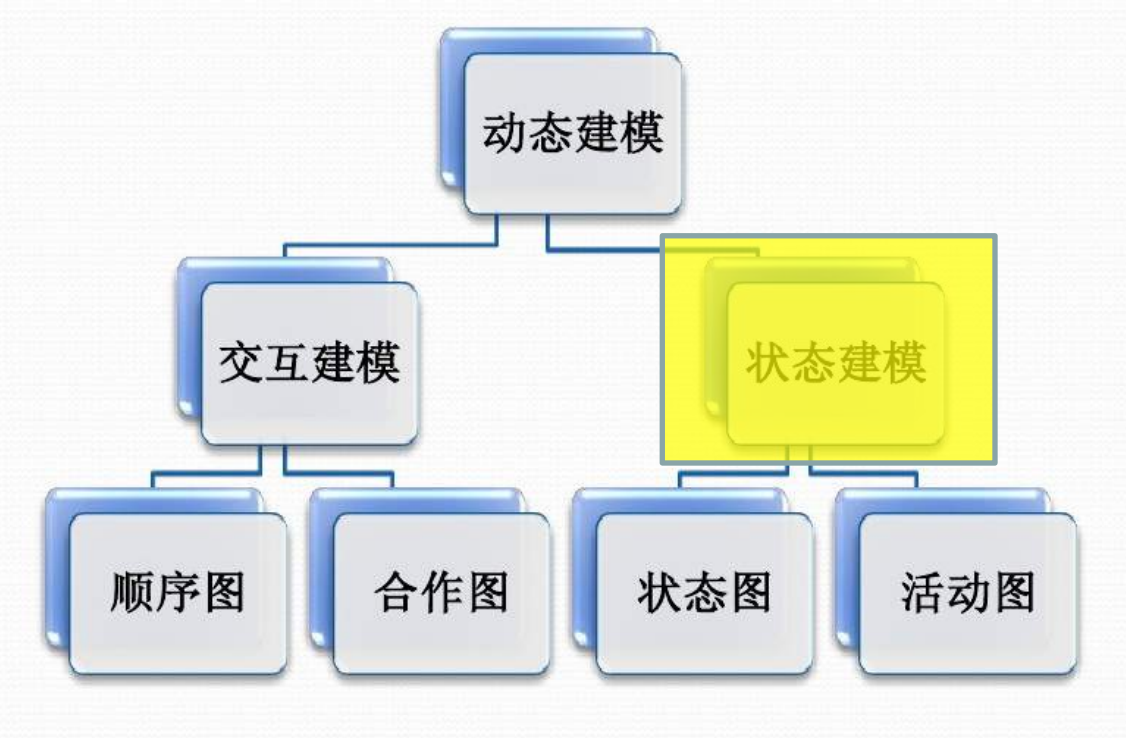


动态建模概述

- UML动态建模对应于系统的行为视图，它描述了软件系统模型的动态
- 系统中的对象能够相互通信，通信是通过调用这些对象的方法来实现的，把对象间的通信的方式称为系统的动态特征
- 系统中动态实体之间的通信，可以使用UML中的状态图、活动图、顺序图、合作图来表示

动态建模概述

- UML动态建模



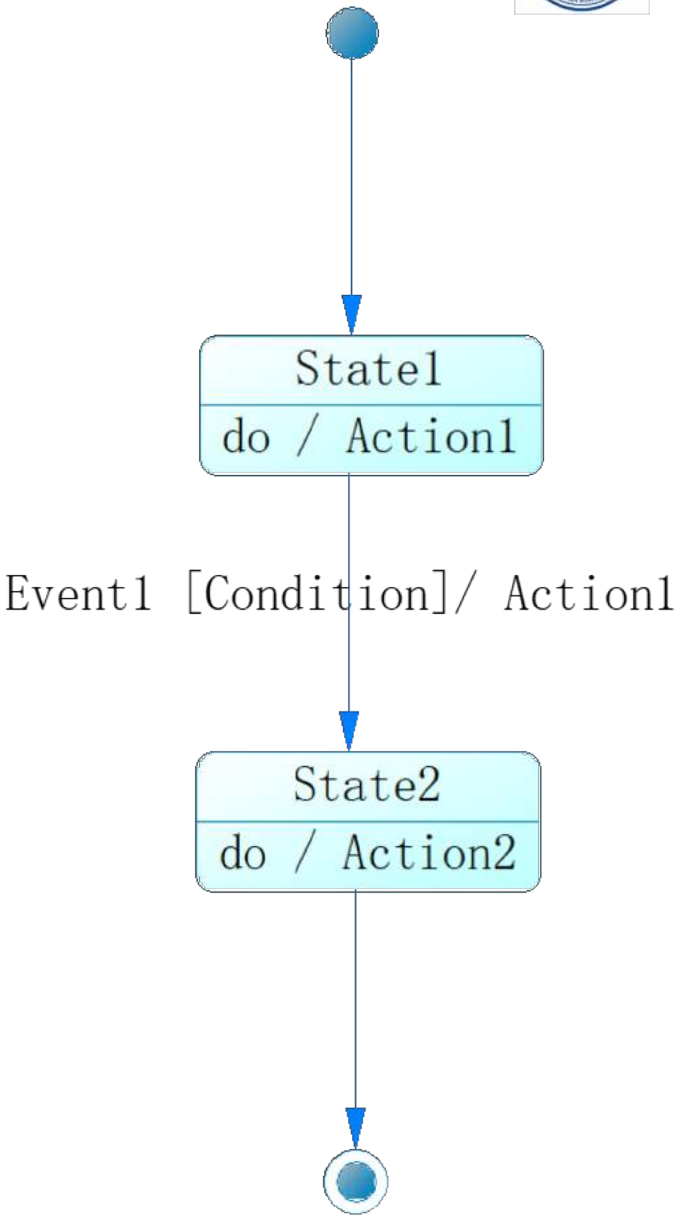
状态图

- 定义
  - 状态图（State Diagram）用来描述一个特定对象的所有可能状态及其引起状态转移的事件
  - 一个状态图包括一系列对象的状态及状态之间的转换
  - 大多数面向对象技术都使用状态图来描述一个对象在其生命周期中的行为

  - 并不是所有的类都需要使用状态图来描述它的行为。只有那些具有重要交互行为的类，才会使用状态图来描述

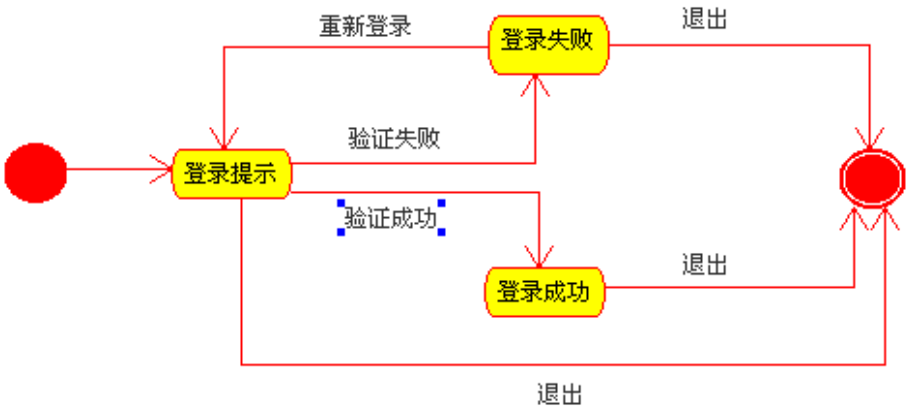
状态图

- 组成元素
  - 初始状态
  - 终止状态
  - 状态
  - 转移
  - 守护条件
  - 事件
  - 动作



状态图

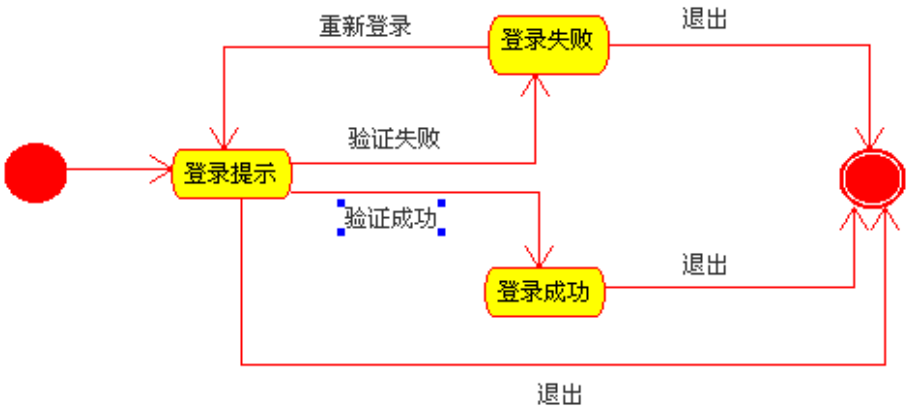
- 组成元素
  - 初始状态（Initial State）：表示对象创建时的状态，每一个状态图一般只有一个初始状态，用实心的圆点表示
  - 终止状态（Final State）：用一个实心圆外加一个圆圈表示，每一个状态图可能有多个终止状态
  - 状态（State）：表示状态名称和对应的系统活动



状态图

- 组成元素
  - 转移（Transition）：从一个状态到另一个状态的连线
  - 事件（Event）：对应系统的动作、条件或活动

  - 状态之间的转移可带有标注，其语法为事件名或动作名

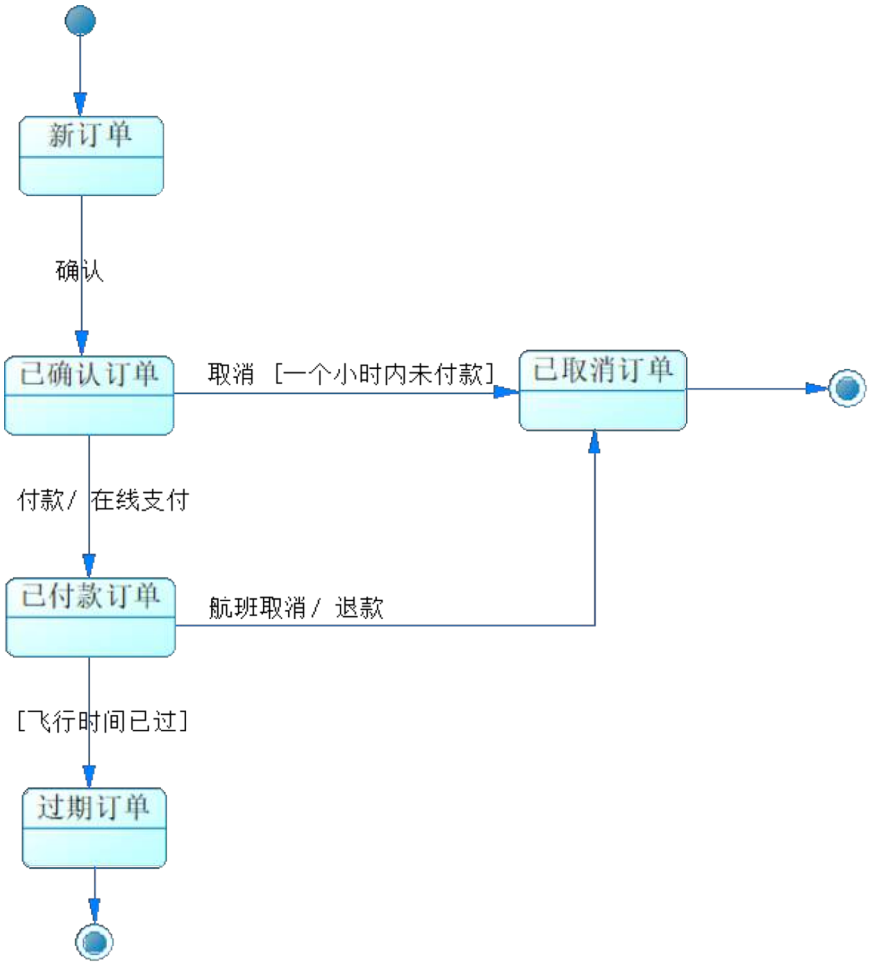


状态图

- 绘制技巧
  - 状态图适合用于表述在不同用例之间的对象行为，但并不适合于表述包括若干协作的对象行为
  - 系统设计中可能有多个对象，但并不需要给出每个对象的状态图，实际的做法是把注意力集中在整体系统或少数关键的对象上，特别是那些状态比较多的对象

状态图

- 实例：某航空订票系统中订单对象的状态图

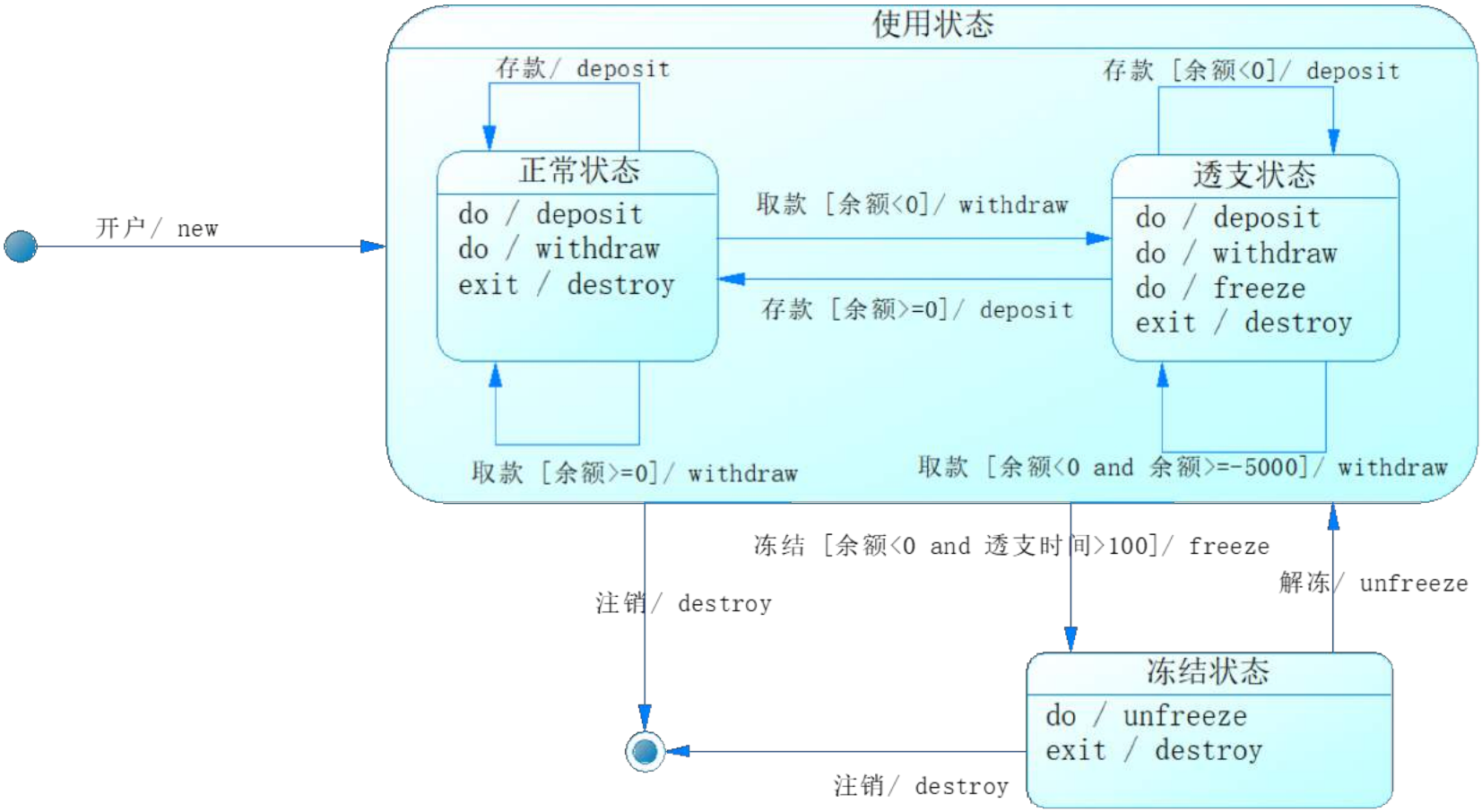


状态图

- 实例分析
  - 某信用卡系统账户具有使用状态和冻结状态，其中使用状态又包括正常状态和透支状态两种子状态。如果账户余额小于零则进入透支状态，透支状态时既可以存款又可以取款，但是透支金额不能超过5000元；如果余额大于零则进入正常状态，正常状态时既可以存款又可以取款；如果连续透支100天，则进入冻结状态，冻结状态下既不能存款也不能取款，必须要求银行工作人员解冻。用户可以在使用状态或冻结状态下请求注销账户。
  - 根据上述要求，绘制账户类的状态图

状态图

- 实例分析


使用状态
存款/ deposit
存款 [余额<0]/ deposit
正常状态
do / deposit
do / withdraw
exit / destroy
透支状态
do / deposit
do / withdraw
do / freeze
exit / destroy
开户/ new
取款 [余额<0]/ withdraw
存款 [余额>=0]/ deposit
取款 [余额>=0]/ withdraw
取款 [余额<0 and 余额>=-5000]/ withdraw
冻结 [余额<0 and 透支时间>100]/ freeze
解冻/ unfreeze
注销/ destroy
冻结状态
do / unfreeze
exit / destroy
注销/ destroy

活动图

- 定义
  - 活动图（Activity Diagram）用来表示系统中各种活动的次序。它既可用来描述用例的工作流程，也可以用来描述类中某个方法的操作行为
  - 活动图是由状态图变化而来的，它们各自用于不同的目的。活动图依据对象状态的变化来捕获动作（将要执行的工作或活动）与动作的结果。活动图中一个活动结束后将立即进入下一个活动
  - 活动图就是UML中的流程图

活动图

- 用途
  - 如果事件流的逻辑复杂且有许多其他事件流，文本形式可能较难阅读和理解，可使用活动图来描述事件流
  - 活动图用于以图形化的方式描述一个业务过程或者一个用例的活动的顺序流

活动图

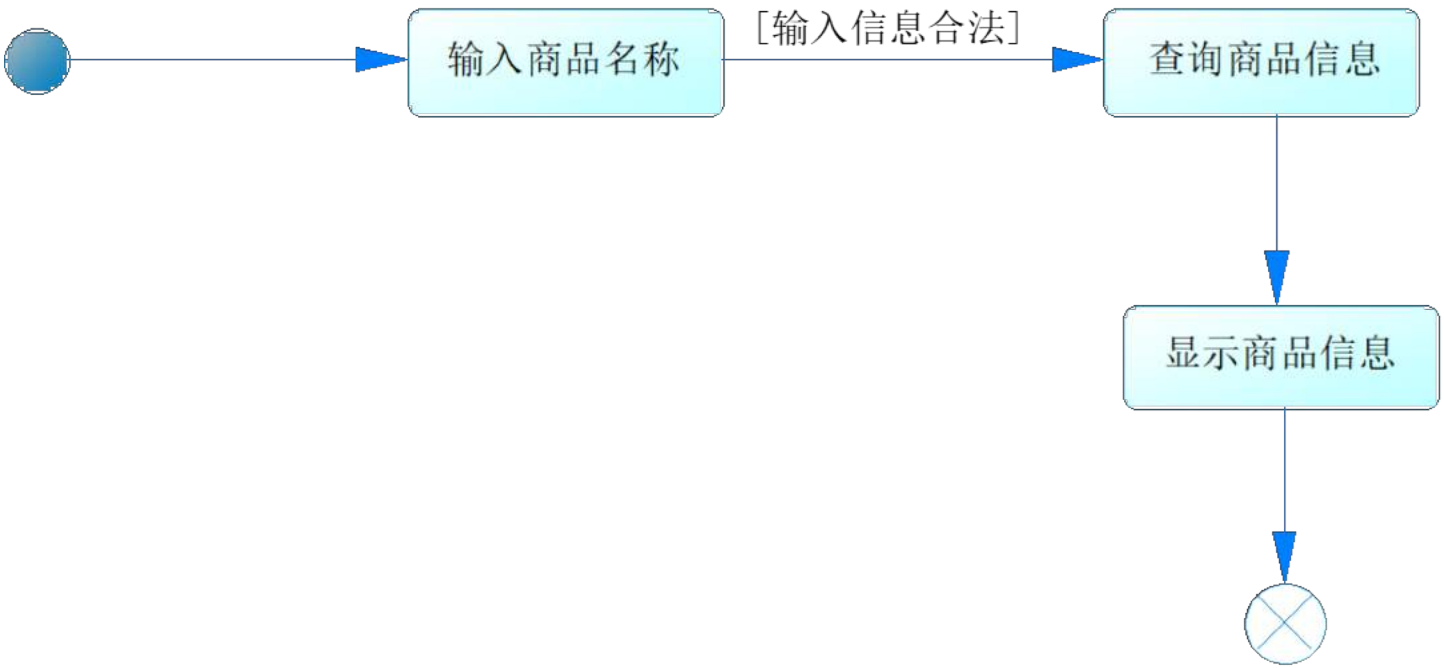
- 组成元素
  - 起始活动（Start Activity）
  - 终止活动（End Activity）
  - 活动（Activity）
  - 转移（Transition）
  - 决策（Decision）
  - 守护条件（Condition）
  - 同步条（Synchronization Bar）
  - 泳道（Swimlane）

活动图

- 组成元素
  - 起始活动：活动图工作流程的开始，用实心圆饼来表示，在一个活动图中，只有一个起始活动
  - 终止活动：活动图的终结活动，用实心圆点外加一个小圆圈来表示。一个活动图中可以有0个或多个终止活动
  - 活动：实现操作的一个步骤，用一个圆角矩形表示，其内部的文本串用来说明采取的动作。
  - 转移：转移是由事件的发生所引起的活动状态的改变，用带有箭头的实线表示。箭头上可能还带有守护条件，发送短句和动作表达式。

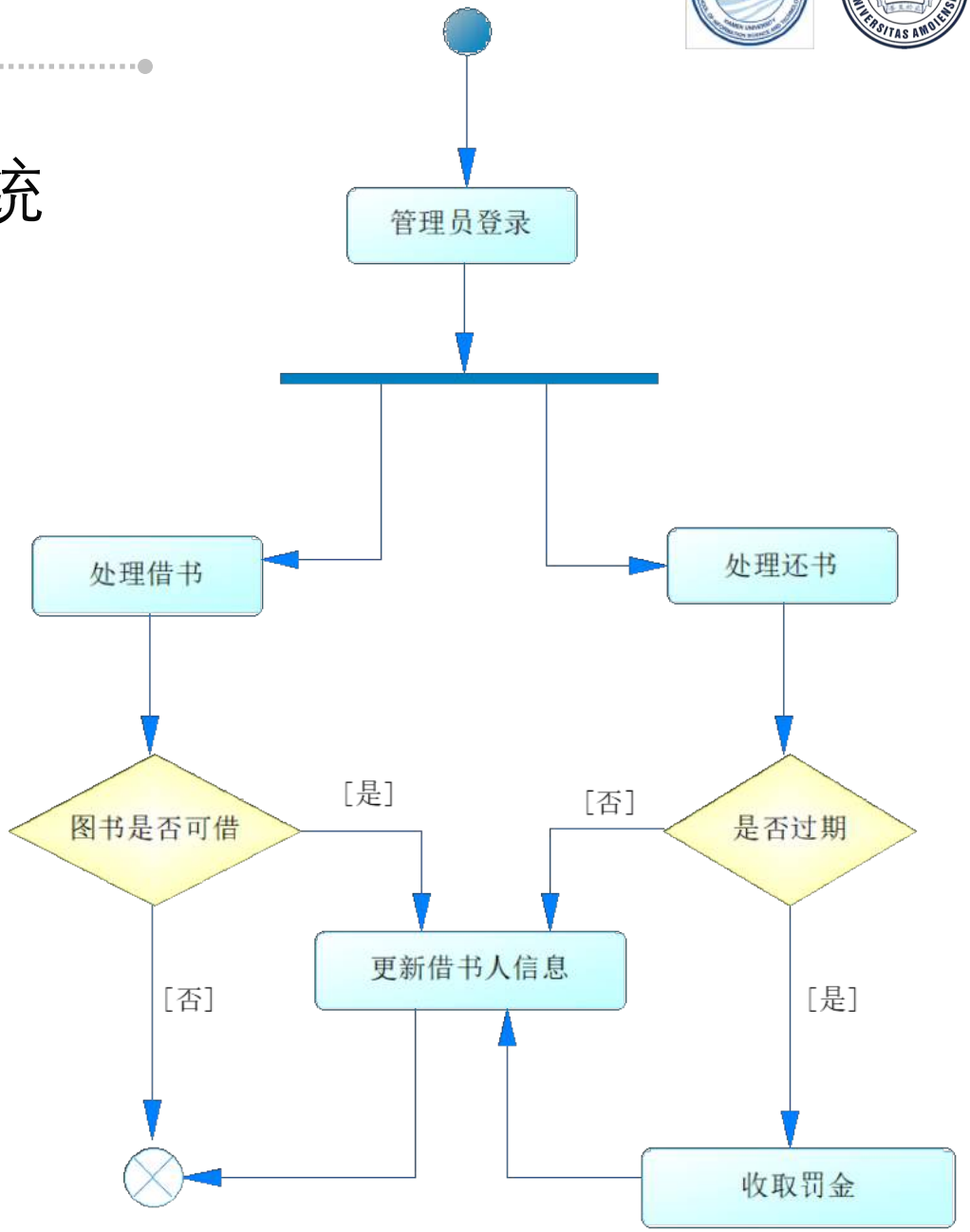
活动图

- 组成元素
  - 守护条件用来约束转移，守护条件为真时转移才可开始



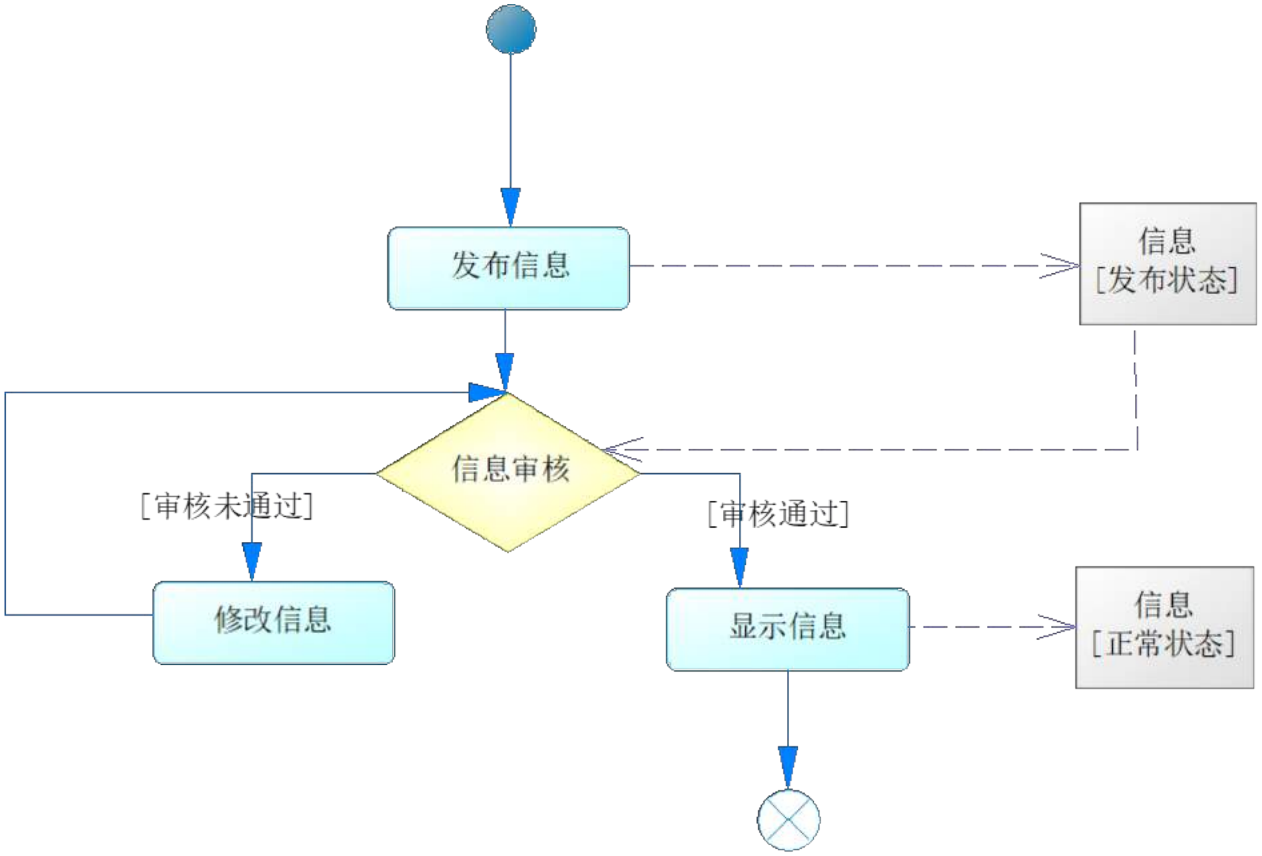
活动图

- 实例一：图书管理系统



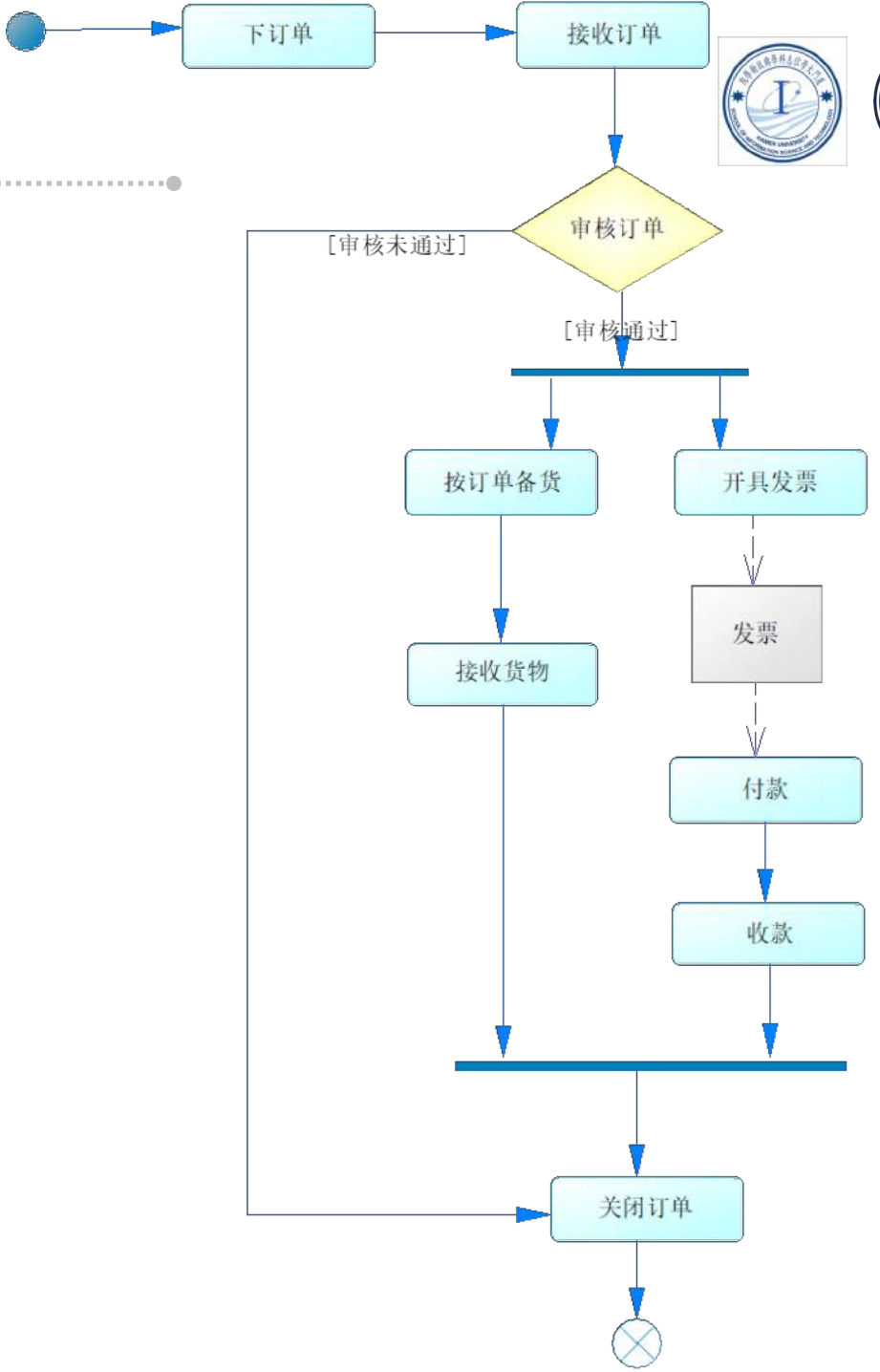
活动图

- 实例二



活动图

- 实例三

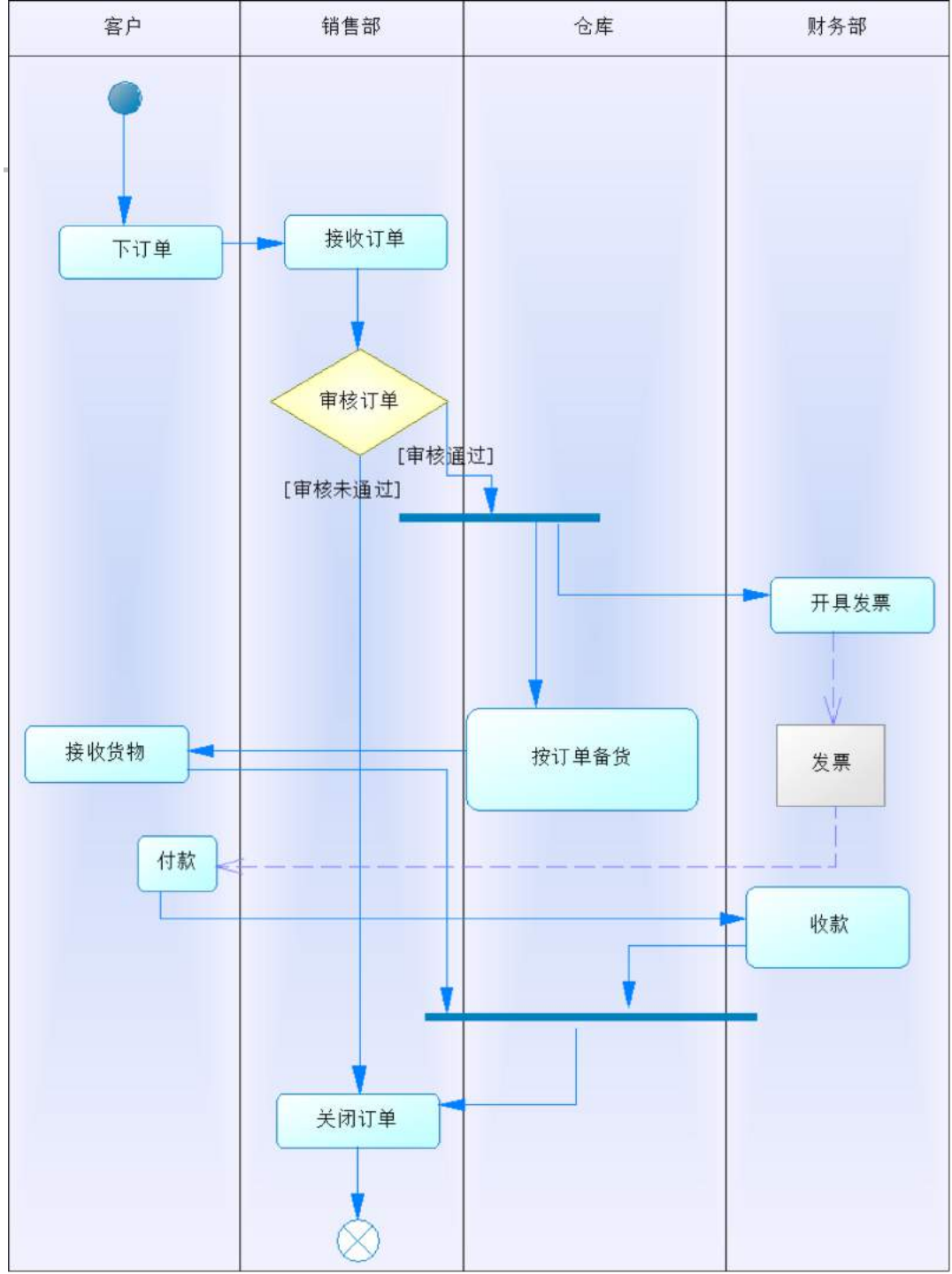


活动图

- 组成元素
  - 泳道用于分割活动图，有助于更好地理解执行活动的场所。泳道划分负责活动的对象，明确地表示哪些活动是由哪些对象进行的，每个活动只能明确地属于一个泳道
  - 在活动图中，泳道一般用垂直实线绘出，垂直线分隔的区域就是泳道

活动图

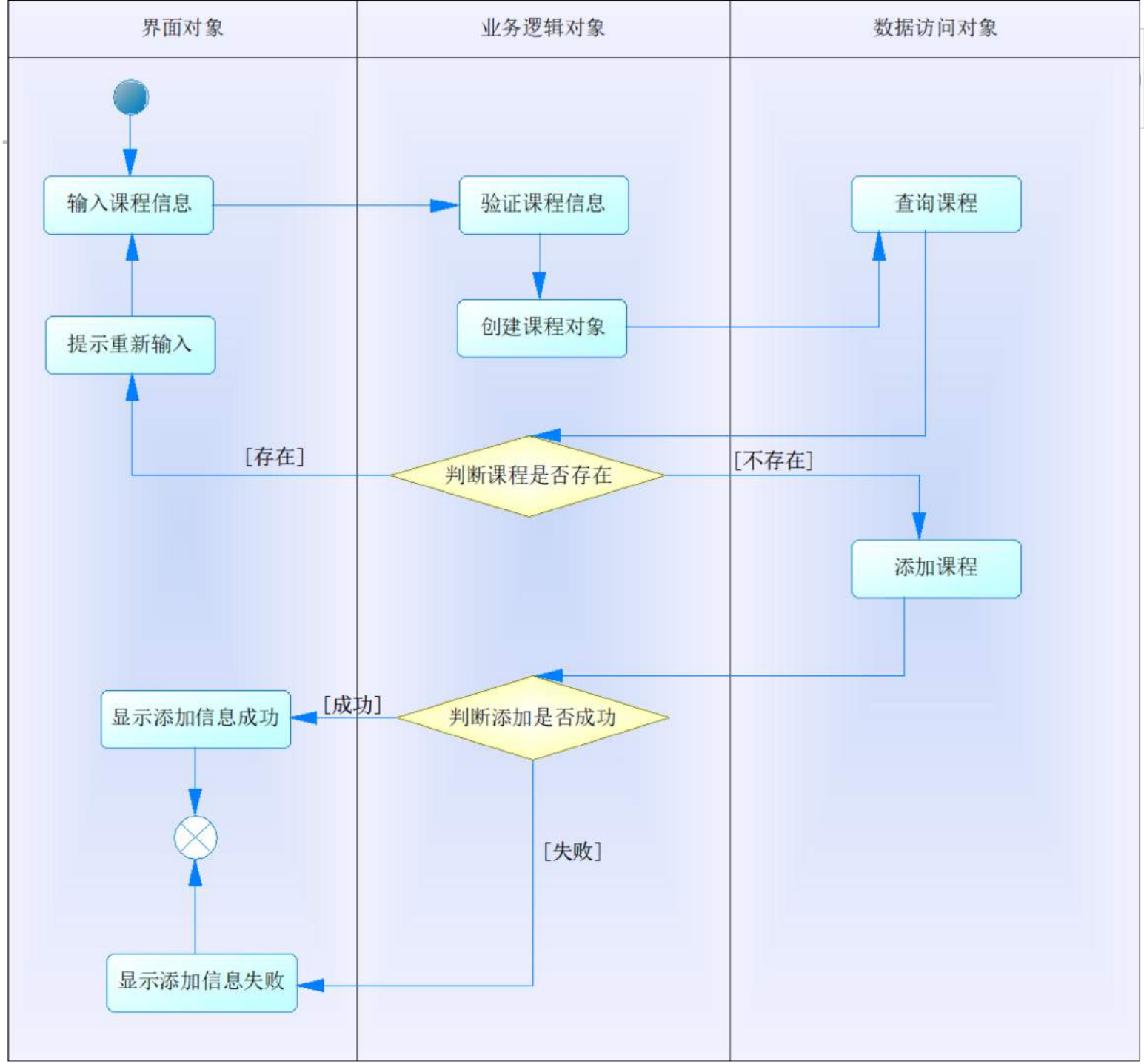
- 活动图组成元素


客户
销售部
仓库
财务部
下订单
接收订单
审核订单
[审核通过]
[审核未通过]
开具发票
接收货物
按订单备货
发票
付款
收款
关闭订单

活动图

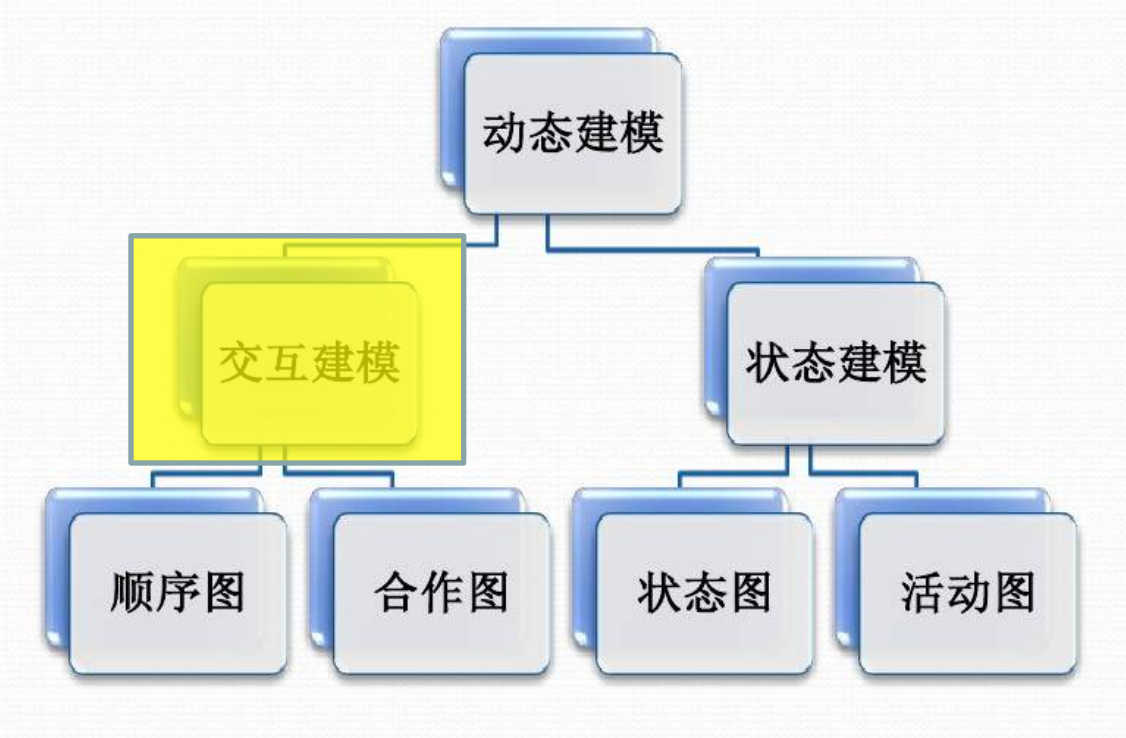
- 实例分析
  - 某教学管理系统增加课程信息用例路径如下，绘制相应的活动图

基本流程	1．管理员选择进入课程管理界面，用例开始。 2．管理员输入课程信息。 3．系统验证是否和已有课程信息冲突。 4．系统添加新课程，提示课程添加成功。 5．系统显示课程添加成功。 6．用例结束。
替代流程	3a. 验证失败 1．系统提示重新输入课程信息。 2．回到主流程第2步，管理员输入课程信息，直到成功为止。 4a. 添加课程失败 1．系统显示课程添加失败。 2．用例结束。


界面对象
业务逻辑对象
数据访问对象
输入课程信息
验证课程信息
查询课程
提示重新输入
创建课程对象
[存在]
判断课程是否存在
[不存在]
添加课程
显示添加信息成功
[成功]
判断添加是否成功
[失败]
显示添加信息失败

动态建模概述

- UML动态建模



交互图

- 在一个待开发的系统中，任何对象都不是孤立存在的，如果对象独立则失去存在的意义，系统中对象之间通过消息的传递来进行交互。因此在设计系统时，需要使用交互图对对象之间的交互进行建模
- 一次交互是指在特定语境下，为了实现某个目标，而在一组对象之间进行交换的一组消息所表示的行为。通常，消息的传递通过方法的调用来实现。

交互图

- 定义
  - 交互图（Interactive Diagram）是表示各对象如何依据某种行为进行协作的模型
  - 通常可以使用一个交互图来表示和说明一个用例的行为
  - 在 UML 中，主要包括两种不同形式的交互图
    - 顺序图
    - 通信图

顺序图

- 定义
  - 顺序图（Sequence Diagram）是一种强调信息时序的交互图。它描述类系统中类与类之间的交互，并将这些交互建模成消息互换
  - 顺序图由执行者（actor）、对象（object）、消息（message）、生命线（lifeline）、激活（activation）等组成。顺序图描述对象之间的动态交互关系，着重体现对象间消息传递的时间顺序。

顺序图

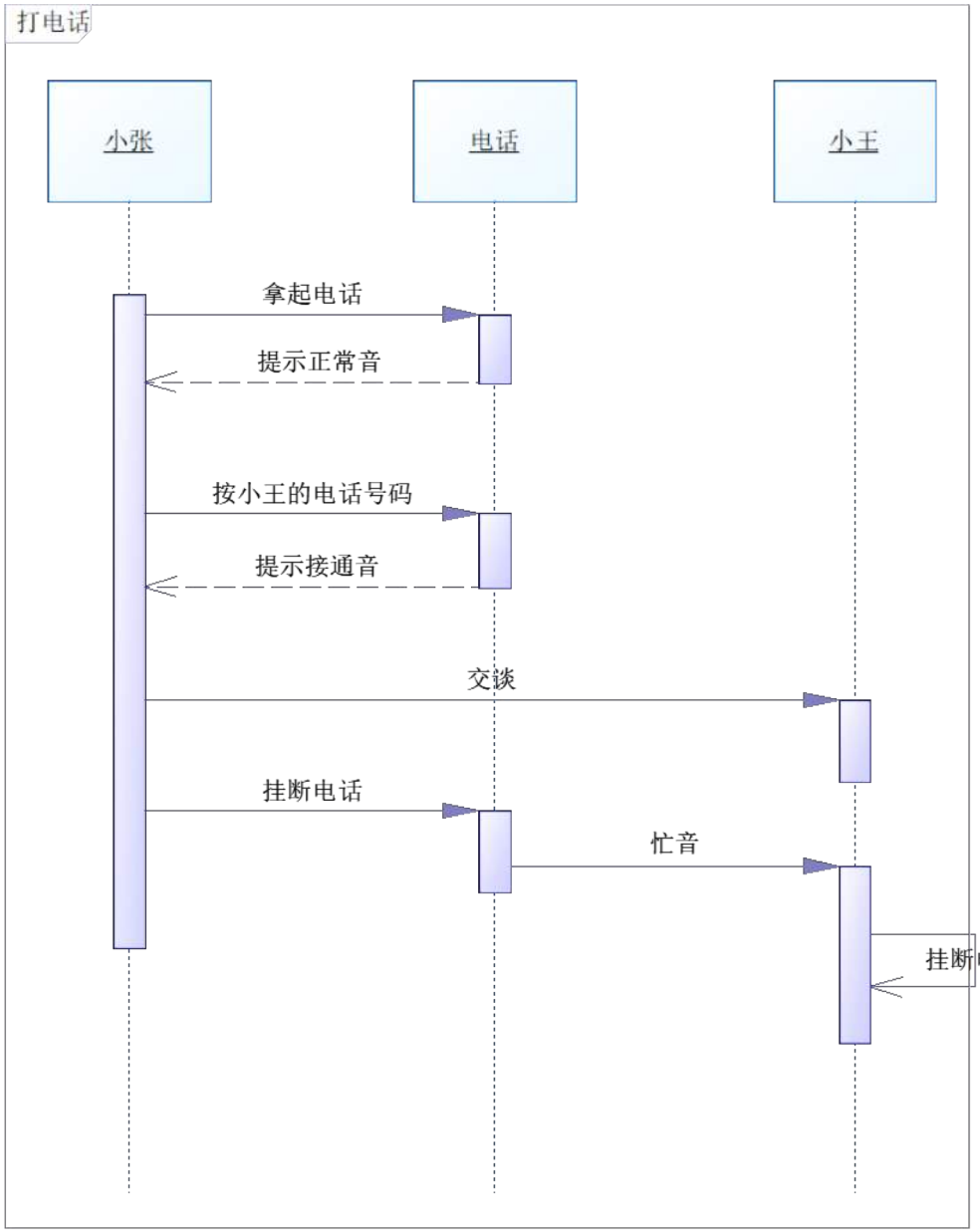
- 用途
  - 顺序图供不同类型的用户使用：用户可以从顺序图中看到业务过程的细节；分析人员可以从顺序图中看到处理流程；开发人员可以看到需要开发的对象和对这些对象的操作；质量保证工程师可以看到过程的细节，并根据这个过程开发测试案例，顺序图对项目的各方面人员都有用。

顺序图

- 组成元素
  - 顺序图将交互关系表现为一个二维图，纵向是时间轴，时间沿竖线向下延伸。横向轴代表了在协作中各独立对象的类元角色，类元角色的活动用生命线表示。

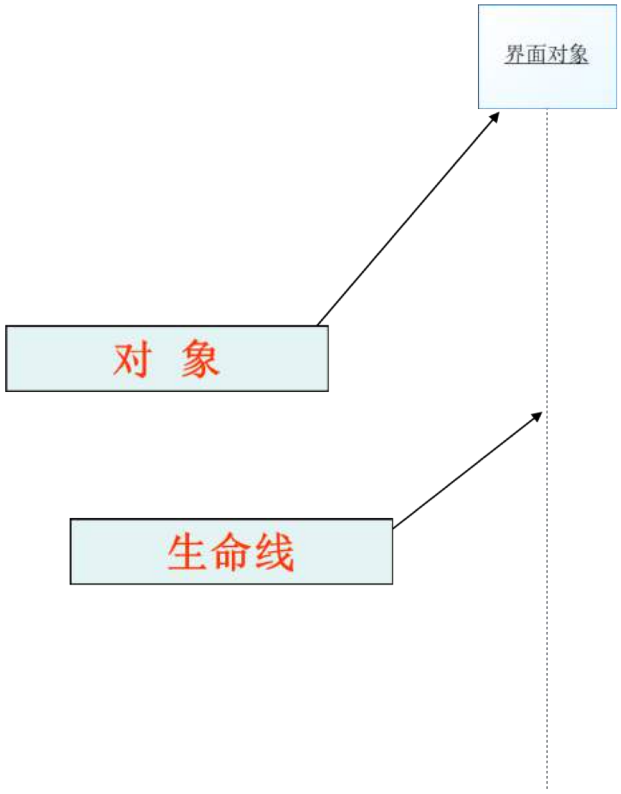
顺序图

- 示例



顺序图

- 组成元素
  - 生命线用一条纵向虚线表示
  - 对象表示为一个矩形，其中对象名称标有下划线

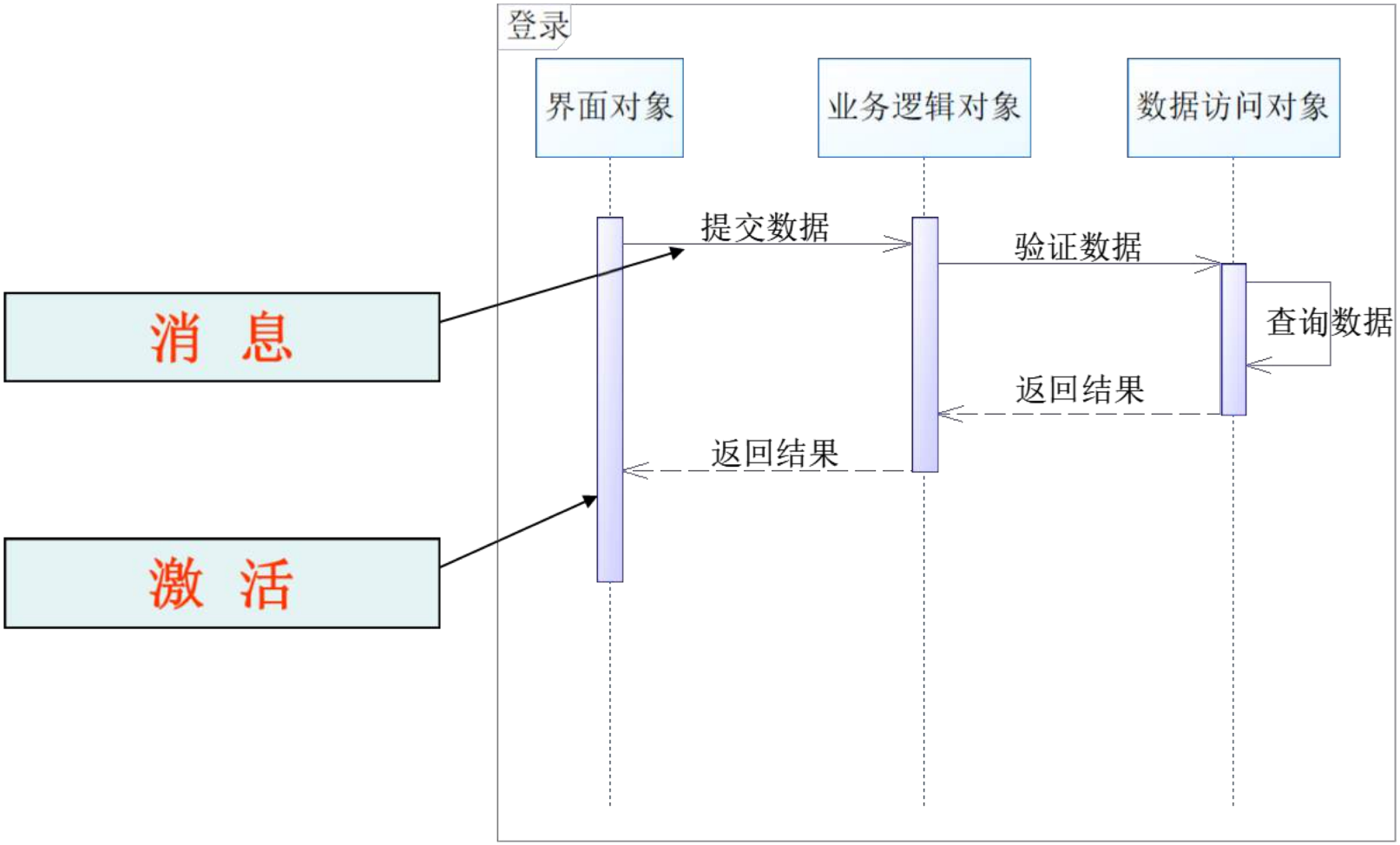


顺序图

- 组成元素
  - 激活：过程的执行。在顺序图中激活部分替换生命线，使用长条的矩形表示。
  - 消息：对象之间的通信，是两个对象之间的单路通信，是从发送者到接收者之间的控制信息流。消息在顺序图中由有标记的箭头表示，箭头从一个对象的生命线指向另一个对象的生命线，消息按时间顺序在图中从上到下排列。

顺序图

- 顺序图组成元素

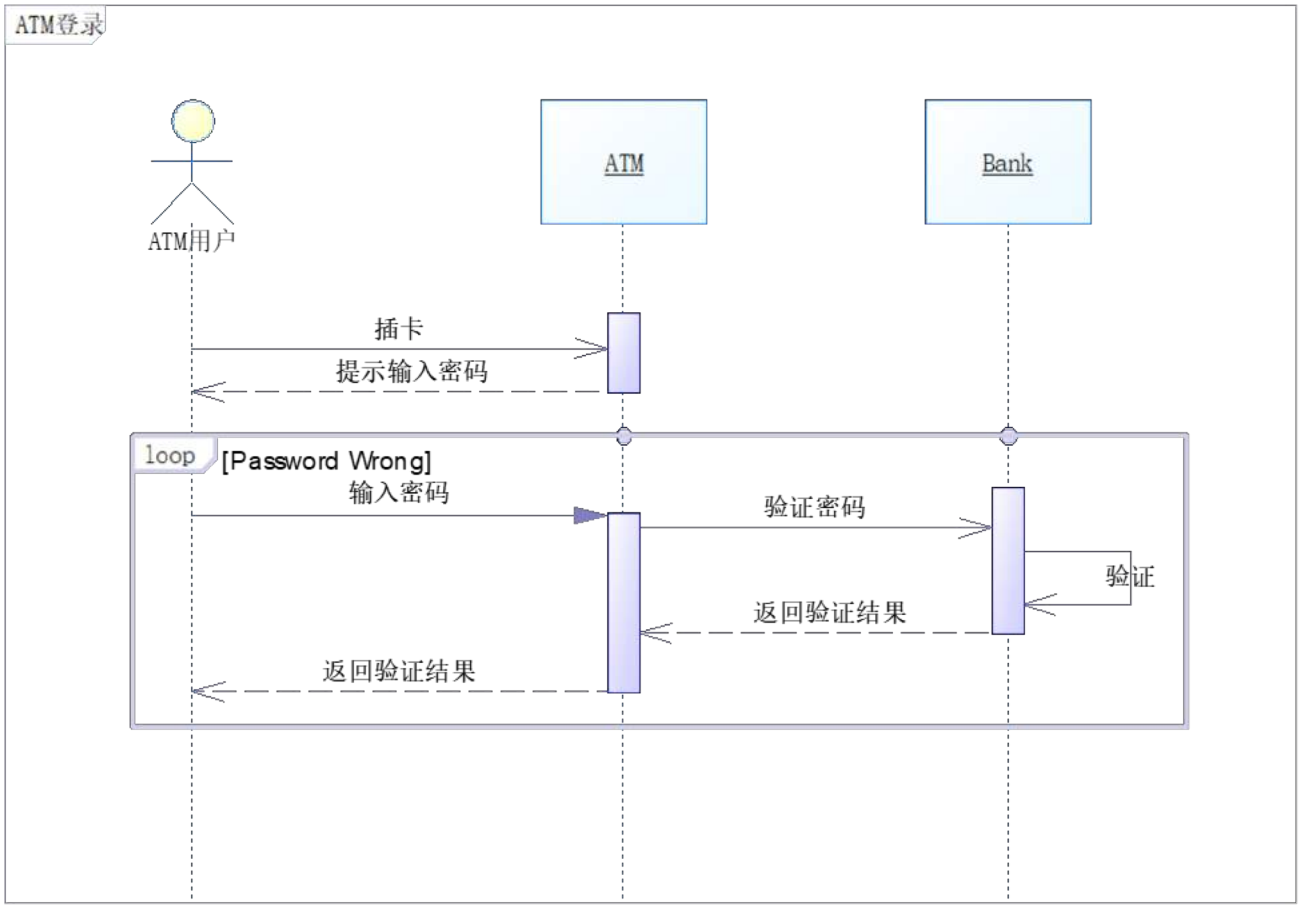


顺序图

- 顺序图组成元素
  - 对象：安排在X轴。启动交互的对象放在最左边，随后放入消息的对象放在启动交互对象的右边。交互中对象发送和接收的消息沿着Y轴以时间增加的次序放置。在顺序图中，有的消息对应于激活，表示它将会激活一个对象，这种消息称为调用消息(Call Message)；如果消息没有对应激活框，表示它不是一个调用消息，不会引发其他对象的活动，这种消息称为发送消息(Send Message)。

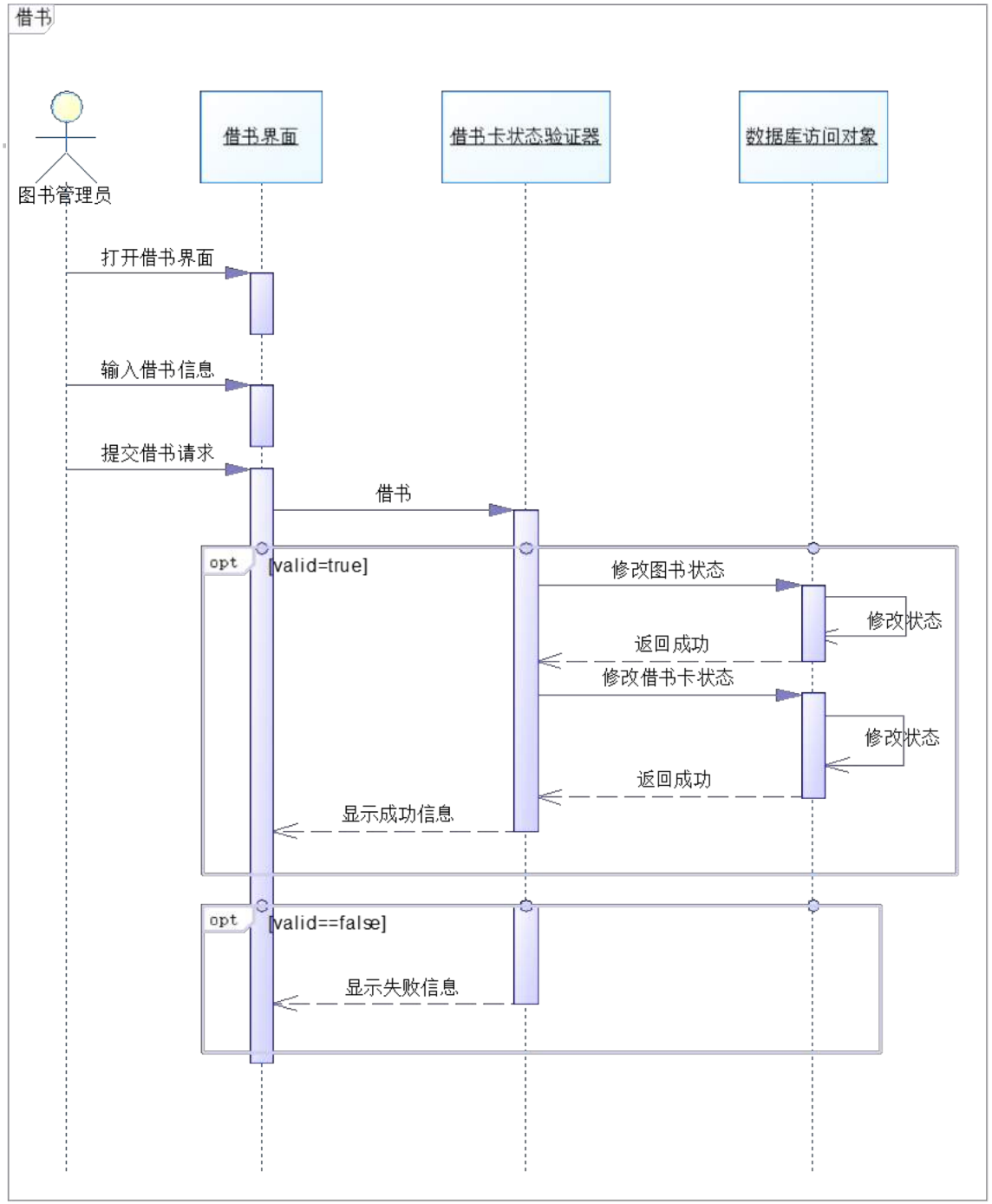
顺序图

- 实例：ATM登录


ATM登录
ATM用户
ATM
Bank
插卡
提示输入密码
loop
[Password Wrong]
输入密码
验证密码
验证
返回验证结果
返回验证结果

顺序图

- 实例分析：图书管理系统
  - 图书管理员打开借书界面，输入借书信息并提交借书请求；系统验证借书卡状态，如果借书卡未借书则记录借书信息且修改图书状态和借书卡状态，并提示借书成功；否则提示借书失败。
  - 绘制图书管理系统“借书”用例的顺序图

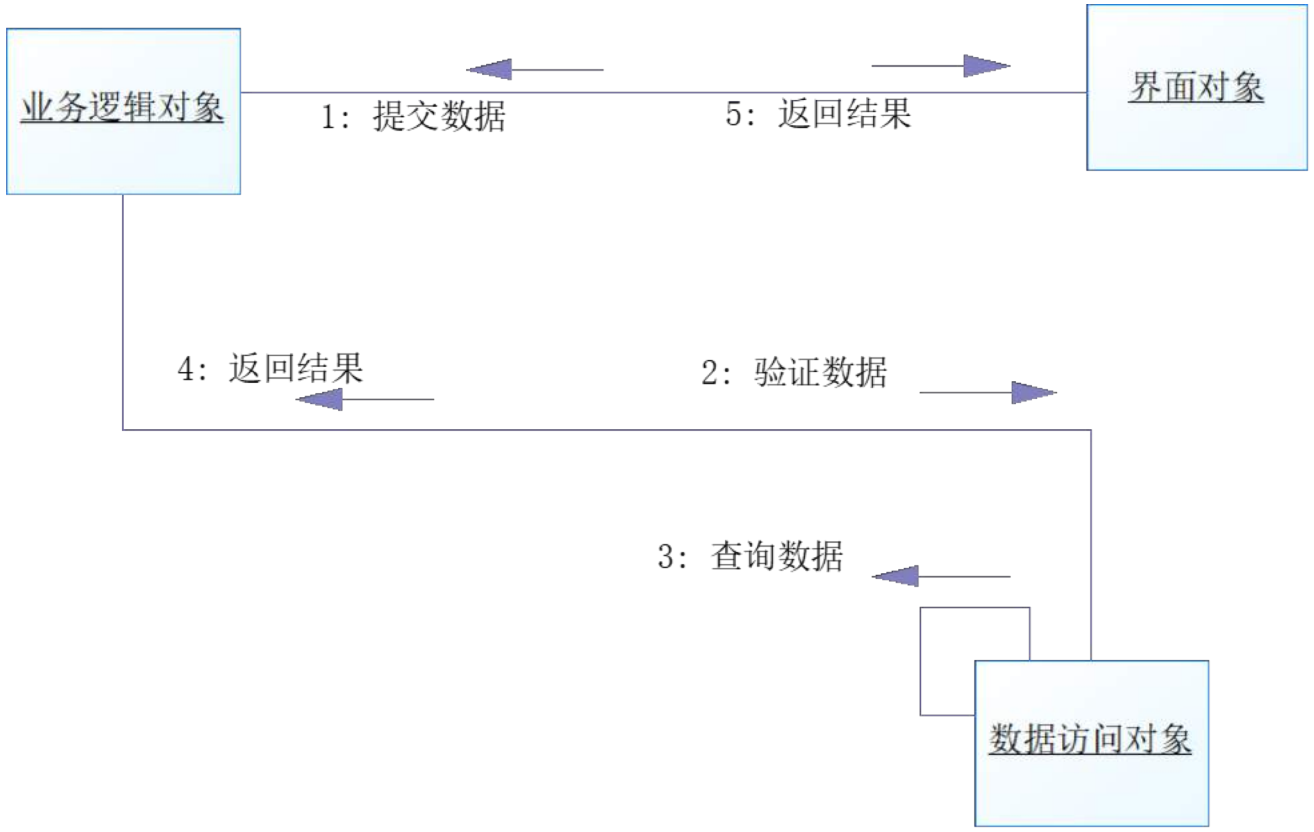

借书
图书管理员
借书界面
借书卡状态验证器
数据库访问对象
打开借书界面
输入借书信息
提交借书请求
借书
opt
[valid=true]
修改图书状态
修改状态
返回成功
修改借书卡状态
修改状态
返回成功
显示成功信息
opt
[valid==false]
显示失败信息

通信图

- 概述
  - 通信图强调参与一个交互对象的组织。它与顺序图是同构图，也就是它们包含了相同的信息，只是表达方式不同而已，通信图与顺序图可以相互转换。
  - 虽然通信图和顺序图均显示了交互，但它们强调了不同的方面。顺序图清晰地显示了时间次序，但没有显式指明对象间关系；通信图清晰地显示了对象间关系，但时间次序必须从顺序号来获得。
  - 顺序图常用于用例场景描述，通信图更适合显示过程设计细节。

通信图

- 实例

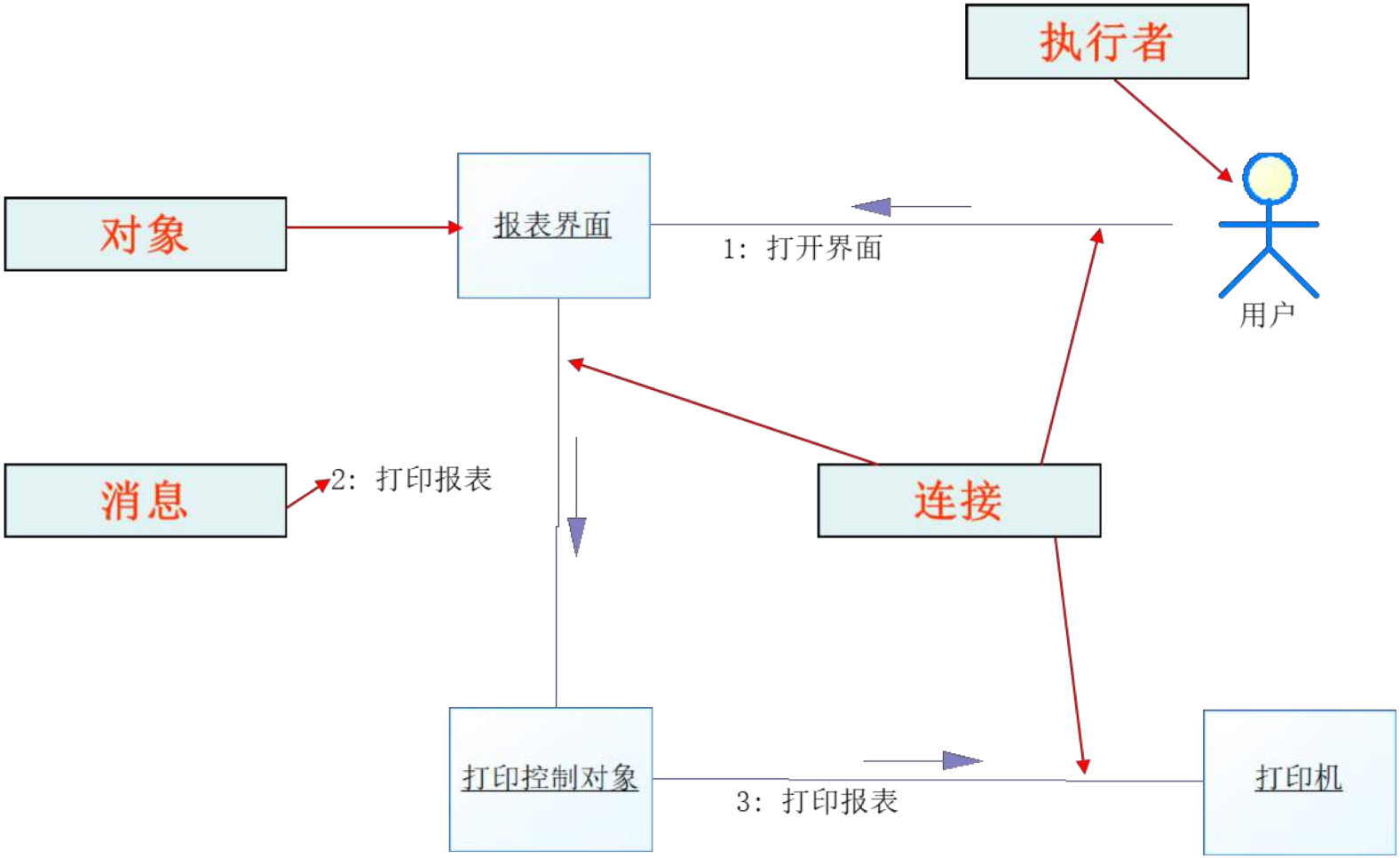


通信图

- 组成元素
  - 由以下基本元素组成：执行者（actor）、对象（object）、连接(link，也称为链)和消息(message)。
  - 在UML中，使用实线表示两个对象之间的连接；通信图中的消息，由在连接上方的带有标记的箭头表示，同时可以用数字注明消息的次序

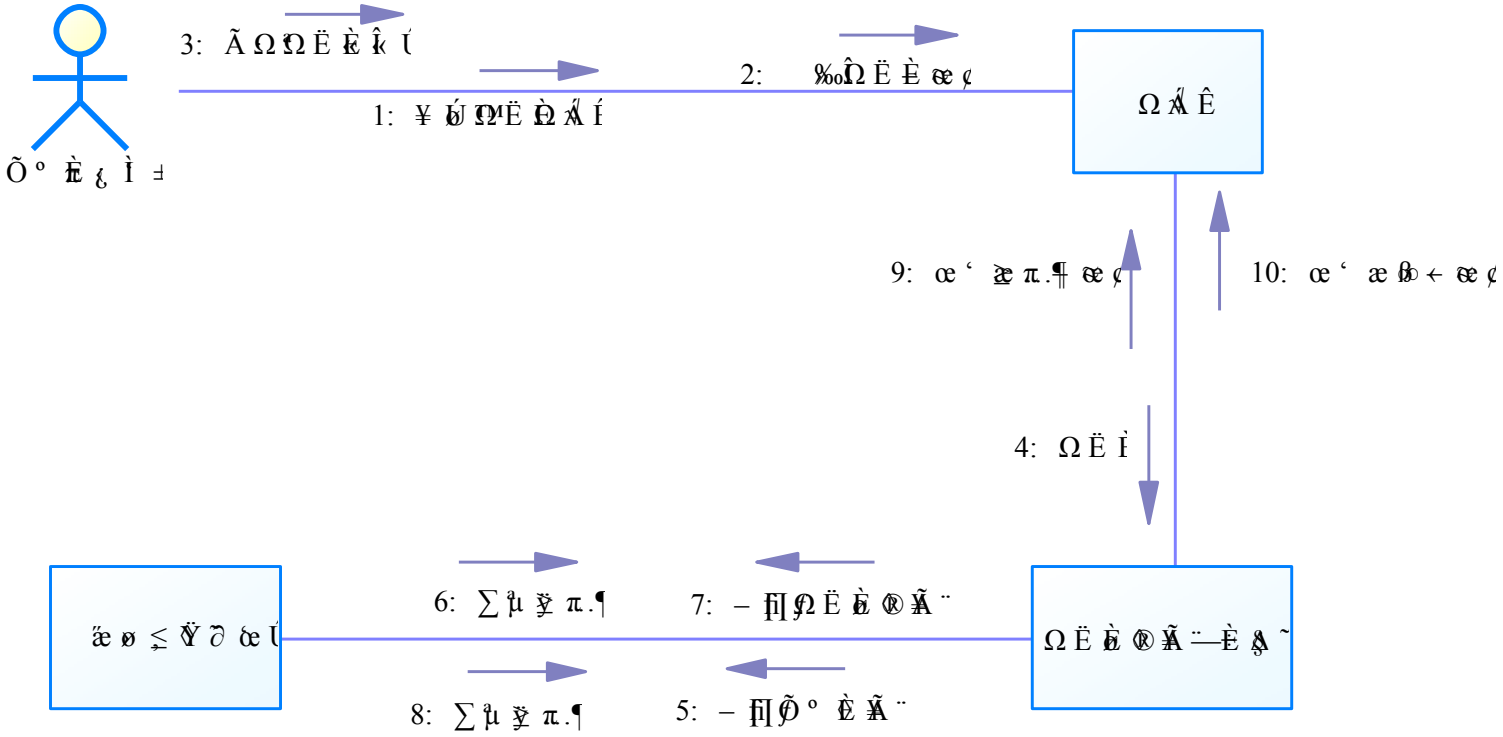
通信图

- 通信图组成元素



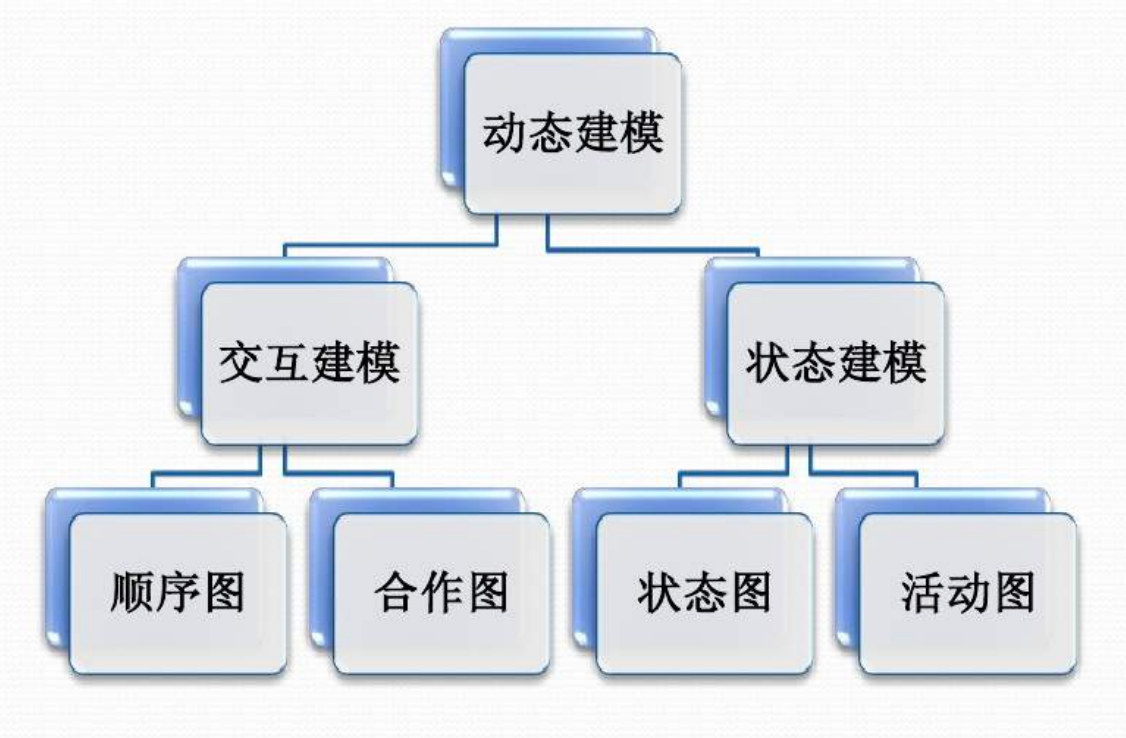
通信图

- 通信图实例分析



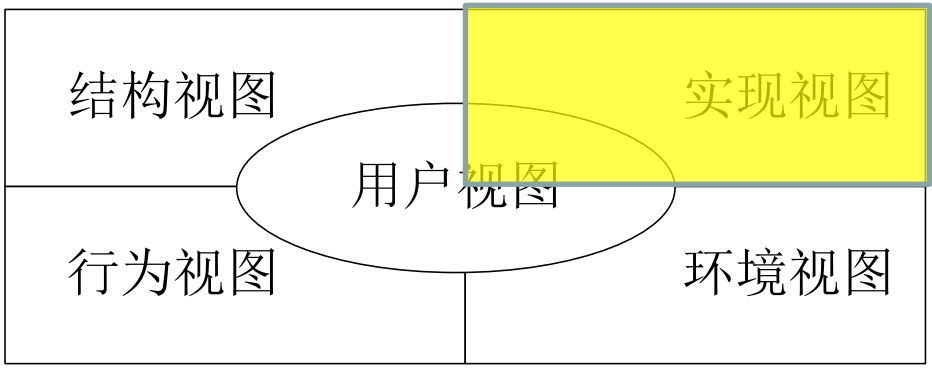
动态建模

- 总结



第五部分

UML架构建模



架构建模概述

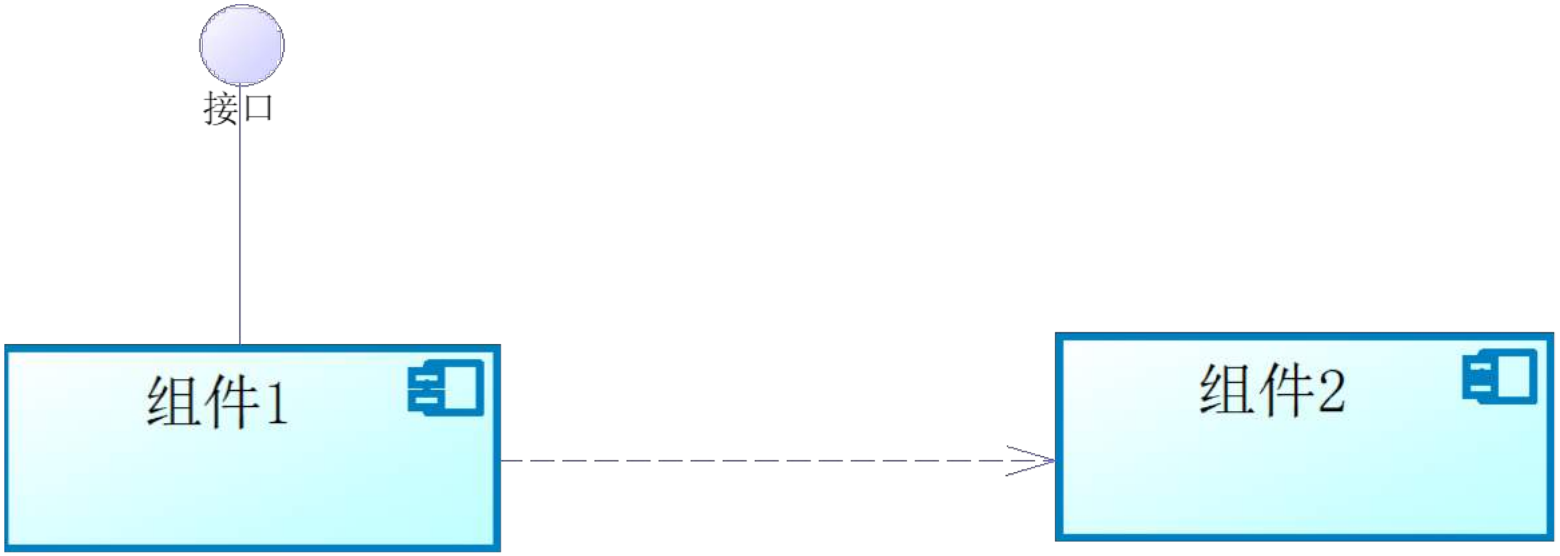
- UML架构模型
  - UML架构模型包括实现视图和部署视图，其中实现视图通过组件图来表示，部署视图通过部署图来表示。架构建模用于帮助开发人员设计系统的整体物理架构。
  - 架构建模在一个更高的层次描述一个应用系统的结构，包括构成系统的组件之间的关系，组件部署情况，以及硬件设备之间的关系。
  - 组件图用于描述每个功能所在的组件位置以及它们之间的关系；部署图用于描述软件中各个组件驻留的硬件位置以及这些硬件之间的交互关系。

组件图

- 定义
  - 组件图（Component Diagram)）包括组件、接口，以及各种关系。
  - 组件图显示组件以及它们之间的依赖关系，它可以用来显示程序代码如何分解成模块或组件。
  - 组件可以有以下几种类型：
    - 源代码组件：一个源代码文件或者与一个包对应的源代码文件
    - 二进制组件：一个目标码文件，一个静态的或者动态的库文件
    - 可执行组件：在一台处理器上可运行的一个可执行的程序单位，即所谓可执行程序

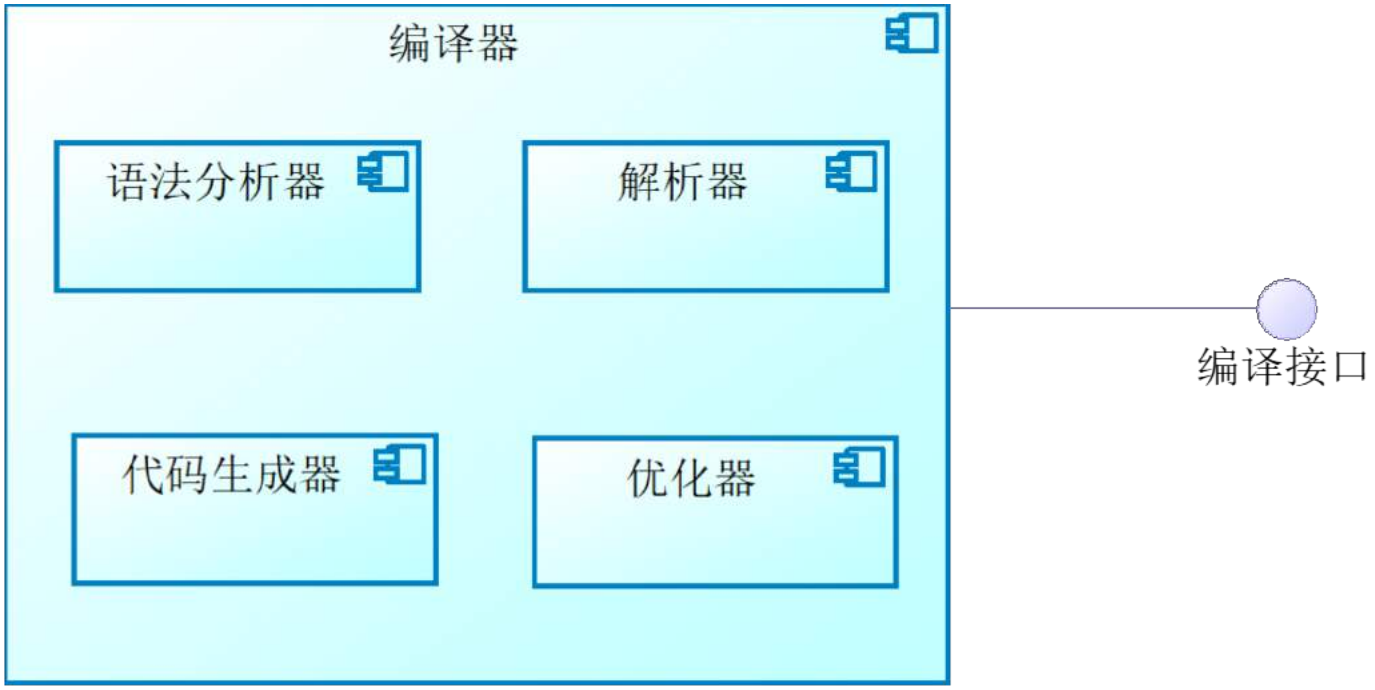
组件图

- 定义
  - 组件图可以用来显示编译、链接或执行时组件之间的依赖关系，以及组件的接口和调用关系。



组件图

- 组件图实例

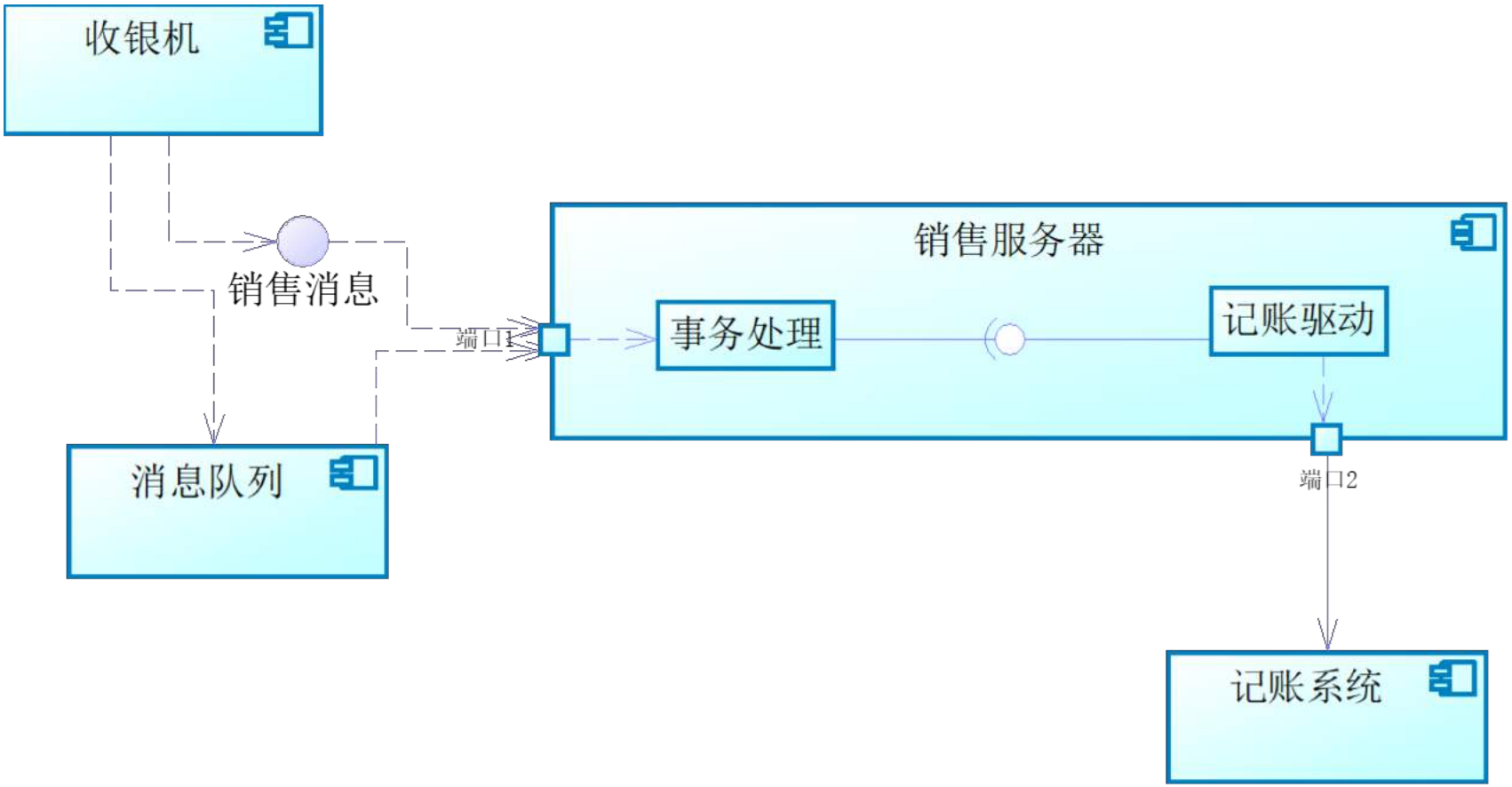


组件图

- 实例分析
  - 在某销售终端系统中，客户端收银机可以通过销售消息接口与销售服务器相连。考虑到网络可能不可靠，需要提供一个消息队列组件。在网络环境畅通时收银机直接与服务器相连；如果网络不可靠则与消息队列交互，当网络可用时队列再与服务器交互。服务器分解为两个主要组件，主要包括事务处理组件和记账驱动组件，记账驱动组件需要和记账系统交互。绘制系统组件图。

组件图

- 组件图实例分析



部署图

- 定义
  - 部署图（Deployment Diagram）和组件图一样，是面向对象系统的物理方面建模的两种图之一。组件图是说明组件之间的逻辑关系的，而部署图则是在此基础上更进一步，描述系统硬件的物理拓扑结构及在此结构上执行的软件。
  - 部署图可以显示计算节点的拓扑结构和通信路径、节点上运行的软件组件，常常用于帮助理解分布式系统。

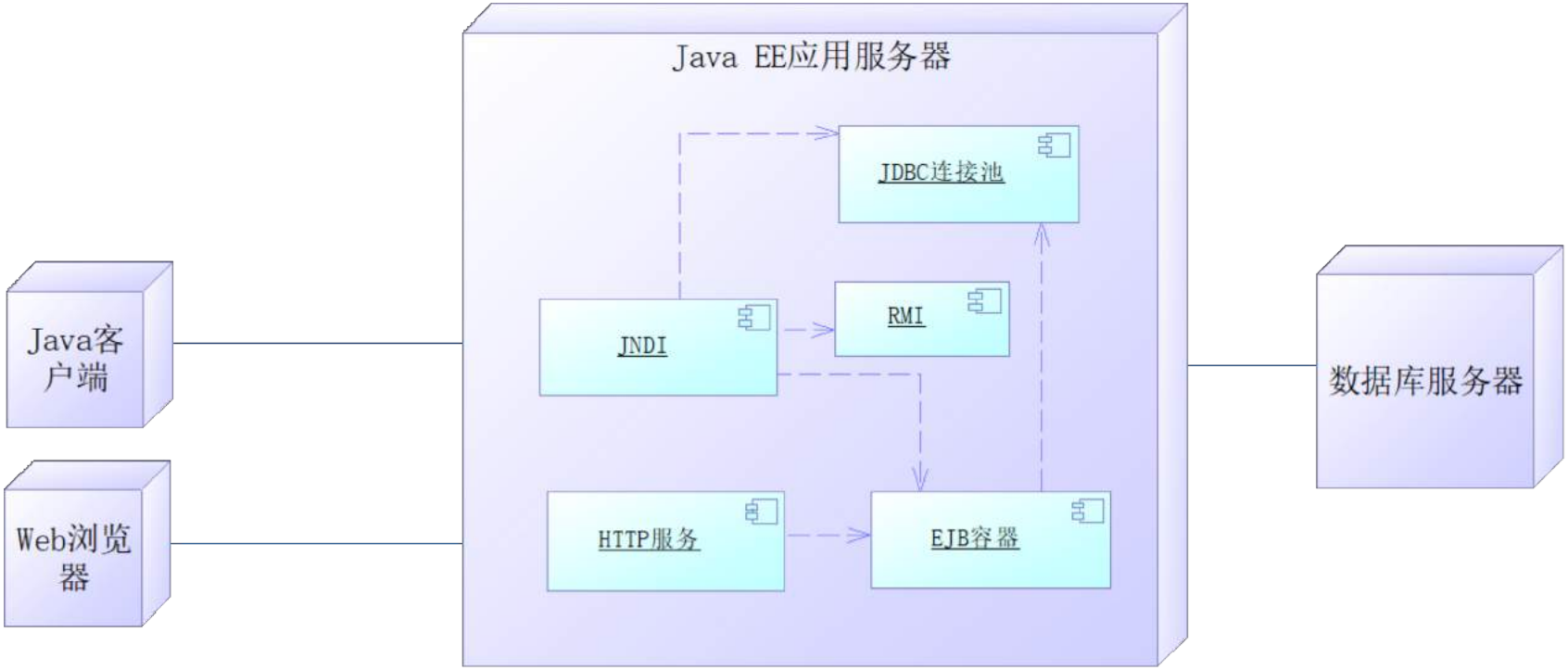
部署图

- 部署图定义
  - 在UML中，部署图显示了系统的硬件和安装在硬件上的软件，以及用于连接异构计算机之间的中间件。部署图通常被认为是一个网络图或者技术架构图。



部署图

- 实例

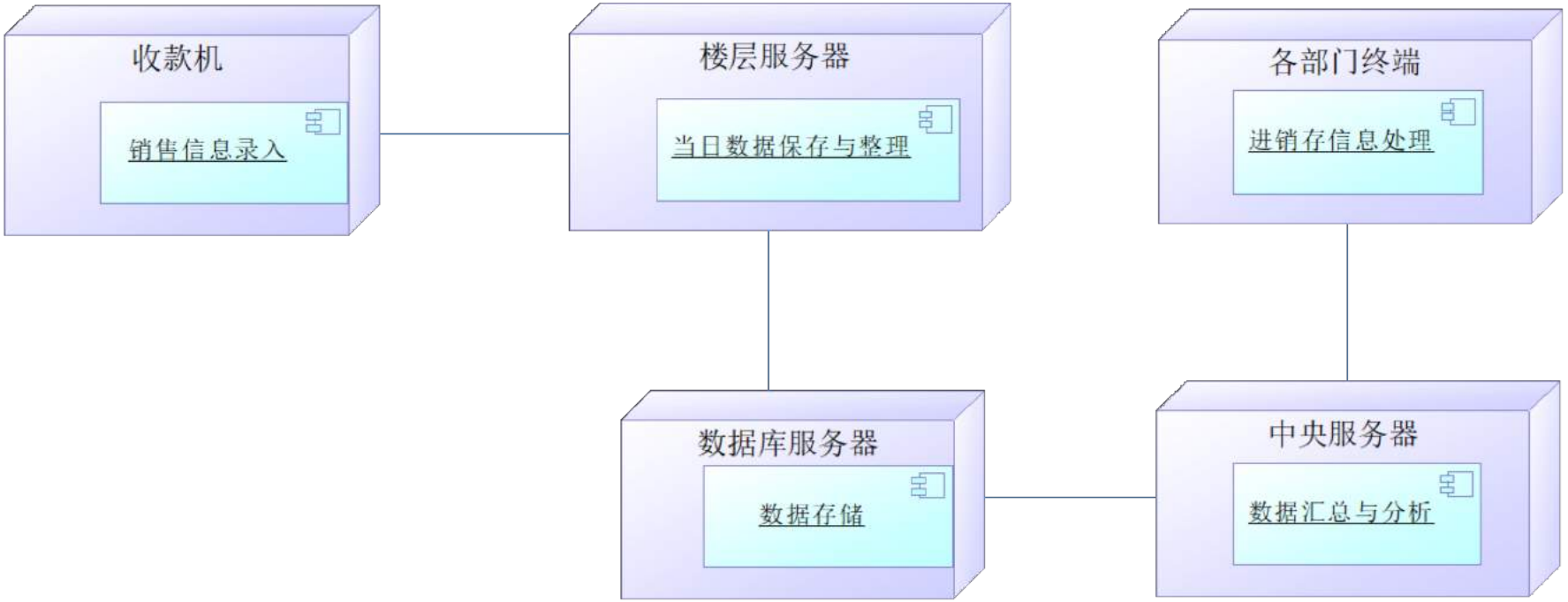

Java EE应用服务器
JDBC连接池
JNDI
RMI
HTTP服务
EJB容器
Java客户端
Web浏览器
数据库服务器

部署图

- 实例分析
  - 某大型商场的信息管理系统是由一个数据库服务器、中央服务器、每个楼层的楼层服务器、各柜台的收款机和各个部门的计算机终端组成的局域网络，它们分别负责商场数据存储、数据的汇总与分析、当日数据的保存与整理、销售信息录入和进销存信息处理等各种业务处理。用部署图描述该系统在不同硬件上的配置情况。

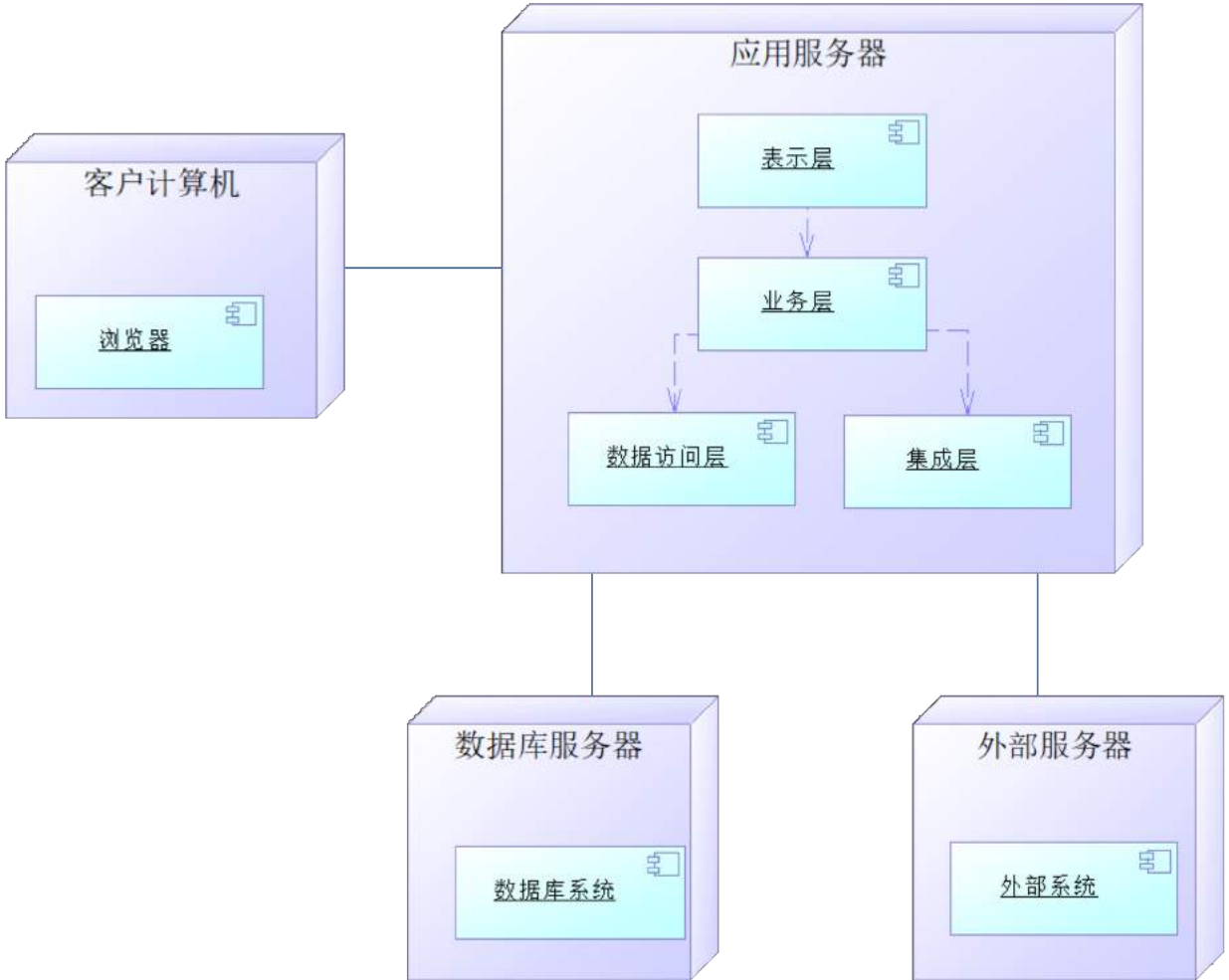
部署图

- 部署图实例分析


收款机
销售信息录入
楼层服务器
当日数据保存与整理
各部门终端
进销存信息处理
数据库服务器
数据存储
中央服务器
数据汇总与分析

部署图

- 实例分析：典型B/S系统部署图



第5章

统一建模语言

陈龙彪
longbiaochen@xmu.edu.cn

厦门大学信息学院
2019年3月